



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Sistema de gestão para um clube desportivo

Diogo Moreira(a106841) Jorge Barbosa(a106799) Nuno Mendes(a106875)
Rafael Silva(107289)

24 de junho de 2026

LI4

Data de Receção	
Responsável	
Avaliação	
Observações	

Sistema de gestão para um clube desportivo

Diogo Moreira(a106841) Jorge Barbosa(a106799) Nuno Mendes(a106875) Rafael Silva(107289)

24 de junho de 2026

<</opcional Dedicatória>>

Resumo

Este relatório apresenta a etapa atual do projeto da unidade curricular de Laboratórios de Informática IV, centrada na definição estruturada de um sistema de apoio à gestão de um clube desportivo eclético. A evolução do trabalho foi guiada por uma abordagem iterativa de engenharia de *software*, com reforço metodológico através da utilização de modelos de *Large Language Models* (LLM) para apoiar a redação técnica, clarificação de requisitos e validação de consistência documental. Nesta fase, a equipa consolidou o domínio do problema, identificando as necessidades administrativas, financeiras e operacionais que condicionam o funcionamento diário do clube. Foi dada especial prioridade à unificação de processos atualmente dispersos por ferramentas heterogéneas, com o objetivo de reduzir redundâncias, melhorar a rastreabilidade da informação e garantir maior coerência entre as dimensões administrativa e desportiva. Paralelamente, foram definidos os eixos principais da solução, estabelecendo-se uma visão de plataforma centralizada, orientada para a gestão de entidades, controlo de elegibilidade, acompanhamento de treinos e suporte à decisão técnica. Esta formulação inicial permite enquadrar de forma robusta as próximas etapas de arquitetura, implementação e avaliação, assegurando alinhamento entre objetivos académicos, requisitos funcionais e qualidade final do sistema produzido.

Área de Aplicação: Engenharia de *Software* Assistida por Inteligência Artificial e Sistemas de Gestão Desportiva.

Palavras-Chave: Engenharia de *Software*, Inteligência Artificial Generativa, LLM, Engenharia de Requisitos, Gestão Desportiva, Sistemas de Informação.

Índice

1. Conceção e Engenharia de Requisitos Assistida por LLM	1
1.1. Contextualização	1
1.2. Apresentação do Caso de Estudo	2
1.3. Âmbito e Restrições do Sistema	2
1.4. Motivação e Objetivos	3
1.5. Análise de Viabilidade do Processo	4
1.6. Recursos Humanos e Estrutura da Equipa	5
1.6.1. Recursos Humanos	5
1.6.2. Recursos Materiais	6
1.7. Plano de Execução do Projeto	6
1.7.1. Metodologia e Suporte Cognitivo	6
1.7.2. Coordenação e Planeamento Temporal	7
1.8. Estrutura do Relatório	9
1.9. Identificação e Mapeamento de <i>Stakeholders</i>	9
1.9.1. Direção Executiva	9
1.9.2. Direção Financeira	10
1.9.3. Secretaria e Administração	10
1.9.4. Direção Técnica da Formação	11
1.9.5. Equipa Técnica	11
1.9.6. Departamento Médico	12
1.9.7. Utilizadores Finais	12
1.10. Eliciação de Requisitos Assistida por LLM	12
1.10.1. Entrevistas Semiestruturadas (Abordagem Qualitativa)	13
1.10.2. Workshops de Alinhamento Operacional (Sessões Conjuntas)	13
1.10.3. Inquéritos por Questionário (Abordagem Quantitativa)	14
1.11. Modelação Operacional: <i>User Stories</i>	17
1.11.1. Validação e Aceitação de <i>User Stories</i>	19
1.12. Engenharia e Especificação de Requisitos	20
1.12.1. Abordagem Metodológica e Extração Assistida por LLM	20
1.12.2. Refinamento Iterativo e Resolução de Conflitos	21
1.12.3. Segregação Arquitetural (Funcional vs. Não-Funcional)	22
1.13. Modelação Conceptual do Domínio	23
1.13.1. Análise Detalhada por Sub-Domínios de Negócio	24
1.14. Especificação de Casos de Uso	28
1.14.1. Derivação e Rastreabilidade	28
1.14.2. Metodologia de Especificação Assistida por IA	29

1.14.3. Caso de Uso em Destaque: [Abrir ocorrência clínica]	30
1.14.4. Modelação Visual	32
2. Arquitetura e Design do Software utilizando LLM	34
2.1. Especificação de APIs do Sistema de Gestão para Clube Desportivo (SIGD)	34
2.1.1. Especificação de APIs	34
2.1.2. Lista de <i>Endpoints</i> Disponíveis por Módulo	35
2.1.3. Detalhe Estruturado dos Principais <i>Endpoints</i>	36
2.1.4. Contratos de Interface (API Contracts)	39
2.1.5. Justificação das Decisões Arquiteturais da API	40
2.2. Prototipagem	42
2.2.1. Design System e Estruturação Modular	42
2.2.2. Especificação Técnica e Documentação de Módulo	42
2.2.3. Execução Assistida por LLM	43
2.2.4. Demonstração da Interface de Alta Fidelidade e Validação de Requisitos	43
2.3. Arquitetura do Sistema e Decisões Técnicas	48
2.4. Modelação e Arquitetura do Sistema	51
2.4.1. Diagrama de Classes	51
2.4.2. Diagramas de Sequência	51
2.4.3. Diagrama de Componentes	52
3. Implementação e Desenvolvimento Assistido por LLM	53
3.1. Configuração do Ambiente de Trabalho	53
3.1.1. Stack Tecnológica e Justificação	53
3.1.2. Orquestração de Serviços com Docker	54
3.1.3. Automatização do Arranque — Script <code>start.ps1</code>	55
3.1.4. Estratégia de Configuração e Perfis	55
3.2. Estratégia de Desenvolvimento Modular	56
3.2.1. Decomposição do Sistema em Módulos Funcionais	56
3.2.2. Ordem de Implementação e Gestão de Dependências	56
3.2.3. Ciclo de Desenvolvimento por Módulo	57
3.3. Geração de Código Assistida por LLM	57
3.3.1. Papel do LLM no Processo de Implementação	57
3.3.2. Metodologia de Interação — Engenharia de Prompts	58
3.3.3. Padrões de Prompt por Tipo de Tarefa	59
3.3.4. Limitações Identificadas e Estratégias de Mitigação	59
3.4. Implementação do Backend	60
3.4.1. Estrutura de Pacotes e Arquitetura em Camadas	60
3.4.2. Gestão Evolutiva do Esquema com Flyway	60
3.4.3. Segurança — Autenticação, Autorização e Proteções Transversais	61
3.4.4. Lógica de Negócio — Decisões Técnicas por Módulo	62
3.4.5. Tratamento Global de Exceções e Padronização de Respostas	62
3.5. Implementação do Frontend	63
3.5.1. Estrutura de Ecrãs e Organização por Perfil	63
3.5.2. Integração com a API REST	63

3.5.3.	Desafios de Compatibilidade e Resolução	64
3.5.4.	Decisões de UX com Impacto Técnico	64
3.6.	Gestão da Base de Dados e Seed de Demonstração	65
3.6.1.	Modelo Relacional Final	65
3.6.2.	Seed Master Demo — Objetivos e Construção	66
3.6.3.	O <i>Seed</i> como “Dono da Verdade”	67
4.	Verificação, Validação e Avaliação da Qualidade do Software Produzido	68
4.1.	Estratégia de Verificação e Validação	68
4.1.1.	Distinção entre Verificação e Validação	68
4.1.2.	Pirâmide de Testes Adoptada	69
4.1.3.	Ferramentas e Infraestrutura de Testes	69
4.1.4.	Papel do LLM na Estratégia de Qualidade	70
4.2.	Geração de Testes Assistida por LLM	70
4.2.1.	Metodologia de Geração	70
4.2.2.	Cobertura por Categorias de Teste	71
4.2.3.	Deteção de Inconsistências entre Implementação e SRS	71
4.3.	Testes Unitários	72
4.3.1.	Âmbito e Organização	72
4.3.2.	Padrões de Implementação	72
4.3.3.	Cenários Cobertos por Módulo	73
4.3.4.	Casos de Fronteira e Testes Negativos	74
4.3.5.	Resultados Finais	75
4.4.	Testes de Integração	75
4.4.1.	Âmbito e Configuração	75
4.4.2.	Padrões de Implementação	76
4.4.3.	Cobertura de Segurança e Contratos de API	77
4.4.4.	Resultados Finais	78
	Conclusões e Trabalho Futuro	79
	Referências Bibliográficas	81
	Lista de Siglas e Acrónimos	82
	Anexos	84

Lista de Figuras

1.1. Diagrama de Gantt com planeamento e atribuição de tarefas nas 4 etapas do projeto.	8
1.2. Principais frustrações identificadas no atendimento presencial na Secretaria. . .	16
1.3. Impacto da caducidade imprevista de Exames Médico-Desportivos nos atletas. .	16
1.4. Áreas de intervenção consideradas prioritárias para a modernização digital dos serviços.	17
1.5. Micro-Contexto de Utilizadores, Sessões e Rastreabilidade	25
1.6. Micro-Contexto de Configuração Desportiva e Escalões	26
1.7. Micro-Contexto de Execução de Treinos, Chamadas e Notas	27
1.8. Micro-Contexto de Liquidação Financeira e Emissão Fiscal	28
1.9. Diagrama de Casos de Uso UML — Módulo D (Equipa Técnica).	33
2.1. Protótipo de Alta Fidelidade: Painel de Visão Executiva	44
2.2. Protótipo de Alta Fidelidade: Interface de Validação Documental	45
2.3. Protótipo Móvel: Registo de Chamada e Estado Clínico	46
2.4. Protótipo Móvel: Lançamento de Avaliação Quantitativa	47
2.5. Diagrama de componentes planeado para o SIGD, ilustrando o encapsulamento modular estipulado e a futura topologia de integração e persistência.	52

Lista de Tabelas

1.1. Especificação da <i>User Story</i> US10 - Segregação de Fluxos por Centros de Responsabilidade (Clube vs. Academia).	18
1.2. Especificação da <i>User Story</i> US12 - Proteção de Dados Sensíveis e Sigilo Clínico (RGPD).	19
1.3. Matriz de validação e aceitação formal das <i>User Stories</i>	20
1.4. Caso de Uso UC-12.1 — Abrir ocorrência clínica	30
2.1. <i>Endpoints</i> principais do ecossistema SIGD.	36
2.2. Especificação do ADR-01.	48
2.3. Especificação do ADR-02.	48
2.4. Especificação do ADR-03.	49
2.5. Especificação do ADR-04.	49
2.6. Especificação do ADR-05.	50
2.7. Especificação do ADR-06.	50
2.8. Especificação do ADR-07.	50
3.1. Componentes principais da <i>stack</i> tecnológica adotada no SIGD.	54
3.2. Mapeamento dos módulos funcionais e domínios de negócio do SIGD.	56
3.3. Mapeamento da evolução do esquema de base de dados através das migrações Flyway.	61
3.4. Mapeamento global de exceções do sistema para códigos de estado HTTP.	63
3.5. Estrutura lógica e relacional das entidades do domínio do SIGD.	66
3.6. Estrutura e volumetria do ficheiro de dados de demonstração (<i>Seed Master Demo</i>).	67
4.1. Ferramentas e versões utilizadas na infraestrutura de testes do SIGD.	69
4.2. Cenários de teste unitário cobertos por módulo funcional.	74
4.3. Evolução dos resultados dos testes unitários ao longo do desenvolvimento.	75
4.4. Evolução dos resultados dos testes de integração ao longo do desenvolvimento.	78
II.1. Resultados quantitativos do inquérito de levantamento de necessidades (258 respostas).	89

Capítulo 1 - Conceção e Engenharia de Requisitos Assistida por LLM

1.1. Contextualização

Fundado em 1903, o Boavista Futebol Clube (BFC) é uma das instituições desportivas mais históricas e tituladas de Portugal. Contudo, após décadas de glória que culminaram na conquista do Campeonato Nacional, o clube foi recentemente atingido por uma crise financeira e administrativa sem precedentes. Este colapso resultou na insolvência da sua Sociedade Anónima Desportiva (SAD) e na consequente despromoção da equipa de futebol aos escalões distritais não-profissionais, forçando a instituição a lutar em tribunal para evitar o fecho definitivo de portas e a liquidação do seu património.

Apesar do cenário adverso, a alma do clube sobreviveu através do seu forte ecletismo e das suas raízes bairristas. A atual direção de emergência foi forçada a iniciar um processo de reestruturação total a partir do zero, focando os seus poucos recursos na manutenção das modalidades amadoras e no desenvolvimento dos escalões de formação, que albergam atualmente milhares de jovens atletas.

Para que esta reestruturação seja bem-sucedida, a sobrevivência diária do clube passou a depender de uma gestão financeira rigorosa ao cêntimo — alicerçada em grande parte na cobrança eficiente de quotas dos Encarregados de Educação — e de uma extrema agilidade logística. Face à ausência de capital para adquirir soluções de *software* empresariais de alto custo, a direção do BFC contratou a Omnisystema, uma jovem empresa de desenvolvimento de *software* constituída por alunos de Engenharia Informática da Universidade do Minho. A missão desta equipa é conceber e desenvolver, de raiz, um Sistema de Informação e Gestão Desportiva de baixo custo, assistido por Inteligência Artificial, capaz de modernizar as operações e apoiar o renascimento da instituição.

1.2. Apresentação do Caso de Estudo

O caso de estudo selecionado incide sobre a modernização e unificação da infraestrutura operacional do BFC. Atualmente, a gestão diária do clube — abrangendo as esferas desportiva, administrativa e financeira — é suportada por um ecossistema tecnológico altamente fragmentado e, em grande medida, analógico.

A fundamentação para a intervenção da Omnisystema baseia-se na necessidade de resolver ineficiências sistémicas críticas. Constata-se o uso predominante de registos em suporte de papel para o controlo de assiduidade, ocorrências de treino e avaliações desportivas. Este método resulta não apenas no moroso retrabalho de transcrição de dados, mas também na frequente perda e degradação física de documentos devido às condições meteorológicas no terreno. Simultaneamente, o controlo financeiro (inscrições e pagamento de quotas) é efetuado através de folhas de cálculo dispersas na secretaria, sem qualquer interoperabilidade com o departamento desportivo. Adicionalmente, a comunicação com os atletas e encarregados de educação (como a divulgação de convocatórias) ocorre em redes informais de mensagens instantâneas, gerando ruído comunicacional e assimetria de informação.

A conceção e adoção de um Sistema Integrado de Gestão Desportiva justifica-se pela urgência em garantir a centralização, integridade, segurança e escalabilidade da informação. Ao invés do atual isolamento de dados, uma plataforma unificada permitirá estruturar as informações de forma coerente e relacional, eliminando redundâncias e falhas de comunicação. A capacidade de cruzar dados de forma avançada — correlacionando, por exemplo, a situação financeira (quotas) de um associado com a elegibilidade desportiva do respetivo educando — fornecerá à direção e à equipa técnica uma visão global e em tempo real das operações do clube, apoiando ativamente a tomada de decisões estratégicas essenciais à sobrevivência e reestruturação da instituição.

1.3. Âmbito e Restrições do Sistema

Para garantir a exequibilidade do projeto, manter um elevado rigor metodológico e evitar o desvio de escopo, a equipa procedeu à delimitação estrita do domínio do sistema em estreita articulação com a direção e os responsáveis operacionais do clube. A filosofia de desenvolvimento adotada privilegia a qualidade arquitetural, a fundamentação técnica e a robustez das funcionalidades centrais, em detrimento de uma expansão horizontal excessiva de módulos operacionais que comprometeria a viabilidade temporal do projeto.

Neste sentido, o SIGD centraliza-se no registo transversal de associados, no apoio ao departamento médico e no controlo base das equipas técnicas. Ficam explicitamente excluídos do âmbito do projeto a diferenciação estrutural entre modalidades desportivas — o sistema assumirá uma arquitetura de dados uniforme e agnóstica em relação ao desporto praticado —, a gestão complexa de logística (como a alocação dinâmica de recintos ou terrenos de jogo) e a

gestão de patrocínios.

Adicionalmente, o sistema não integrará portais de pagamento reais nem atuará como software de contabilidade certificada. O processamento financeiro e as respectivas estatísticas analíticas serão validados através de dados simulados ou de simples inserção manual, focando-se a engenharia na lógica de estruturação e segurança dessa informação, e não na complexidade da transação bancária em si.

O desenho do software encontra-se ainda balizado por restrições operacionais e legais rigorosas. A nível de infraestrutura, a arquitetura de alojamento está restrita a um ambiente de servidor local para contenção de custos, assumindo-se que a interação móvel será suportada pelos dispositivos pessoais da equipa técnica. Do ponto de vista legal, o tratamento de dados clínicos e de informações relativas a atletas menores de idade nos escalões de formação obriga a que a modelação da base de dados e os perfis de acesso cumpram estritamente as diretrizes de privacidade e o Regulamento Geral sobre a Proteção de Dados (RGPD).

Por fim, a execução da plataforma obedece a uma restrição temporal inegociável, definida pelo prazo contratual do projeto, o que exige a implementação, testagem e documentação integral da solução até ao final do mês de maio de 2026.

1.4. Motivação e Objetivos

O colapso administrativo do BFC e a consequente necessidade de reestruturação evidenciaram a insustentabilidade das práticas de gestão vigentes. Com o crescimento do esforço de recuperação do clube e o aumento do volume de dados a processar diariamente, as ferramentas tradicionais (suportes em papel e folhas de cálculo não interligadas) provaram-se manifestamente inviáveis.

A motivação primordial para o desenvolvimento deste projeto nasce da urgência em resolver a severa sobrecarga cognitiva e operacional que afeta os diversos departamentos da instituição. Atualmente, a dependência de bases de dados isoladas obriga os funcionários da secretaria a um constante cruzamento manual de informação, gerando inconsistências que degradam a imagem do clube. A nível financeiro, a ausência de um mecanismo automatizado que separe o fluxo de receitas gera estrangulamentos de tesouraria. No plano desportivo, a falta de ferramentas móveis adequadas consome tempo útil de treino aos técnicos e provoca conflitos logísticos na partilha de infraestruturas. Por fim, do lado do cliente final (o associado), a obrigatoriedade de deslocações físicas para operações de rotina cria uma fricção desnecessária que desincentiva a regularização de quotas. Face a este cenário de rutura eminente, a Omnisistema foi mobilizada para desenhar uma solução tecnológica que trave a perda de dados e rentabilize os recursos humanos e financeiros do clube.

Para dar resposta a estas necessidades, a conceção do novo SIGD tem como propósito central a otimização dos processos internos e a melhoria substancial da experiência dos utilizadores. Com a implementação deste sistema, a equipa propõe-se a alcançar os seguintes objetivos

operacionais:

- **Centralização e Integridade da Informação:** Assegurar a convergência de todos os dados referentes aos associados e atletas num repositório único e relacional. O objetivo passa por eliminar a redundância de processos administrativos, garantir a fiabilidade dos dados em tempo real e otimizar substancialmente os tempos de resposta nos serviços de atendimento ao público.
- **Otimização e Transparência Financeira:** Automatizar o rastreio e a categorização dos fluxos de receita, permitindo uma separação lógica, ágil e auditável dos proveitos do clube. Pretende-se mitigar o risco de erro humano inerente aos controlos analógicos e dotar a estrutura dirigente de métricas exatas para apoio à decisão financeira.
- **Modernização da Gestão Operacional Desportiva:** Digitalizar o acompanhamento diário das equipas, desde o controlo de assiduidade à gestão da ocupação das infraestruturas físicas. O foco reside em substituir os métodos vulneráveis em papel por processos ágeis, libertando o corpo técnico de sobrecargas burocráticas para que este se concentre no planeamento desportivo.
- **Maximização da Acessibilidade e Autonomia:** Desmaterializar o acesso aos serviços do clube, reduzindo drasticamente a necessidade de deslocações físicas para a resolução de trâmites administrativos rotineiros. O propósito é promover a autonomia do associado, mitigar o atrito no cumprimento de obrigações (quotas) e modernizar a sua experiência de interação com a instituição.

1.5. Análise de Viabilidade do Processo

A implementação do novo SIGD pela Omnisystema apresenta um potencial de retorno financeiro e operacional altamente quantificável para o BFC. Através da análise do volume de Encarregados de Educação, do fluxo de tesouraria e do esforço despendido pelos vários departamentos, a direção do clube estabeleceu as seguintes projeções de viabilidade a médio prazo:

- Aumentar a receita efetiva de quotas em cerca de 25%, impulsionada pela redução drástica do atrito no processo de cobrança, possibilitada pela adoção de pagamentos *online* no novo Portal do Encarregado de Educação e pela emissão de alertas automáticos de regularização.
- Aumentar a produtividade administrativa em 30%, fruto da eliminação do retrabalho (dupla inserção de dados) e da supressão da triagem manual de receitas entre as contas do Clube e da SAD.
- Reduzir em cerca de 20% os custos logísticos e de consumíveis recorrentes, nomeadamente através da transição integral para formatos digitais e da substituição da emissão de cartões de plástico por credenciais virtuais (*QR Codes*).
- Mitigar na totalidade (100%) o risco de coimas associativas, assegurando a validação automática da elegibilidade desportiva (exames médicos válidos, quotas em dia e sanções disciplinares) no momento exato da geração de cada convocatória.

Face a estas métricas, conclui-se que o projeto é criticamente viável e rentável. A equação financeira é ainda maximizada pelo baixo custo inicial de desenvolvimento da solução, uma vez que a instituição recorreu aos serviços de consultoria da Omnisystema — uma equipa de engenheiros informáticos em formação —, garantindo assim um Retorno sobre o Investimento (ROI) extremamente célere e alinhado com o atual rigor orçamental exigido ao clube.

1.6. Recursos Humanos e Estrutura da Equipa

1.6.1. Recursos Humanos

A estrutura de recursos humanos integra três grupos complementares. A equipa externa corresponde à OmniSystema e assegura a execução técnica do projeto, repartindo responsabilidades de desenvolvimento *full-stack*, engenharia de *prompts*, análise de requisitos e validação arquitetural do *software*. A composição da equipa é a seguinte:

- Diogo Moreira
- Jorge Barbosa
- Nuno Mendes
- Rafael Silva

Em articulação com a equipa de desenvolvimento, os *stakeholders* internos do BFC garantem o enquadramento operacional e estratégico necessário à definição e validação dos requisitos. Os perfis funcionais considerados são os seguintes:

- Jorge Silva (CEO): atua como *Sponsor* do projeto e principal decisor estratégico, supervisionando a viabilidade global e o alinhamento das operações com os objetivos de reestruturação do clube.
- Henrique Leal (CFO): é responsável pela tesouraria, assegurando o rastreamento, a categorização e a separação lógica das receitas entre o Clube e a SAD, recorrendo a fluxos de dados simulados no sistema.
- Carla Mendes (Gestora da Secretaria): representa as operações de *front-office*, sendo o principal ponto de contacto presencial para a gestão de inscrições, atualização de dados cadastrais e marcação de quotas regularizadas.
- Armando Silva (Diretor Técnico da Formação): coordena a operação desportiva a nível tático, focando-se na supervisão transversal das equipas, na padronização da avaliação técnica dos atletas e na consulta de estatísticas agregadas.
- Miguel Santos (Treinador Principal): representa a perspetiva operacional do relvado, gerindo o plantel, o registo diário de assiduidade e a geração de convocatórias com validação automática de elegibilidade.
- Nuno Ramos (Chefe do Departamento Médico): assegura a conformidade clínica da instituição, monitorizando a validade legal dos exames médico-desportivos e gerindo os estados de aptidão (semáforo clínico) de cada atleta.

A componente de utilização final é assegurada por Encarregados de Educação, atletas e encarregados de educação, que interagem regularmente com os serviços do clube e condicionam o sucesso da adoção do sistema através do seu nível de adesão, autonomia e regularidade no cumprimento de quotas e inscrições.

1.6.2. Recursos Materiais

A infraestrutura de *hardware* assenta em quatro computadores pessoais, alocados individualmente aos elementos da OmniSystema para suportar as atividades de análise, modelação, implementação e validação do sistema. O ambiente de *deployment* será assegurado por servidor local, num contexto *on-premise*, garantindo controlo direto da equipa sobre a configuração técnica e o ciclo de disponibilização da solução.

Complementarmente, a camada de *software* foi definida para assegurar produtividade técnica, rastreabilidade de projeto e consistência arquitetural ao longo de todo o desenvolvimento. Neste enquadramento, adotou-se o seguinte conjunto de ferramentas e tecnologias:

- Gemini Pro: suporte à eliciação de requisitos, simulação de *stakeholders* e apoio à geração de código.
- Prism: assistente de *prompts* e plataforma de edição colaborativa em tempo real para $\text{\LaTeX}$.
- GitHub: controlo de versões do código-fonte e gestão do histórico de alterações.
- GitHub Projects: gestão ágil do *backlog*, planeamento e acompanhamento das tarefas da equipa.
- Microsoft Excel: elaboração das atas de reunião e do Diagrama de Gantt para planeamento temporal.
- PlantUML: geração de diagramas UML através de código estruturado.
- React Native + Expo: *framework* de *frontend* para a aplicação multiplataforma, com base em JavaScript/TypeScript.
- Java Spring Boot: *framework* de *backend* para implementação da API.
- MySQL: sistema de gestão de base de dados relacional para persistência da informação.
- Visual Studio Code: ambiente de desenvolvimento integrado para implementação e manutenção do código.

1.7. Plano de Execução do Projeto

1.7.1. Metodologia e Suporte Cognitivo

Para o desenvolvimento do SIGD do BFC, a OmniSystema adotou uma abordagem de *Cascata Enriquecida com IA*, conciliando previsibilidade de execução com reforço técnico na produção de artefactos. Esta opção decorre da natureza sequencial do projeto, em que cada etapa exige validação formal dos resultados anteriores, nomeadamente a consolidação da especificação de

requisitos antes da evolução para arquitetura, implementação e validação.

Neste enquadramento, o modelo em cascata assegura disciplina de planeamento, controlo de entregáveis e cumprimento dos marcos académicos, enquanto os LLMs funcionam como suporte cognitivo transversal às tarefas de análise, redação técnica, exploração de alternativas e apoio à codificação. A utilização destes modelos segue um protocolo operacional exigente: a equipa define *prompts* contextualizados com regras de negócio e restrições do domínio, mantém sempre revisão *human-in-the-loop* antes de integrar qualquer resultado no projeto e aplica práticas sistemáticas de mitigação de alucinações através do confronto com normas técnicas de referência, incluindo IEEE 830/29148 e ISO/IEC 25010.

1.7.2. Coordenação e Planeamento Temporal

A coordenação de atividades, a distribuição de responsabilidades e a rastreabilidade da execução são asseguradas através do ecossistema *GitHub*, com recurso ao *GitHub Projects* para gestão do fluxo de trabalho da equipa e monitorização contínua do progresso. Este modelo de governação permite manter alinhamento operacional entre intervenientes internos e externos, reduzindo ambiguidades na priorização e no acompanhamento de tarefas. O quadro de acompanhamento e o código-fonte podem ser consultados no repositório oficial do projeto em: <https://tinyurl.com/4hh6dppx>.

Em paralelo, o planeamento temporal foi estruturado para acomodar as quatro macro-etapas do ciclo de desenvolvimento, prevendo janelas controladas de *paralelismo* sempre que tal melhora o aproveitamento do calendário sem comprometer dependências críticas. O Diagrama de Gantt (Figura 1.1) materializa esta estratégia, evidenciando a sequência das atividades, os pontos de interdependência e a alocação de esforço ao longo das semanas de execução até à fase de entrega.

Sistema Integrado de Gestão Desportiva

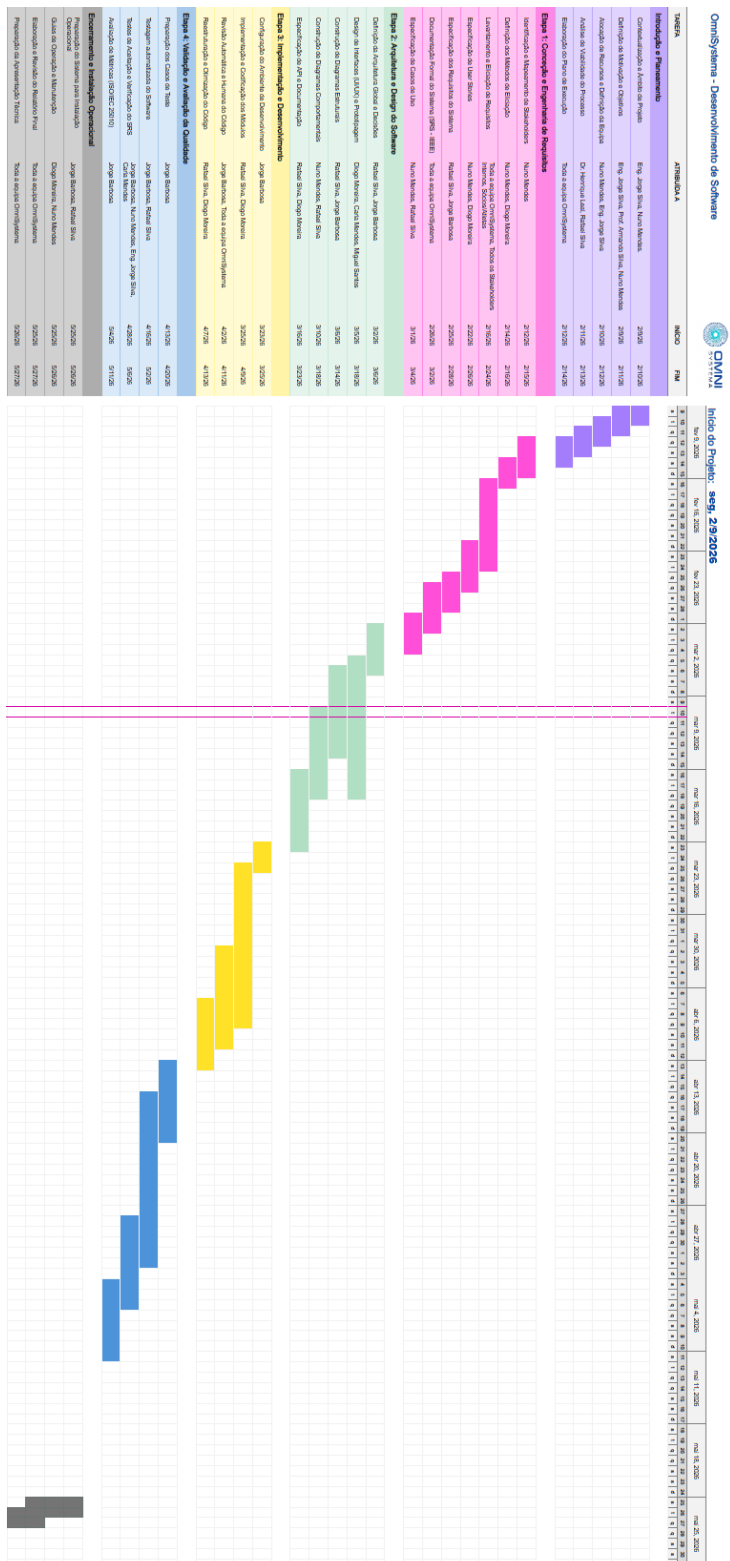


Figura 1.1.: Diagrama de Gantt com planeamento e atribuição de tarefas nas 4 etapas do projeto.

1.8. Estrutura do Relatório

Após o enquadramento inicial apresentado na primeira parte deste capítulo — onde se estabeleceram o contexto organizacional, os objetivos do projeto e o respetivo plano de execução —, a secção remanescente aprofunda a Engenharia de Requisitos assistida por LLM. Esta fase foca-se na identificação e caracterização dos *stakeholders*, na simulação de entrevistas para eliciação de necessidades, na formulação de *User Stories* e na modelação de Casos de Uso, culminando na consolidação do documento SRS como base formal de especificação.

O Capítulo 2 centra-se na arquitetura e no *design* do *software*. Nesta etapa, detalham-se as decisões estruturantes da solução, o *design* de interfaces, a especificação da API e os diagramas UML de natureza estrutural e comportamental que sustentam a coerência técnica do sistema.

No Capítulo 3, o foco recai sobre a implementação e o desenvolvimento. O texto descreve a configuração do ambiente de trabalho, a estratégia de codificação modular e os mecanismos de revisão humana e automática do código, incluindo atividades de *refactoring* orientadas à qualidade e à manutenção evolutiva da base de código.

Por fim, o Capítulo 4 é dedicado à verificação, validação e avaliação da qualidade do produto gerado. Este capítulo cobre as estratégias de testagem (unitária, de integração e de sistema), a análise de cobertura de código e a avaliação técnica de métricas, em estrita conformidade com a norma ISO/IEC 25010.

1.9. Identificação e Mapeamento de *Stakeholders*

Dada a natureza histórica e a atual exigência de reestruturação do BFC, a gestão diária da instituição exige uma articulação constante entre a vertente administrativa, financeira e a realidade desportiva operacional. O sistema a desenvolver terá de servir múltiplos perfis de utilizadores, cujas necessidades, níveis de literacia digital e contextos de utilização são drasticamente diferentes.

Por conseguinte, antes de se proceder à fase de eliciação de requisitos, torna-se imperativo mapear os principais atores que irão interagir com a plataforma. A correta identificação e categorização destes utilizadores garante que o levantamento de necessidades abrange toda a cadeia de valor do clube, desde o planeamento estratégico e controlo financeiro, passando pela gestão diária no terreno, até à experiência final do associado.

1.9.1. Direção Executiva

A Direção Executiva é representada pelo Eng. Jorge Silva, atual CEO do BFC. A sua rotina centra-se no planeamento estratégico e na supervisão transversal dos departamentos des-

portivo, financeiro e clínico, assumindo a liderança executiva do processo de reestruturação institucional.

Enquanto *Sponsor* do projeto, a sua inclusão no levantamento de requisitos é essencial para definir as prioridades arquiteturais do sistema. Ao contrário dos perfis operacionais, a sua interação com a plataforma não passa pela inserção massiva de dados, mas sim pela sua leitura e consolidação. A sua perspetiva dita os requisitos analíticos de topo, exigindo que o *software* disponibilize painéis de indicadores (*dashboards*) com métricas globais e em tempo real — nomeadamente a taxa de regularização de quotas e o volume de atletas inscritos. O mapeamento deste perfil garante que a solução final não é apenas um repositório de dados, mas sim uma ferramenta capaz de produzir informação consolidada para apoiar a tomada de decisão estratégica do clube.

1.9.2. Direção Financeira

A Direção Financeira é representada pelo Dr. Henrique Leal, atual CFO do BFC. A sua função centraliza-se na gestão da tesouraria, com especial incidência no controlo de receitas correntes, nomeadamente a cobrança de quotas de associados e mensalidades da formação. Adicionalmente, assume a responsabilidade de assegurar a segregação lógica e a rastreabilidade dos fundos entre as contas do Clube e da SAD.

A auscultação deste *stakeholder* revela-se determinante para estruturar um modelo de dados financeiro consistente. Embora o sistema não se proponha a atuar como *software* de contabilidade certificada, a perspetiva do CFO dita os requisitos operacionais para a categorização automática de pagamentos, a configuração de planos de quotização e a extração de relatórios de fluxos de caixa. A integração destas regras de negócio na plataforma permite eliminar a dependência de folhas de cálculo fragmentadas e reduzir a suscetibilidade a erro humano, dotando a direção de mecanismos fiáveis de auditoria e controlo orçamental.

1.9.3. Secretaria e Administração

A componente administrativa e de atendimento ao público é assegurada por Carla Mendes, Gestora da Secretaria do BFC. Atuando como o principal ponto de contacto presencial para associados, atletas e encarregados de educação, as suas responsabilidades diárias englobam a gestão de inscrições, a atualização de dados cadastrais e o processamento de quotas e mensalidades desportivas.

A auscultação deste perfil é fundamental para garantir a adoção orgânica do sistema. Ao contrário da direção executiva, que atua como consumidora de indicadores consolidados, a secretaria representa a principal fonte de introdução e validação primária de dados. Consequentemente, a sua perspetiva é essencial para orientar o desenho de uma *interface* ergonómica. O mapeamento das suas necessidades assegura que a futura plataforma automatize o

cruzamento de informação — como a verificação instantânea da regularidade financeira de um Encarregado de Educação —, focando-se na minimização do atrito operacional, na redução de interações manuais e na eliminação do retrabalho na transcrição de dados.

1.9.4. Direção Técnica da Formação

A coordenação estratégica e operacional do futebol jovem é assegurada pelo Prof. Armando Silva, Diretor Técnico da Formação do BFC. Atuando como elo de ligação entre a visão desportiva da direção e a execução no relvado, a sua função foca-se na gestão transversal dos escalões de base e na supervisão das metodologias de treino. Em paralelo, assume a responsabilidade pela avaliação contínua e integrada do rendimento desportivo das equipas e do respetivo corpo técnico.

A auscultação deste perfil é determinante para modelar o domínio desportivo do sistema de forma escalável. Ao contrário de um treinador principal, cuja interação com o *software* se centra no microciclo da sua própria equipa, o Diretor Técnico requer uma visão macroscópica sobre toda a operação. A sua perspetiva é indispensável para definir a agregação e consolidação de dados operacionais, nomeadamente através da geração de relatórios transversais de assiduidade, evolução técnica dos atletas e monitorização do estado clínico global. O mapeamento destas necessidades garante que a plataforma transcende o mero armazenamento isolado de plantéis, afirmando-se como uma ferramenta de apoio à decisão e de harmonização da estrutura formativa.

1.9.5. Equipa Técnica

A condução desportiva no terreno é representada por Miguel Santos, Treinador Principal do escalão de Sub-15. As suas responsabilidades centrais abrangem o planeamento dos microciclos de treino, a avaliação técnica contínua dos atletas e a elaboração das convocatórias semanais. Adicionalmente, assume o papel de principal ponto de contacto operacional com os encarregados de educação para o acompanhamento da assiduidade e da rotina competitiva.

A integração deste perfil é fundamental para introduzir o contexto de mobilidade e a dinâmica operacional de campo na conceção do sistema. Ao contrário das funções administrativas de escritório, a perspetiva do treinador assegura que a futura plataforma seja pensada para cenários de utilização no relvado, onde a rapidez e a facilidade de interação são primordiais. O seu contributo é determinante para modelar o fluxo de recolha de dados de assiduidade e os critérios de seleção desportiva, garantindo que o desenho da solução considere a necessidade de validar a elegibilidade dos atletas em tempo real. Desta forma, assegura-se que a plataforma atue como um facilitador tecnológico adaptado ao ambiente de treino, superando as limitações de um sistema de gestão puramente estático.

1.9.6. Departamento Médico

A conformidade clínica e a integridade física dos atletas são asseguradas pelo Dr. Nuno Ramos, Chefe do Departamento Médico. As suas funções centram-se na monitorização do estado de aptidão, na gestão do histórico de lesões e, com particular criticidade, no controlo rigoroso da validade dos exames médico-desportivos (EMD), cuja caducidade impede legalmente a participação desportiva do atleta.

A integração deste perfil é fundamental para garantir o enquadramento clínico e a segurança jurídica da instituição no novo sistema. A perspetiva do Chefe do Departamento Médico assegura que o domínio da saúde dos atletas seja modelado com o rigor necessário, permitindo que a futura plataforma considere os estados de aptidão física e a validade legal dos exames obrigatórios. Adicionalmente, dada a natureza sensível da informação processada — que envolve dados de saúde de menores de idade —, a sua participação é determinante para orientar as políticas de privacidade e as hierarquias de acesso aos dados, garantindo que o desenho da solução esteja alinhado com as diretrizes de proteção de dados (RGPD) e com a ética médica desportiva.

1.9.7. Utilizadores Finais

Os utilizadores finais do sistema englobam a massa associativa, os atletas da formação e os respetivos encarregados de educação. Este grupo heterogéneo interage com a instituição a múltiplos níveis, abrangendo o cumprimento de obrigações financeiras (pagamento de quotas e mensalidades) e o acompanhamento logístico da atividade desportiva (consulta de horários e convocatórias).

A auscultação e compreensão deste vasto conjunto de intervenientes é vital para assegurar a adoção alargada da plataforma. O mapeamento das suas necessidades dita o desenvolvimento de uma interface focada no autoatendimento, promovendo a autonomia do utilizador na atualização de dados cadastrais, na regularização de pagamentos à distância e na receção de notificações oficiais. Dada a expectável disparidade de literacia digital deste público, a usabilidade e a acessibilidade assumem-se como prioridades técnicas fundamentais no desenho da solução. O *design* de *frontend* deverá privilegiar a clareza interativa e a redução de atrito, com o duplo objetivo de modernizar a experiência de ligação ao clube e descongestionar o atendimento presencial da secretaria.

1.10. Eliciação de Requisitos Assistida por LLM

A engenharia de requisitos constitui a fase central do projeto, traduzindo as necessidades do Boavista Futebol Clube BFC em especificações técnicas concretas. Para assegurar o rigor metodológico, o processo foi orientado pelas diretrizes da norma ISO/IEC/IEEE 29148:2018.

Dada a complexidade dos processos a mapear e as restrições logísticas de acesso contínuo aos intervenientes reais do clube, optou-se por uma abordagem de eliciação não intrusiva. Neste contexto, a recolha tradicional presencial foi substituída por um modelo de simulação assistida por LLM, garantindo a extração de requisitos sem causar disrupção na operação diária da instituição.

1.10.1. Entrevistas Semiestruturadas (Abordagem Qualitativa)

A dimensão qualitativa do levantamento focou-se no mapeamento dos processos e regras de negócio internas, sendo assegurada através da condução de entrevistas semiestruturadas. Dado o contexto do projeto, estas sessões foram executadas num ambiente de *roleplay* tecnológico suportado por LLM.

Para garantir a validade dos dados e mitigar o risco de alucinação ou fuga de escopo por parte da Inteligência Artificial, cada sessão foi iniciada com a configuração de uma *persona* específica (abrangendo a Direção, Secretaria, Tesouraria, Equipas Técnica e Médica), à qual foram injetados o contexto de crise administrativa do clube, o tom de voz expectável e, mais criticamente, fronteiras estritas de domínio (ex: proibir o Diretor Técnico de abordar logística de transportes, focando-o apenas em estatísticas desportivas).

Esta abordagem permitiu emular, com elevado grau de fidelidade, as dores reais dos *stakeholders*. Das sessões extraíram-se constrangimentos operacionais que moldarão a arquitetura do sistema, dos quais se destacam:

- Segurança e Auditoria Clínica: A descoberta de que a validação de Exames Médico-Desportivos (EMD) dita a elegibilidade legal de um atleta para ir a jogo, exigindo um sistema de permissões (RBAC) onde apenas o Departamento Médico pode atribuir a "Alta Clínica".
- Complexidade Financeira Oculta: A necessidade imperativa de separar os fluxos financeiros do Clube (Quotas) e da SAD (Mensalidades da Formação) logo na origem, implicando o encaminhamento automático de verbas.

As transcrições integrais destas sessões, contendo as questões formuladas pela equipa e as respostas detalhadas de cada *stakeholder*, constam no Anexo I.

1.10.2. Workshops de Alinhamento Operacional (Sessões Conjuntas)

Após a conclusão e análise das entrevistas semiestruturadas individuais, a equipa de Engenharia de Requisitos identificou a existência de necessidades operacionais contraditórias entre diferentes departamentos do Boavista FC. Uma vez que as entrevistas isolam a perspetiva de cada *stakeholder*, tornou-se imperativo cruzar estes dados para evitar a especificação de *User*

Stories ambíguas ou conflituosas.

Para mitigar este risco e garantir que o desenho da arquitetura refletia processos de negócio exequíveis e consensuais, optou-se por realizar reuniões conjuntas de alinhamento. O objetivo central destas sessões foi colocar frente a frente os responsáveis com interesses divergentes e, através de mediação analítica, chegar a uma regra de negócio unificada antes de iniciar a redação formal dos requisitos.

Nesta fase preliminar, foram conduzidas três sessões fundamentais para a estabilização do escopo do projeto:

- Alinhamento Médico-Desportivo (Clínica vs. Desporto): Reunião focada na resolução do conflito entre a necessidade de bloqueio legal absoluto por caducidade de Exames Médico-Desportivos (exigência do Diretor Clínico) e a necessidade de flexibilidade e planeamento das convocatórias (exigência do Treinador Principal).
- Alinhamento Financeiro-Administrativo (Tesouraria vs. Secretaria): Sessão orientada para conciliar a necessidade de um atendimento ágil e unificado ao balcão com a exigência legal e contabilística de segregação estanque de fundos entre as contas do Clube e da SAD.
- Alinhamento de Reporte Técnico (Direção vs. Operação): Debate destinado a equilibrar a exigência de dados estatísticos rigorosos no arranque da semana pela Direção Técnica com a necessidade de um preenchimento rápido, móvel e sem atrito por parte dos Delegados e Treinadores no final dos jogos.

As decisões de negócio resultantes destas sessões estabeleceram as fundações lógicas para o desenvolvimento do sistema. As atas integrais destas reuniões, contendo o detalhe dos conflitos debatidos e as soluções de compromisso acordadas, encontram-se documentadas no Anexo III deste relatório.

1.10.3. Inquéritos por Questionário (Abordagem Quantitativa)

A componente quantitativa materializou-se na conceção de um inquérito digital. O objetivo principal desta fase foi validar as dores operacionais identificadas internamente nas entrevistas aos *stakeholders* e auscultar a perspetiva dos utilizadores finais — nomeadamente Encarregados de Educação, encarregados de educação e atletas — face aos processos atuais do clube.

O instrumento de recolha foi estruturado em quatro eixos temáticos essenciais: caracterização demográfica e tecnológica, atendimento e gestão administrativa, comunicação e acompanhamento desportivo, e identificação de prioridades de modernização. Com esta abordagem, procurou-se quantificar o atrito na interação diária com os serviços do Boavista FC e aferir o nível de urgência para a transição digital. Para suportar esta análise, foi recolhida uma amostra sólida de 258 respostas válidas, representativa da heterogeneidade da massa associativa.

A análise rigorosa aos dados extraídos confirmou a necessidade de intervenção imediata, evidenciando padrões de ineficiência que suportam e justificam as decisões de arquitetura e as

regras de negócio previamente estabelecidas nas sessões de alinhamento (Atas). Destacam-se as seguintes conclusões fundamentais:

- **Atendimento Presencial e Faturação Fragmentada:** A maioria dos inquiridos que visita a secretaria com regularidade apontou a segregação de processos (Clube vs. Futebol de Formação) e a consequente lentidão do sistema como as suas maiores frustrações. Este dado quantitativo valida em absoluto a necessidade da "Interface Unificada de Faturação" e do desdobramento automático de verbas acordados com a Tesouraria.
- **Prevenção Clínica e Risco Documental:** Verificou-se que uma fatia expressiva dos inquiridos (40%) já foi apanhada de surpresa pela caducidade do Exame Médico-Desportivo (EMD), resultando em impedimentos desportivos para os atletas. Simultaneamente, existe uma elevada aceitação (89%) à submissão digital segura desta documentação, o que corrobora a exigência do Departamento Médico em implementar bloqueios sistémicos e o envio de alertas preventivos automáticos com 30 dias de antecedência.
- **Informalidade da Comunicação Desportiva:** A auscultação revelou uma dependência crítica de canais não oficiais (grupos de WhatsApp) para a gestão logística e acompanhamento do rendimento (assiduidade e minutos de utilização). A ausência de uma via institucional centralizada gerou uma exigência clara por uma "Área do Atleta" e por notificações *Push* oficiais, suportando as diretrizes da Direção Técnica para a digitalização célere da ficha de jogo.
- **Prioridades e Identidade Digital:** A modernização dos serviços é encarada com elevada urgência (níveis 4 e 5) por 67% da amostra. A introdução de um Cartão de Encarregado de Educação/Atleta 100% digital emergiu como a prioridade absoluta (identificada por 79% dos inquiridos), superando o simples esquecimento do suporte físico e revelando uma procura generalizada por maior conveniência no ecossistema do clube.

Para ilustrar de forma clara o atrito atual nas interações operacionais e as áreas de intervenção consideradas prioritárias pelos utilizadores finais, apresentam-se de seguida as Figuras 1.2, 1.3 e 1.4.

6. Qual é a sua maior frustração quando precisa de regularizar a sua situação administrativa presencialmente na Secretaria?

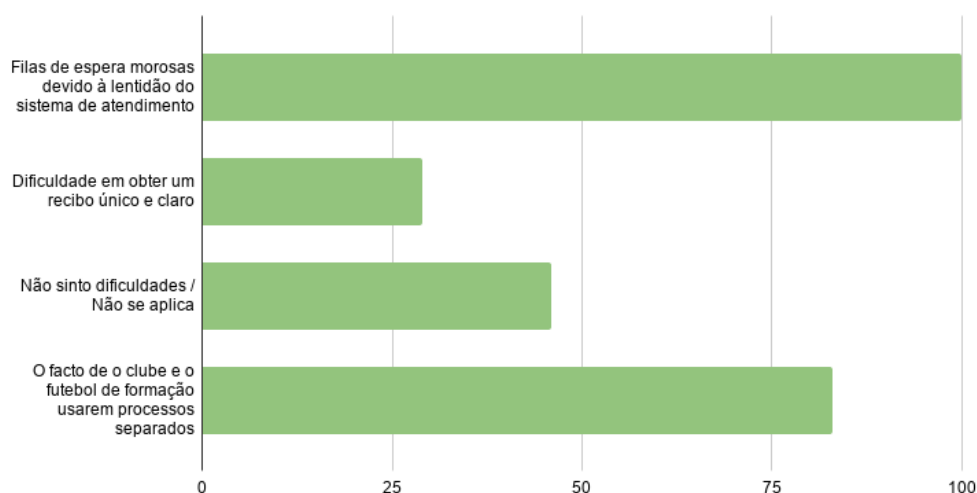


Figura 1.2.: Principais frustrações identificadas no atendimento presencial na Secretaria.

13. Já alguma vez o seu educando (ou o próprio) ficou impedido de treinar ou de ser convocado devido à caducidade do Exame Médico-Desportivo (EMD) sem que se tivesse apercebido com antecedência?

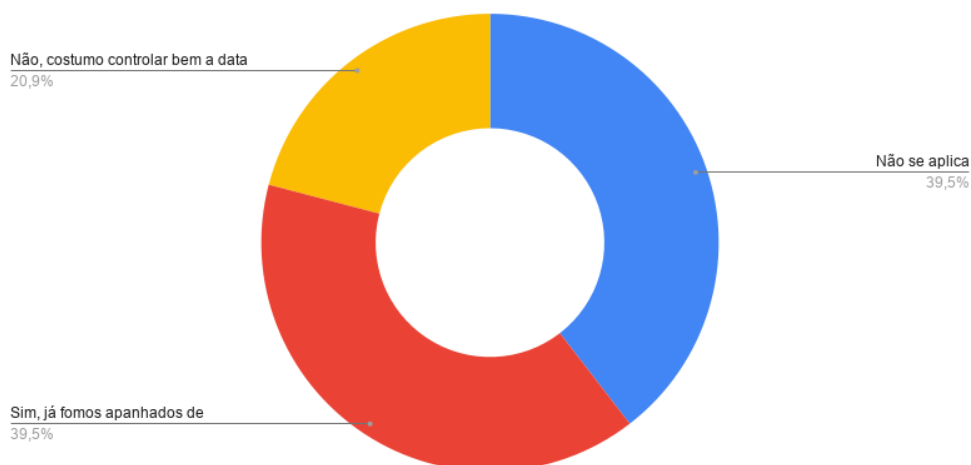


Figura 1.3.: Impacto da caducidade imprevista de Exames Médico-Desportivos nos atletas.

18. Face ao que respondeu, quais destas 3 áreas considera mais urgentes para melhorar a sua experiência com o clube? (Escolha 3)

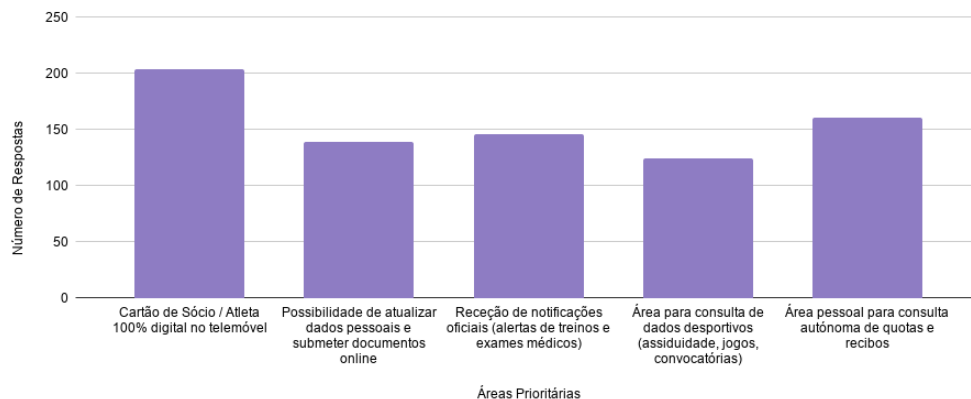


Figura 1.4.: Áreas de intervenção consideradas prioritárias para a modernização digital dos serviços.

O detalhe metodológico desta auscultação, incluindo a totalidade das questões formuladas, a distribuição exata de frequências para cada resposta e o cruzamento estatístico integral, encontra-se documentado no Anexo II.

1.11. Modelação Operacional: *User Stories*

Com a recolha de dados estabilizada pelas fases de eliciação (entrevistas, *workshops* e inquéritos), tornou-se imperativo traduzir as necessidades e constrangimentos dos *stakeholders* numa linguagem estruturada, padronizada e orientada ao valor de negócio. Para este efeito, adotou-se a especificação através de *User Stories*.

No contexto da Engenharia de Requisitos, as *User Stories* assumem o papel de ponte de comunicação entre o domínio do negócio e a equipa de engenharia. Elas não descrevem a arquitetura técnica, as bases de dados ou o código do sistema, mas refletem a perspetiva pura e expectável do utilizador final. A sua redação é propositadamente curta, objetiva e estruturada na sintaxe padrão da indústria: *Como [Ator], Quero [Ação], Para [Valor/Benefício]*. Esta formulação garante que o foco do sistema se mantém na resolução do problema real do Boavista FC, e não apenas na implementação cega de funcionalidades.

Para neutralizar qualquer ambiguidade e garantir a testabilidade objetiva de cada história, as especificações foram enriquecidas com Critérios de Aceitação. Adotou-se a estrutura comportamental orientada a cenários (*Given / When / Then*), definindo as fronteiras lógicas exatas que ditam o sucesso ou a rejeição da funcionalidade.

A extração sistemática destas histórias a partir das transcrições das entrevistas foi suportada por Inteligência Artificial (LLM), utilizando uma engenharia de *prompts* estrita para garantir a consistência com a literatura de desenvolvimento de *software*. O modelo foi alimentado iterativamente com as transcrições de um único *stakeholder* de cada vez, garantindo elevado detalhe na extração.

Este esforço de transposição direta resultou num catálogo exaustivo e sólido, totalizando 41 *User Stories*. De forma a ilustrar o nível de granularidade e o rigor exigido na especificação comportamental, apresentam-se de seguida dois exemplos representativos extraídos do catálogo final: um focado na tesouraria (US10) e outro na privacidade clínica (US12). O catálogo integral da especificação, bem como a metodologia detalhada de geração e o *prompt* arquitetural utilizado, encontram-se detalhados no Anexo IV.

ID	US10
Título	Segregação de Fluxos por Centros de Responsabilidade (Clube vs. Academia)
Como	CEO
Quero	Que o sistema categorize as receitas em Centros de Responsabilidade distintos (Clube e Formação).
Para	Garantir a rastreabilidade pura das origens de capital e identificar a autossustentabilidade da academia face ao orçamento institucional.
CrITÉrios de Aceitação	Cenário: Distinção de receitas institucionais e desportivas. Dado que existem entradas de capital registadas no sistema. Quando o CEO filtra a visão financeira por “Centro de Responsabilidade”. Então o sistema deve isolar as receitas de Quotas (Clube) das receitas de Mensalidades e Inscrições (Academia). E apresentar o rácio de cobertura de despesas para cada unidade de negócio.

Tabela 1.1.: Especificação da *User Story* US10 - Segregação de Fluxos por Centros de Responsabilidade (Clube vs. Academia).

ID	US12
Título	Proteção de Dados Sensíveis e Sigilo Clínico (RGPD)
Como	CEO
Quero	Que os dados clínicos dos atletas sejam estritamente confidenciais e inacessíveis a perfis administrativos ou desportivos.
Para	Cumprir rigorosamente com o RGPD e garantir a confidencialidade da saúde dos menores e profissionais do BFC.
Critérios de Aceitação	Cenário: Visualização de disponibilidade sem exposição de diagnóstico. Dado que um Treinador consulta a lista de atletas para o treino. Quando visualiza um atleta sob cuidados médicos. Então o sistema deve exibir apenas o estado de aptidão (Disponível/Indisponível/Condicionado). E omitir qualquer diagnóstico detalhado ou exames médicos associados.

Tabela 1.2.: Especificação da *User Story* US12 - Proteção de Dados Sensíveis e Sigilo Clínico (RGPD).

1.11.1. Validação e Aceitação de *User Stories*

Concluída a especificação formal das *User Stories*, procedeu-se à fase de validação. O objetivo desta etapa foi garantir que a tradução das necessidades operacionais para linguagem técnica não gerou desvios de escopo nem omissões críticas.

Para evitar a disrupção da operação diária do Boavista FC, optou-se por um ciclo de validação assíncrona. O caderno de especificações foi segmentado por domínio de negócio e submetido, via correio eletrónico, à apreciação individual de cada responsável de departamento. Foi solicitado a cada interveniente que confirmasse se os Critérios de Aceitação refletiam com exatidão as regras do seu departamento.

Após uma iteração de pequenos ajustes de vocabulário e clarificação de regras de negócio (nomeadamente a afinação dos alertas de caducidade médica e a segregação de IBANs), obteve-se a aprovação formal de todos os módulos. A Tabela 1.3 sintetiza o estado de aceitação do caderno de *User Stories*.

Responsável	Módulo / Domínio	Método de Validação	Estado
Miguel Santos	Desportivo (Treinadores)	Revisão Assíncrona	Aprovado
Jorge Silva	Gestão de Topo (Dashboard)	Revisão Assíncrona	Aprovado
Henrique Leal	Financeiro e Tesouraria	Revisão Assíncrona	Aprovado
Carla Mendes	Atendimento e Secretaria	Revisão Assíncrona	Aprovado
Armando Silva	Direção Técnica	Revisão Assíncrona	Aprovado
Nuno Ramos	Clínico e Auditoria Legal	Revisão Assíncrona	Aprovado

Tabela 1.3.: Matriz de validação e aceitação formal das *User Stories*.

1.12. Engenharia e Especificação de Requisitos

A presente secção detalha a transição metodológica entre a fase de levantamento de necessidades e a modelação técnica do sistema. Com base na informação recolhida na fase anterior — materializada através de *User Stories*, entrevistas aos *stakeholders*, atas de reuniões e respostas a questionários —, o passo seguinte consistiu em consolidar estes dados num referencial técnico estruturado.

O objetivo desta fase não se limitou à transcrição das intenções dos utilizadores, mas focou-se na aplicação de princípios de engenharia de *software* para definir o comportamento esperado da plataforma. Tratou-se de traduzir os constrangimentos operacionais e as restrições regulamentares do clube em regras de negócio claras, limites arquiteturais e lógicas de transação.

Para materializar esta ponte entre o domínio do negócio e a engenharia, a equipa adotou um processo iterativo composto por quatro fases lógicas: extração assistida, refinamento iterativo, segregação arquitetural e formalização documental. Este percurso, que culminou na especificação exaustiva do produto, é detalhado nas subsecções seguintes.

1.12.1. Abordagem Metodológica e Extração Assistida por LLM

A fase de especificação iniciou-se com uma abordagem *top-down*, partindo dos objetivos de negócio de alto nível para a definição das funcionalidades técnicas. Para acelerar e sistematizar esta conversão, a equipa recorreu a um LLM como ferramenta de apoio à extração de requisitos.

A metodologia consistiu em parametrizar o modelo com o contexto integral do projeto, fornecendo-lhe as *User Stories*, os constrangimentos técnicos identificados nas reuniões e os fluxos de trabalho descritos nas entrevistas. A estratégia de interação (*prompt engineering*) foi desenhada para ultrapassar a mera tradução de pedidos diretos, obrigando à identificação de duas categorias distintas:

- **Requisitos Explícitos:** Funcionalidades solicitadas de forma direta pelos *stakeholders* (ex: registo de assiduidade no relvado, verificação de quotas, bloqueio de convocatórias).
- **Requisitos Implícitos e de Suporte:** Funcionalidades que o cliente não formulou explicitamente, mas que são exigências arquiteturais para a coerência do sistema (ex: gestão de máquinas de estados, transição temporal de épocas desportivas e rastreabilidade de transações financeiras).

O resultado desta primeira iteração foi a geração de um catálogo preliminar de requisitos. Esta lista inicial serviu como base estruturada de trabalho, permitindo à equipa avançar para a fase de auditoria humana e resolução de ambiguidades. A transcrição integral dos *templates* de instrução utilizados nesta fase, bem como a lista final de requisitos gerada, encontram-se detalhados no Anexo V.

1.12.2. Refinamento Iterativo e Resolução de Conflitos

Após a extração inicial, o catálogo bruto de requisitos foi submetido a um rigoroso processo de auditoria humana. O propósito desta iteração foi erradicar ambiguidades lógicas, fundir requisitos duplicados — frequentemente resultantes de *User Stories* sobrepostas — e aplicar métricas determinísticas e testáveis, em conformidade com as normas de qualidade de *software* (ISO/IEC 25010).

O esforço de refinamento garantiu que descrições narrativas vagas fossem convertidas em parâmetros matemáticos e regras de negócio estritas. Para ilustrar a profundidade e o rigor analítico deste processo, apresenta-se a evolução do requisito de Gestão de Convocatórias Oficiais, onde jargões subjetivos gerados inicialmente pela inteligência artificial foram substituídos por limites operacionais exatos.

Exemplo 1: Refinamento de Ambiguidade Temporal [REQ-04]

- **Estado Inicial:** «(...) O sistema expõe indicadores consolidados de desempenho recente, englobando a média avaliativa de treino e a taxa de assiduidade.»
- **Estado Refinado:** «(...) O sistema deve calcular e expor a média avaliativa das últimas 5 sessões de treino com presença validada [RF-03] e a taxa de assiduidade das últimas 4 semanas [RF-01]. Quando o histórico disponível for inferior ao período mínimo de cálculo, deve ser emitida uma notificação de dados insuficientes.»

Neste primeiro exemplo, a métrica ambígua "desempenho recente" — impossível de validar num protocolo de testes — foi substituída por janelas temporais estritas ("últimas 5 sessões" e "últimas 4 semanas"). Adicionalmente, previu-se a exceção lógica de ausência de dados, garantindo que o sistema não falha perante novas inscrições.

Adicionalmente, a resolução de falhas de lógica transaccional constituiu outro vetor crítico da

nossa intervenção. O caso da Avaliação de Desempenho [REQ-03] ilustra a transição de um conceito abstrato para um modelo matemático seguro e tolerante a falhas do utilizador.

Exemplo 2: Resolução de Lacunas Matemáticas e Estados Lógicos [REQ-03]

- Estado Inicial: *«(...) O sistema disponibiliza um mecanismo de classificação baseado numa escala quantitativa parametrizada (...) A ausência de registo nas 24 horas transita a lista para um estado estrito de leitura.»*
- Estado Refinado: *«(...) O sistema disponibiliza um seletor de classificação numérica no intervalo de 1.0 a 5.0, com incrementos de 0.5 pontos. (...) A ausência de nota submetida dentro da janela [24h] é registada no sistema como 'Não Avaliado', sendo identificada através de um estado lógico próprio e excluída do cálculo de médias longitudinais.»*

Neste segundo caso, a expressão genérica "escala quantitativa" foi materializada em restrições exatas de *input* (1.0 a 5.0, *steps* de 0.5). Mais criticamente, a equipa identificou uma falha arquitetural na versão bruta: a ausência de tratamento matemático para o esquecimento de avaliação. A introdução do estado excecional "Não Avaliado" garantiu que falhas humanas não enviassem os cálculos estatísticos longitudinais (evitando a assunção incorreta de notas nulas).

Este nível de abstração e rigor foi aplicado transversalmente a todo o catálogo, solidificando as bases para a posterior classificação técnica e segregação dos atributos arquiteturais.

1.12.3. Segregação Arquitetural (Funcional vs. Não-Funcional)

A etapa final do processo de engenharia de requisitos consistiu na separação rigorosa entre as funcionalidades de negócio (o que o sistema faz) e as restrições de qualidade e infraestrutura (como o sistema se comporta). Durante a fase de extração, constatou-se que alguns requisitos aglomeravam ações do utilizador e regras arquiteturais num único bloco narrativo, violando o princípio da separação de preocupações (*Separation of Concerns*).

A equipa procedeu a uma auditoria arquitetural para segregar estes domínios, originando dois catálogos distintos no Documento SRS final: Requisitos Funcionais (RF) e Requisitos Não-Funcionais (RNF). Esta segregação materializou-se de duas formas: através da reclassificação integral de requisitos puramente técnicos e através da extração de atributos de qualidade embutidos em fluxos operacionais.

Exemplo 1: Reclassificação Integral (Transações de Base de Dados)

Na lista preliminar, a garantia de integridade da base de dados foi gerada sob a forma de uma funcionalidade de negócio:

- Estado Bruto [REQ-43]: «*Persistência Relacional com Garantia de Transações ACID para Operações Financeiras.*»

A equipa identificou que o modelo ACID não descreve uma interação do utilizador, mas sim um atributo de fiabilidade da infraestrutura. Consequentemente, este requisito foi totalmente expurgado do catálogo funcional e reclassificado no SRS como o Requisito Não-Funcional [RNF-18] — Garantia Transaccional ACID para Operações Financeiras, integrando a secção de Fiabilidade.

Exemplo 2: Extração de Atributo de Qualidade (Pesquisa e Performance)

Noutros cenários, requisitos funcionais válidos continham constrangimentos técnicos embutidos na sua descrição. É o caso do motor de pesquisa da Secretaria:

- Estado Bruto [REQ-36]: «*(...) A solicitação ao repositório impõe um constrangimento tecnológico (debounced search), acautelando recargas massivas da base...*»

A mecânica de pesquisa (o input mínimo de caracteres e a devolução de resultados) foi mantida no catálogo funcional sob o identificador [RF-35] — Motor de Pesquisa Unificada. No entanto, a regra de proteção contra sobrecarga de rede foi extraída cirurgicamente, quantificada e mapeada para o catálogo não-funcional de Desempenho como [RNF-04] — Controlo de Sobrecarga de Pesquisa (Debounce), estipulando a métrica exata de contenção de pedidos ($\geq 300\text{ms}$).

Este trabalho de segregação arquitetural assegurou que o documento final de requisitos cumpra os atributos de qualidade exigidos pela norma IEEE 830, providenciando instruções claras e isoladas tanto para as equipas de desenvolvimento aplicacional como para a engenharia de infraestruturas. As diretivas de instrução utilizadas para orientar esta purificação assim como o resultado integral do processo — o documento *Software Requirements Specification* (SRS) — encontram-se documentadas no Anexo VI.

1.13. Modelação Conceptual do Domínio

Com os catálogos funcionais e não-funcionais devidamente segregados, a etapa subsequente consistiu em mapear a ontologia do sistema através de um *Modelo de Domínio Conceptual*. Esta modelação tratou de construir uma representação abstrata de alta fidelidade que espelhasse os conceitos do mundo real e as complexas regras de negócio que governam o ecossistema do clube, funcionando como uma ponte semântica crucial entre a linguagem dos utilizadores e a engenharia de implementação.

O principal valor desta abordagem reside na capacidade de observar o ecossistema do SIGD sob uma perspetiva macroestrutural. Ao consolidar as entidades da Secretaria, Tesouraria, Departamento Médico e Operações de Relvado num único referencial, o modelo atua como

uma âncora de consistência. Esta visão de escala afastada permite antever de que forma as regras de negócio de diferentes domínios interagem entre si, mitigando o risco de conceber módulos isolados ou com lógicas transacionais concorrentes. Garante-se, assim, que toda a arquitetura partilha o mesmo modelo mental e o mesmo vocabulário operacional da instituição.

Apesar de constituir uma abstração preliminar — inteiramente desvinculada de constrangimentos físicos de tabelas de bases de dados ou de classes de programação —, este modelo define as leis estruturais da plataforma. Através da formalização de hierarquias de especialização (como a herança dos múltiplos papéis funcionais a partir de um utilizador comum) e de associações regidas por verbos ativos descritivos e cardinalidades estritas, o modelo mapeia com precisão a lógica do negócio. Torna-se evidente, por exemplo, como o estado de elegibilidade clínica de um atleta comunica diretamente com o módulo de convocatórias e com a validação documental da secretaria, blindando a integridade das regras do clube antes da escrita de qualquer linha de código.

Pela sua natureza multidimensional e densidade de informação, este mapa conceptual estabelece a fundação teórica sobre a qual toda a infraestrutura técnica e o modelo físico de dados são edificados. O fluxo metodológico da sua execução, bem como o próprio código descritivo integral que suporta esta ontologia, encontram-se documentados e disponíveis para consulta detalhada no Anexo VIII.

1.13.1. Análise Detalhada por Sub-Domínios de Negócio

Dada a envergadura e a natureza integrada do ecossistema SIGD, a validação integral da ontologia exige uma descida de escala analítica. Para o efeito, o modelo mestre foi segmentado em quatro micro-contextos operacionais críticos, analisados individualmente nas subsecções seguintes através do isolamento das suas regras transacionais e fluxos lógicos.

A. Sub-Domínio de Identidade, Autenticação e Rastreabilidade

O primeiro vetor estrutural do sistema foca-se na abstração da segurança e na generalização dos atores humanos, conforme ilustrado no recorte da Figura 1.5.

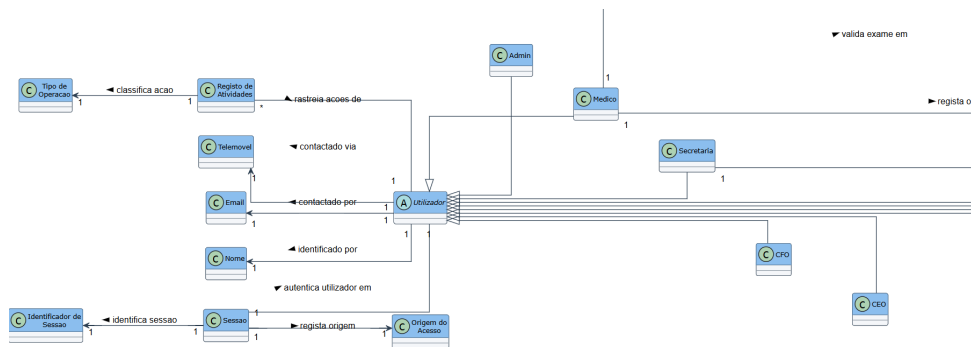


Figura 1.5.: Micro-Contexto de Utilizadores, Sessões e Rastreabilidade

Para mitigar a redundância de dados, foi definida a classe abstrata *Utilizador*, centralizando as propriedades nucleares de identificação e contacto civil (*Nome*, *Email*, *Telemóvel*). Desta superclasse derivam por herança estrita ($< | - -$) todos os perfis operacionais e executivos do clube (*Admin*, *Médico*, *Secretaria*, *CFO*, *CEO*, entre outros).

Do ponto de vista da governança de dados e auditoria de acessos, o modelo desacopla os conceitos de infraestrutura técnica e foca-se em entidades de negócio puras:

- A entidade *Sessão* autentica o utilizador a partir de um *Identificador de Sessão* e mapeia a sua *Origem do Acesso*, assegurando limites de concorrência.
- A entidade *Registo de Atividades* rastreia as ações de forma imutável, classificando cada comportamento através de um *Tipo de Operação* (ex: criação, edição ou eliminação), garantindo a responsabilização legal sobre os dados sensíveis do clube.

B. Sub-Domínio de Estrutura Desportiva e Parametrização Financeira

O segundo recorte, visível na Figura 1.6, explicita a hierarquia desportiva do clube e a forma como o planeamento macro se converte em constrangimentos financeiros.

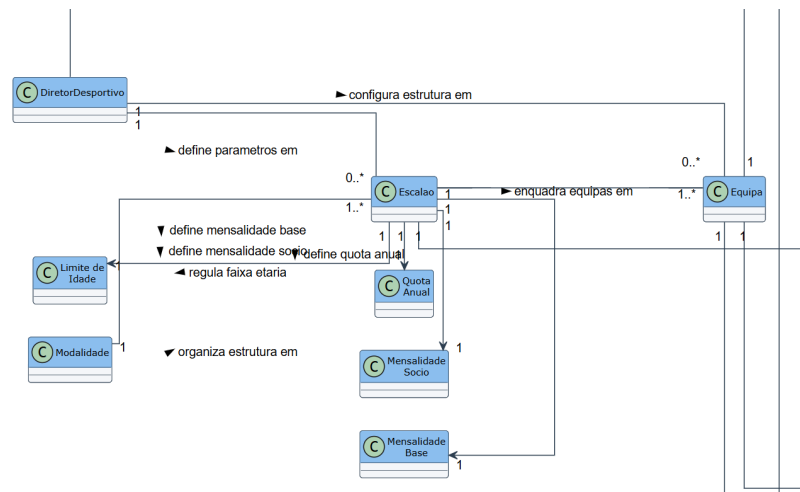


Figura 1.6.: Micro-Contexto de Configuração Desportiva e Escalões

O fluxo organizacional inicia-se na *Modalidade*, que segmenta a estrutura desportiva em múltiplos *Escalões*. O *Escalão* atua como o grande agregador de regras administrativas e financeiras do sistema. É este bloco que regula a faixa etária permitida através do *Limite de Idade* e dita os custos operacionais associados, ramificando-se radialmente nas entidades satélites *Quota Anual*, *Mensalidade Base* e *Mensalidade Sócio*.

A transição para o plano competitivo faz-se através da relação unívoca onde o *Escalão* enquadra as *Equipas*. Toda esta arquitetura de tabelas mestres é supervisionada de forma exclusiva pelo papel do *Diretor Desportivo*, o único ator com competência semântica para configurar a estrutura e definir os parâmetros dos escalões no início de cada ciclo de gestão.

C. Sub-Domínio de Operações de Relvado, Assiduidade e Avaliação

A atividade diária e o núcleo operacional do ecossistema desportivo encontram-se mapeados no micro-contexto da Figura 1.7, onde o *Treinador* assume o papel de orquestrador central.

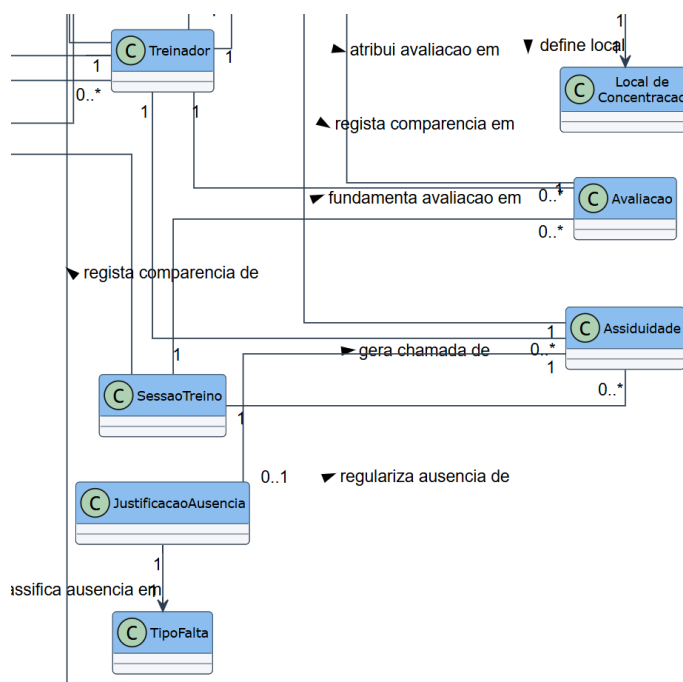


Figura 1.7.: Micro-Contexto de Execução de Treinos, Chamadas e Notas

A execução de uma *Sessão de Treino* despoleta automaticamente duas ações transacionais paralelas no sistema:

- Gera uma chamada de *Assiduidade* (cardinalidade 0..*), onde o *Treinador* regista a comparencia ou ausência física dos atletas.
- Fundamenta uma *Avaliação* quantitativa de rendimento técnico-tático, permitindo ao *Treinador* pontuar o desempenho pós-sessão.

Para salvaguardar as exceções lógicas decorrentes de faltas, o modelo integra a entidade *Justificação de Ausência*, que regulariza o estado da assiduidade. Esta entidade valida o motivo apresentado classificando a ausência num *Tipo de Falta* parametrizado. Por fim, o modelo acautela as obrigações logísticas de convocação ao ancorar a entidade *Local de Concentração* às diretivas do evento, centralizando as comunicações de balneário.

D. Sub-Domínio de Tesouraria e Fluxo de Pagamentos Autónomos

O fecho do ciclo de negócio do ERP materializa-se na liquidação financeira e na automatização fiscal das obrigações, conforme exposto na Figura 1.8.

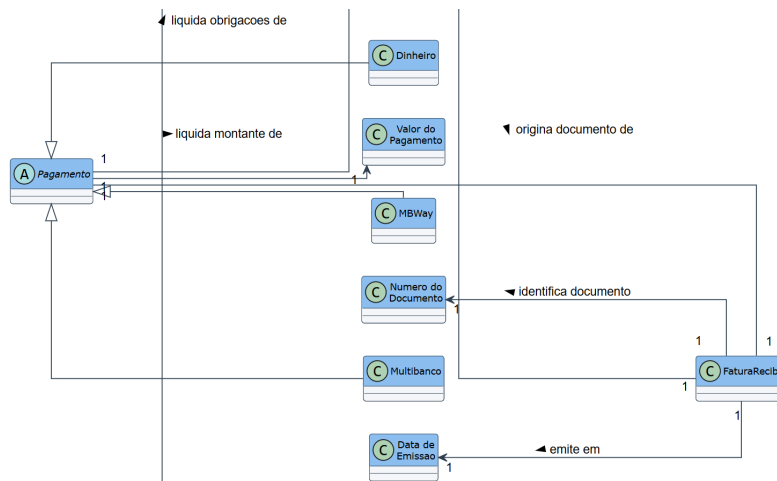


Figura 1.8.: Micro-Contexto de Liquidação Financeira e Emissão Fiscal

O modelo conceptual padroniza a extinção de dívidas através da generalização da classe *Pagamento*, ramificada nas modalidades físicas e digitais aceites pelo clube (*Dinheiro*, *MBWay* e *Multibanco*). Um *Pagamento* liquida uma ou mais obrigações financeiras, validando de forma estrita o *Valor do Pagamento* em trânsito.

A segurança e a conformidade legal do fluxo são asseguradas pela relação direta e obrigatória de cumplicidade transacional: a validação de um *Pagamento* origina obrigatoriamente (relação 1 para 1) a emissão de uma *Fatura-Recibo*. Este documento fiscal nasce blindado no domínio através das suas caixas de metadados exclusivas — o *Número do Documento* (gerado sequencialmente e de forma inalterável pela lógica do sistema) e a *Data de Emissão* —, garantindo auditoria financeira total e mitigando cenários de evasão ou falhas de reconciliação bancária.

1.14. Especificação de Casos de Uso

1.14.1. Derivação e Rastreabilidade

Numa primeira fase, o escopo arquitetural foi delimitado através do catálogo de **Requisitos Funcionais (RFs)**. Estes artefactos atuaram como a fundação macro do projeto, definindo de forma agnóstica *o que* o sistema deveria ser capaz de executar numa perspetiva global de negócio (por exemplo, a necessidade de processar mensalidades financeiras ou gerir ocorrências clínicas).

Para garantir que a implementação técnica não perdia o foco na usabilidade e no valor entregue aos utilizadores finais, os RFs foram subsequentemente decompostos e refinados em **Histórias de Utilizador (USs)**. Adotando as práticas da metodologia Ágil, as USs permitiram identificar inequivocamente o ator de domínio (Treinador, Médico, Secretaria, Encarregado de Educação),

a sua intenção de interação e o valor acrescentado esperado, focando a análise no *para quem* e no *porquê*.

Por fim, a formalização contratual da interação entre o ator e o sistema materializou-se na especificação algorítmica dos **Casos de Uso (UCs)**. Enquanto as USs mantêm uma linguagem natural e orientada ao problema, os UCs estabeleceram o contrato técnico e rigoroso da solução (o *como*). Cada Caso de Uso documentou de forma atômica as pré-condições, os passos de validação do sistema e os respetivos fluxos de exceção, garantindo que o sistema nunca termina num estado inválido.

Esta rastreabilidade contínua ($RF \rightarrow US \rightarrow UC$) assegurou que nenhum requisito de negócio fosse implementado sem enquadramento no catálogo inicial e estabeleceu a base fundamental para a criação das posteriores suites de testes automáticos.

1.14.2. Metodologia de Especificação Assistida por IA

A especificação massiva dos Casos de Uso (UCs) foi alavancada por técnicas avançadas de Engenharia de Prompts, adotando uma metodologia de especificação assistida por Inteligência Artificial (IA). O processo baseou-se na execução de um único *prompt* de especificação estrito e padronizado (documentado no **Anexo IV**), submetido de forma individual para a redação detalhada de cada cenário de utilização. Este guião instruiu o modelo linguístico a operar sob o rigor de uma *State Machine*, impondo regras estruturais absolutas: a abstração total de elementos de interface (*Zero UI*), a atomização das validações do sistema (onde cada passo corresponde a uma única verificação independente) e a utilização de uma numeração matemática herdada nas ramificações lógicas, assegurando o fecho determinístico de qualquer fluxo alternativo ou de exceção.

Para garantir a máxima fiabilidade técnica, a equipa de engenharia adotou ativamente o padrão metodológico *Human-in-the-Loop*, estabelecendo um fluxo iterativo e contínuo de revisão e aprovação. A especificação documental ocorreu através de geração em lote, iniciando-se com a seleção do UC atribuído e o respetivo envio do *prompt* de especificação ao modelo linguístico (LLM). O *output* gerado foi, de imediato, submetido a uma fase de análise humana rigorosa (focada na revisão das regras e formatação), validando o fluxo contra a lógica de negócio intrínseca ao domínio desportivo e clínico do SIGD.

Sempre que a verificação de conformidade do conteúdo falhava (e.g., o modelo exibia alucinações contextuais ou quebrava o determinismo estrutural), a equipa despoletava um ciclo de correção iterativo (*Chat IA*), solicitando a retificação exata das falhas detetadas. Uma vez atingida a conformidade total, o documento era aprovado, integrado de forma consolidada nos ficheiros `.tex` via Prism, e o ciclo reiniciava para o Caso de Uso seguinte. Esta abordagem sistémica garantiu que a IA atuasse exclusivamente como um acelerador processual, mantendo a equipa técnica com o controlo absoluto sobre o rigor da especificação final.

1.14.3. Caso de Uso em Destaque: [Abrir ocorrência clínica]

Tabela 1.4.: Caso de Uso UC-12.1 — Abrir ocorrência clínica

Use Case	UC-12.1: Abrir ocorrência clínica
Descrição	O sistema registra formalmente uma nova condição de saúde associada a um atleta, documentando o diagnóstico e aplicando as inerentes restrições à sua elegibilidade e prática desportiva.
Atores	Médico (Primário)
Pré-condição	▪ O ator encontra-se autenticado no sistema com privilégios de acesso ao módulo clínico. ▪ O perfil do atleta alvo encontra-se registado e ativo no diretório do clube.
Pós-condição (Sucesso)	▪ A ocorrência clínica é arquivada e associada ao histórico de saúde do atleta. ▪ O estado operacional e a elegibilidade do atleta nas ferramentas da equipa técnica são atualizados, refletindo o grau de restrição imposto.
Fluxo Normal	1. O ator solicita a abertura de uma nova ocorrência clínica. 2. O sistema verifica os privilégios de acesso clínico do ator. 3. O ator submete a identificação do atleta alvo e os parâmetros clínicos (diagnóstico, região anatómica afetada, grau de restrição desportiva e data prevista de reavaliação). 4. O sistema verifica se a identificação submetida corresponde a um perfil de atleta ativo. 5. O sistema verifica a integridade temporal, validando se a data de reavaliação estipulada é estritamente futura. 6. O sistema verifica a inexistência de ocorrências clínicas ativas prévias para a mesma região afetada. 7. O sistema regista a ocorrência clínica no processo de saúde do atleta. 8. O sistema atualiza o estado global de elegibilidade desportiva do atleta em concordância com o grau de restrição. 9. O sistema confirma a conclusão do registo com sucesso ao ator.

Continua na página seguinte...

Tabela 1.4.: Caso de Uso UC-12.1 — Abrir ocorrência clínica (continuação)

Use Case	UC-12.1: Abrir ocorrência clínica
Fluxos Alternativos	▪ Substituição de diagnóstico prévio concorrente 6a. O sistema deteta a existência de uma ocorrência clínica em aberto para a exata mesma região anatômica. 6b. O sistema alerta o ator para o conflito e solicita a diretriz para a sobreposição ou fusão do diagnóstico. 6c. O ator submete a confirmação de agravamento ou substituição do quadro clínico. 6d. O sistema transita a ocorrência anterior para um estado de histórico/resolvida. 6e. O fluxo retoma no passo 7. ▪ Ocorrência clínica isenta de impacto na elegibilidade (Apto) 8a. O sistema deteta que o grau de restrição desportiva submetido é nulo. 8b. O sistema mantém o estado de elegibilidade desportiva do atleta inalterado, preservando a sua aptidão plena para competir e treinar. 8c. O fluxo retoma no passo 9.

Continua na página seguinte...

Tabela 1.4.: Caso de Uso UC-12.1 — Abrir ocorrência clínica (continuação)

Use Case	UC-12.1: Abrir ocorrência clínica
Fluxos de Exceção	▪ Acesso negado por restrição de perfil 2a. O sistema deteta que o ator não dispõe da autorização médica necessária para manipular processos clínicos restritos. 2b. O sistema notifica o ator da quebra de confidencialidade e bloqueia a operação. 2c. O Caso de Uso encerra. ▪ Perfil de atleta inválido 4a. O sistema deteta que o atleta referenciado não existe ou o seu perfil encontra-se administrativamente arquivado. 4b. O sistema notifica o ator da invalidade da entidade desportiva e descarta a abertura do processo clínico. 4c. O Caso de Uso encerra. ▪ Incoerência cronológica da reavaliação 5a. O sistema constata que a data de reavaliação submetida pertence ao passado ou é coincidente com o momento atual. 5b. O sistema notifica o ator da impossibilidade de estipular prognósticos de recuperação retroativos e rejeita os dados. 5c. O Caso de Uso encerra.

Para demonstrar a aplicação prática da metodologia de especificação adotada, apresenta-se na Tabela 1.4 o mapeamento integral do Caso de Uso correspondente à abertura de uma ocorrência clínica. Este cenário foi selecionado para escrutínio detalhado devido à sua elevada complexidade, evidenciando não só a densidade de validações cronológicas e de negócio exigidas, mas também a forte interdependência modular do sistema (nomeadamente, o impacto direto de uma decisão do Módulo Clínico na elegibilidade desportiva imposta no Módulo do Treinador).

1.14.4. Modelação Visual

Para complementar a especificação textual algorítmica e proporcionar uma perspetiva macroscópica sobre as fronteiras do sistema, foram desenvolvidos diagramas de Casos de Uso baseados na notação *Unified Modeling Language* (UML). De forma a preservar a clareza analítica e evitar entropia visual, a modelação não foi condensada num único esquema global, tendo sido estrategicamente segmentada pelos diversos domínios funcionais do SIGD.

A Figura 1.9 ilustra, a título exemplificativo, o diagrama UML correspondente ao **Módulo**

D (Equipa Técnica). Este modelo visualiza inequivocamente as fronteiras de interação do Treinador com o sistema, evidenciando o seu escopo de atuação primário nas famílias funcionais de Gestão de Sessão de Treino, Gestão de Convocações e Ficha de Jogo. Adicionalmente, o diagrama expõe as relações estruturais de dependência e extensão («*include*» e «*extend*») intrínsecas aos fluxos desportivos do clube.

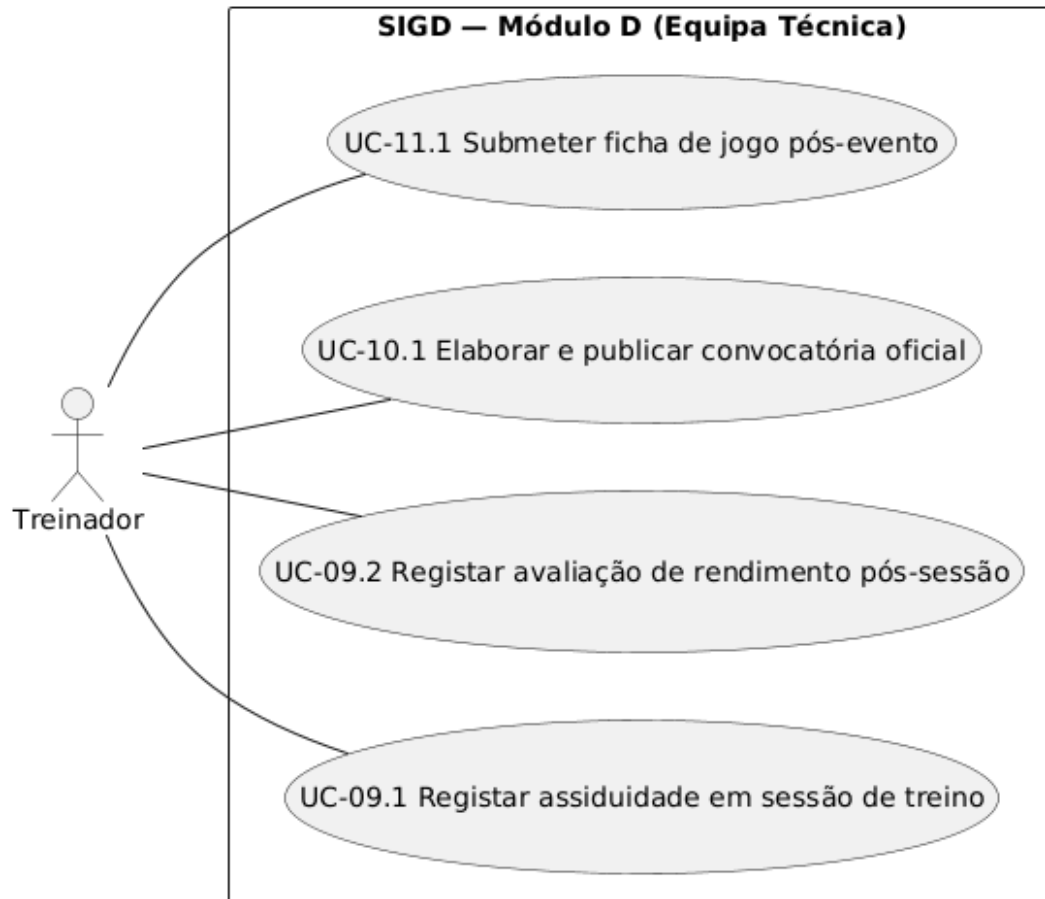


Figura 1.9.: Diagrama de Casos de Uso UML — Módulo D (Equipa Técnica).

Capítulo 2 - Arquitetura e Design do Software utilizando LLM

2.1. Especificação de APIs do Sistema de Gestão para Clube Desportivo (SIGD)

Esta secção apresenta a especificação técnica detalhada das interfaces de programação de aplicação (APIs) desenvolvidas para o **Sistema de Gestão para Clube Desportivo (SIGD)**. A especificação segue padrões de mercado e reflete o estado de implementação previsto para a aplicação *backend* do SIGD, desenvolvida em Java com *Spring Boot*.

2.1.1. Especificação de APIs

Visão Geral da Arquitetura de Comunicação

A arquitetura de comunicação do SIGD assenta sobre o modelo **REST (Representational State Transfer)**, tirando partido das semânticas nativas do protocolo **HTTP/S**. O *backend* funciona como um fornecedor centralizado de serviços de dados e lógica de negócio, expondo *endpoints* RESTful consumidos pelo cliente web de administração e pela aplicação móvel dos atletas e encarregados de educação.

Princípios de Comunicação do SIGD:

1. **Comunicação Sem Estado (*Stateless*):** O servidor *backend* não retém qualquer estado de sessão do cliente. Todas as requisições devem conter as credenciais necessárias para a sua autorização.
2. **Formato de Intercâmbio de Dados:** Toda a comunicação de dados utiliza o formato **JSON**, exceto em *endpoints* específicos de exportação de ficheiros, como relatórios em formato PDF.
3. **Autenticação baseada em *Tokens* (JWT):** O controlo de acesso é implementado via **JSON Web Tokens**. Após a autenticação bem-sucedida, o cliente recebe um *token* de acesso (*access_token*) e um *token* de renovação (*refresh_token*). Todas as

chamadas subsequentes a rotas protegidas devem incluir o *token* no cabeçalho HTTP *Authorization*, no formato *Bearer <token>*.

4. **Controlo de Acesso Baseado em Perfis (RBAC):** A segurança ao nível do método é imposta através de anotações *@PreAuthorize*, validando as permissões do utilizador de acordo com o seu perfil (*ROLE_ADMIN*, *ROLE_CEO*, *ROLE_CFO*, entre outros).

2.1.2. Lista de *Endpoints* Disponíveis por Módulo

Pretende-se mapear a totalidade dos *endpoints* disponíveis, organizados de forma modular no sistema, através da Tabela 2.1.

Módulo	Método	Rota	Descrição
Autenticação	POST	/api/v1/auth/login	Efetua <i>login</i> e emite os <i>tokens</i> JWT.
	POST	/api/v1/auth/refresh	Renova o <i>token</i> usando o <i>refresh_token</i> .
Portal (EE)	GET	/api/v1/portal/me	Detalhes do Encarregado de Educação e dependentes.
	GET	/api/v1/portal/obrigacoes	Obrigações financeiras dos dependentes.
	GET	/api/v1/portal/agenda	Agenda unificada de treinos e jogos.
Atletas	GET	/api/v1/tesouraria/atletas	Lista e pesquisa atletas de forma paginada.
	POST	/api/v1/tesouraria/atletas	Cria um novo atleta e utilizador.
	PUT	/api/v1/tesouraria/atletas/{id}	Atualiza a ficha cadastral do atleta.
Clínica	POST	/api/v1/clinica/ocorrencias	Regista uma ocorrência clínica, como uma lesão.
	GET	/api/v1/clinica/fila-emd	Consulta a fila de exames médico-desportivos pendentes.
	POST	/api/v1/clinica/ocorrencias/{id}/deliberar	Regista a deliberação médica sobre uma submissão de EMD.
Treinos	POST	/api/v1/treinador/sesoes	Cria objetivos e sumário de uma sessão.
	POST	/api/v1/treinador/sesoes/{id}/chamada	Regista presenças e faltas dos atletas.
Financeiro	GET	/api/v1/cfo/resumo-financeiro	Demonstração financeira segregada entre Clube e SAD.
Admin	DELETE	/api/v1/admin/utilizadores/{id}	Direito ao esquecimento: anonimiza dados em conformidade com o RGPD.
	GET	/api/v1/admin/audit-log	Registo de auditoria interna das ações.

Tabela 2.1.: *Endpoints* principais do ecossistema SIGD.

2.1.3. Detalhe Estruturado dos Principais *Endpoints*

1. Autenticação de Utilizador

- **Método e Rota:** POST /api/v1/auth/login
- **Descrição:** Autentica um utilizador do sistema fornecendo as suas credenciais. Retorna os *tokens* JWT e respetivos metadados.

- **Dependência:** Nenhuma, por se tratar de rota pública.

- **Parâmetros de Entrada (Body JSON):**

```
{
  "username": "joao.treinador",
  "password": "SenhaSegura123!"
}
```

- **Resposta de Sucesso (200 OK):**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.rf...",
  "username": "joao.treinador",
  "role": "ROLE_TREINADOR",
  "nome": "João Silva"
}
```

2. Registo de Ocorrência Clínica

- **Método e Rota:** POST /api/v1/clinica/ocorrencias

- **Descrição:** Regista uma nova ocorrência clínica, como uma lesão, para um atleta específico.

- **Dependência:** Requer autenticação prévia com perfil de Médico (ROLE_MEDICO).

- **Parâmetros de Entrada (Body JSON):**

```
{
  "atletaId": 45,
  "tipo": "LESAO",
  "descricao": "Entorse no tornozelo direito durante o treino tático.",
  "grauRestricao": "INAPTO",
  "observacoesMedicas": "Repouso absoluto por 5 dias."
}
```

- **Resposta de Sucesso (201 Created):**

```
{
  "id": 102,
  "atletaId": 45,
  "tipo": "LESAO",
  "estadoOcorrencia": "ATIVA",
  "criadoEm": "2026-05-27T21:15:30Z"
}
```

3. Registo de Deliberação EMD

- **Método e Rota:** POST /api/v1/clinica/ocorrencias/{id}/deliberar
- **Descrição:** Emite a deliberação médica oficial quanto a um Exame Médico-Desportivo (EMD).
- **Parâmetros de Entrada (Body JSON):**

```
{
  "estadoElegibilidade": "APTO",
  "deliberacaoNotas": "Exames cardíacos normais. Atleta em condições
    ideais.",
  "validadeEmd": "2027-05-27"
}
```

4. Registo de Assiduidade (Chamada)

- **Método e Rota:** POST /api/v1/treinador/sesoes/{id}/chamada
- **Descrição:** Regista as presenças, ausências justificadas e faltas dos atletas na sessão.
- **Parâmetros de Entrada (Body JSON):**

```
{
  "registos": [
    { "atletaId": 45, "estado": "PRESENTE", "justificacao": null },
    { "atletaId": 46, "estado": "AUSENTE", "justificacao": "Consulta" }
  ]
}
```

5. Resumo Financeiro Consolidado (CFO)

- **Método e Rota:** GET /api/v1/cfo/resumo-financeiro
- **Descrição:** Retorna os KPIs macrofinanceiros consolidados do clube e da SAD de forma segregada.
- **Resposta de Sucesso (200 OK):**

```
{
  "clube": { "receitaTotal": 24500.50, "dividaPendente": 3100.00 },
  "sad": { "receitaTotal": 78900.00, "dividaPendente": 12400.00 },
  "global": { "receitaAcumulada": 103400.50, "taxaLiquidacao": 86.96 }
}
```

6. Direito ao Esquecimento (RGPD)

- **Método e Rota:** DELETE /api/v1/admin/utilizadores/{id}
- **Descrição:** Executa a anonimização irreversível dos dados pessoais identificáveis (PII) de um utilizador do sistema em conformidade com o RGPD.
- **Resposta de Sucesso:** 204 No Content, sem corpo de resposta.

2.1.4. Contratos de Interface (API Contracts)

Apresenta-se a especificação formal parcial das APIs do SIGD utilizando o formato **OpenAPI 3.0.3** em sintaxe **YAML**:

```
openapi: 3.0.3
info:
  title: Sistema de Gestão para Clube Desportivo (SIGD) - API Engine
  description: Especificação formal das APIs REST do SIGD.
  version: 1.0.0
servers:
  - url: http://localhost:8080/api/v1
    description: Servidor de Desenvolvimento Local
paths:
  /auth/login:
    post:
      summary: Autentica utilizador e emite Tokens JWT
      operationId: login
      tags:
        - Autenticação
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/LoginRequest'
      responses:
        '200':
          description: Login efetuado com sucesso
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/LoginResponse'
        '401':
          description: Credenciais incorretas

components:
```



```

securitySchemes:
  BearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT
schemas:
  LoginRequest:
    type: object
    required:
      - username
      - password
    properties:
      username:
        type: string
        example: joao.treinador
      password:
        type: string
        format: password
        example: SenhaSegura123!
  LoginResponse:
    type: object
    properties:
      accessToken:
        type: string
      refreshToken:
        type: string
      role:
        type: string

```

2.1.5. Justificação das Decisões Arquiteturais da API

A definição e desenho das APIs REST do SIGD foram fundamentados em boas práticas de engenharia de *software*. De seguida, justificam-se as principais decisões tomadas.

Escolha do Protocolo RESTful por Simplicidade e Desacoplamento

Optou-se por **REST sobre HTTP/S** como paradigma principal de comunicação. As razões foram:

- **Desacoplamento de Clientes:** Sendo o ecossistema composto por uma aplicação web e outra móvel, o REST garante que ambos consomem a mesma lógica sem acoplamento rígido de modelos de dados.
- **Cacheabilidade Eficiente:** O uso estrito dos verbos HTTP permite a utilização de

mecanismos nativos de cache HTTP em *proxies* reversos para rotas de leitura constante, como calendários e tabelas.

Autenticação *Stateless* baseada em *Tokens JWT*

A autenticação do utilizador é realizada de forma inteiramente ***Stateless*** recorrendo a JSON Web Tokens (JWT). Esta decisão favorece a **escalabilidade horizontal**, uma vez que elimina o estado de sessão em memória no servidor *Spring Boot*, permitindo distribuir a aplicação em múltiplos contentores sem necessidade de replicação de sessão.

Role-Based Access Control (RBAC) granular

Devido à presença de dados altamente sensíveis, como informação clínica e dados fiscais, as APIs implementam um modelo RBAC rigoroso. Através do *Spring Security*, valida-se o perfil de acesso antes de qualquer execução na base de dados, garantindo conformidade com regras de negócio críticas.

Paginação de Recursos e Performance

Para mitigar riscos de indisponibilidade por sobrecarga de CPU e rede, as consultas a grandes volumes de dados, como filas de EMD e listagens de atletas, utilizam paginação explícita através da interface *Pageable* do *Spring Data*, limitando o consumo de memória do servidor.

Segregação Financeira de Entidades Jurídicas (Clube vs. SAD)

As obrigações financeiras são classificadas sob a entidade jurídica **CLUBE** ou **SAD** de forma rígida diretamente na API, como no *endpoint* `/api/v1/cfo/resumo-financeiro`. Isto permite tratamento fiscal e contabilístico diferenciado, distinguindo quotas do clube e faturação associada à atividade comercial da SAD.

Conformidade com o RGPD (Anonimização Automatizada)

A especificação do *endpoint* administrativo DELETE `/api/v1/admin/utilizadores/{id}` afasta-se de um *hard delete* destrutivo tradicional, implementando um mecanismo de **anonimização irreversível**. Ao substituir informações pessoais identificáveis por representações anónimas, preservam-se as referências relacionais necessárias para estatísticas e auditoria, cumprindo as obrigações legais impostas pelo **RGPD**.

2.2. Prototipagem

A transposição da lógica de negócio, previamente especificada no documento SRS, para uma estrutura visual interativa constituiu uma etapa crucial para a validação da exequibilidade do SIGD. Este ciclo de desenvolvimento foi guiado por uma estratégia modular, desenhada para garantir a consistência estética e funcional em todo o ecossistema. Todo o conjunto de artefactos técnicos, incluindo os cadernos de encargos de módulo e as definições de *design*, encontra-se centralizado para consulta técnica em: <https://tinyurl.com/32xrp7a>.

2.2.1. Design System e Estruturação Modular

A fase inicial de prototipagem focou-se na mitigação da inconsistência visual que frequentemente surge em projetos de desenvolvimento incremental. Para assegurar que o SIGD fosse percecionado como uma plataforma unificada e não como um conjunto de módulos isolados, a equipa definiu um *Design System* centralizado, materializado no ficheiro `DESIGN.md`. Este referencial impôs regras estritas de estilização à ferramenta de geração, como paletas de cores semânticas em formato hexadecimal, comportamentos padronizados de componentes (sombras e *border-radius*) e diretrizes de tipografia.

Esta padronização permitiu que elementos comuns, como sistemas de navegação e *badges* de estado, apresentassem uma uniformidade rigorosa em todos os módulos do ERP. A estruturação do projeto seguiu uma abordagem *top-down*, onde primeiro se definiram as normas visuais globais, servindo estas de alicerce para a implementação específica de cada ecrã, garantindo assim que a interface, embora gerada de forma modular, respeitasse uma identidade visual coerente.

2.2.2. Especificação Técnica e Documentação de Módulo

Com o padrão visual definido, procedeu-se à elaboração de especificações individuais para cada módulo do sistema, estruturadas sob a forma de documentos de interface ('MODULE.md'). Estes ficheiros funcionaram como os "contratos" técnicos entre a equipa de engenharia e as ferramentas de IA, detalhando a anatomia funcional de cada ecrã.

Para cada perfil de utilizador (Direção, Secretaria, Equipa Técnica, Departamento Médico, Portal e Administração), foi gerado um mapeamento que ditou a disposição lógica das abas, os KPIs a exibir e as ações interativas permitidas. Esta documentação não serviu apenas para balizar a geração de código, mas também para assegurar a conformidade visual com a Matriz de Controlo de Acessos (RBAC) definida, garantindo que a hierarquia de privilégios fosse preservada na interface. Ao definir com exatidão a anatomia dos cartões, os estados de fluxo e as regras de negócio de cada formulário, estes documentos eliminaram interpretações arbitrárias por parte da IA, constituindo a base de documentação definitiva para a posterior fase de implementação técnica.

2.2.3. Execução Assistida por LLM

A transição final para a interface visual foi executada através de uma abordagem de desenvolvimento assistido, onde os modelos de linguagem atuaram como o motor de construção de *frontend*. A interação com as ferramentas de desenho foi regida por uma engenharia de *prompts* que forçou a IA a seguir estritamente as especificações contidas nos cadernos de encargos e no *Design System*.

A execução seguiu um fluxo de trabalho faseado e controlado. Dada a complexidade dos módulos, a equipa adotou uma estratégia de implementação incremental: a geração de código era solicitada por blocos lógicos (ex: Fase 1: *Frame* de navegação e *Layout* base; Fase 2: Fluxos transacionais e ecrãs de detalhe), garantindo que a densidade de funcionalidades fosse implementada sem comprometer os limites de contexto das ferramentas. Este processo de construção foi iterativo: após cada submissão, a equipa realizava uma auditoria humana para identificar desvios, aplicando correções cirúrgicas via *prompt* até atingir a alta fidelidade visual pretendida. Os *prompts* utilizados para desencadear estas construções, estruturados por persona, contexto e regras de execução, encontram-se no Anexo IX, servindo de testemunho da metodologia de controlo exercida sobre a IA.

2.2.4. Demonstração da Interface de Alta Fidelidade e Validação de Requisitos

Para validar a eficácia operacional do *Design System* e comprovar a correta transposição dos contratos técnicos definidos em sede de especificação, apresentam-se nesta subsecção os ecrãs mais representativos do protótipo final do SIGD. Estes artefactos comprovam a consistência de identidade visual e a aplicação nativa das regras de negócio em ambientes *desktop* e móveis.

A. Painel de Visão Executiva e Tomada de Decisão (CEO)

A Figura 2.1 apresenta a interface de alta fidelidade desenhada para o perfil de Presidência e CEO, materializada a partir do ficheiro de configuração *DESIGN.md*.

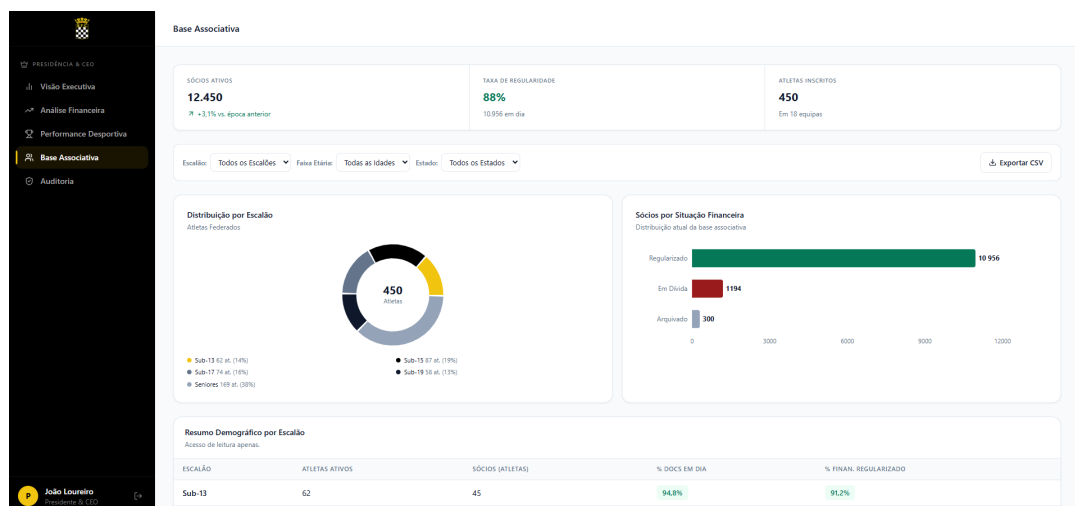


Figura 2.1.: Protótipo de Alta Fidelidade: Painel de Visão Executiva

O ecrã evidencia a consolidação macroscópica exigida para a gestão de topo do clube. O topo da interface é dominado por um módulo de *Alertas Estratégicos* que expõe em tempo real os indicadores de criticidade do sistema (como escalões em incumprimento de prazos ou rácios financeiros abaixo da meta).

Abaixo, os *widgets* financeiros e desportivos demonstram a segregação de dados do ERP: os fluxos de caixa são discriminados por canais de liquidação (Numerário, Multibanco e MBWay), enquanto o módulo de *Atletas Bloqueados* reflete de forma imediata a integração automática com as decisões da Secretaria e do Departamento Médico, segregando os impedimentos por natureza documental, clínica ou financeira.

B. Módulo de Validação Documental e Fluxos Administrativos (Secretaria)

O ecrã de *Validação Documental*, exposto na Figura 2.2, ilustra a transposição dos estados lógicos mapeados no modelo de domínio conceptual puro.

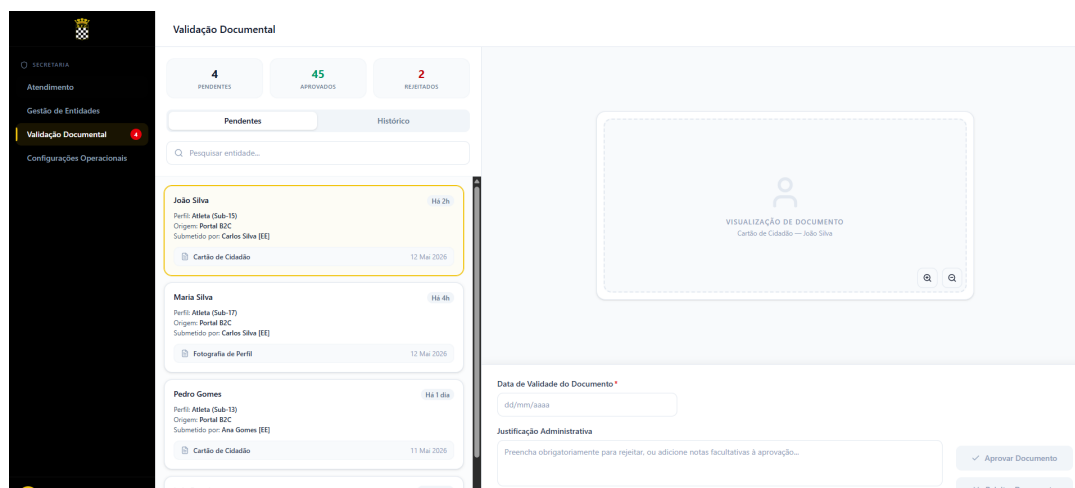


Figura 2.2.: Protótipo de Alta Fidelidade: Interface de Validação Documental

Esta interface condensa um fluxo transacional crítico: a receção, triagem e despacho de documentação oficial submetida remotamente pelos encarregados de educação. O topo esquerdo expõe contadores analíticos categorizados por estados de fluxo (*Pendentes*, *Aprovados* e *Rejeitados*).

A seleção de um processo na lista mestre ativa dinamicamente a área de trabalho central para visualização em miniatura do documento vetorial. O painel lateral direito materializa os constrangimentos regulamentares do clube, obrigando o utilizador a introduzir uma data de validade estrita e disponibilizando uma área de justificação administrativa que condiciona os botões de ação binária (*Aprovar* ou *Rejeitar Documento*), eliminando arbitrariedades no tratamento burocrático.

C. Controlo de Operações de Relvado e Chamada Digital (Equipa Técnica)

Para os ecrãs de campo, a equipa adotou uma abordagem de interface móvel ajustada ao contexto físico do *Treinador*. A Figura 2.3 demonstra a aplicação das regras de assiduidade e elegibilidade clínica em tempo real.

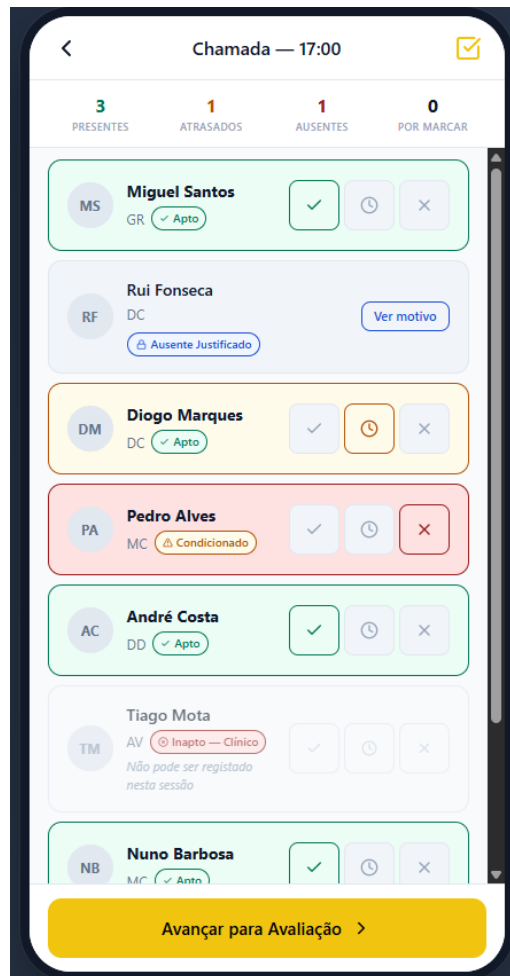


Figura 2.3.: Protótipo Móvel: Registo de Chamada e Estado Clínico

O ecrã de *Chamada* traduz graficamente os estados abstratos concebidos na engenharia de requisitos. Os atletas são listados com *badges* semânticos de prontidão desportiva (*Apto*, *Ausente Justificado*, *Condicionado* ou *Inapto*). O rigor arquitetural do sistema é visível no tratamento do caso do atleta Tiago Mota: categorizado pelo departamento médico com o estado *Inapto - Clínico*, o sistema desativa os seletores de presença e emite um aviso explícito de bloqueio, impedindo fisicamente o treinador de violar a restrição de segurança médica do clube.

D. Módulo de Avaliação Técnico-Tática Longitudinal (Equipa Técnica)

Fechando o ciclo diário de treino, a Figura 2.4 apresenta a interface móvel desenvolvida para o preenchimento da ficha de rendimento desportivo.



Figura 2.4.: Protótipo Móvel: Lançamento de Avaliação Quantitativa

Esta interface espelha de forma milimétrica o refinamento de métricas operado na secção 1.12.2 [REQ-03]. O seletor de classificação quantitativa encontra-se limitado ao intervalo exato de 1.0 a 5.0 com incrementos discretos de 0.5 pontos através de controlos mecânicos de incremento e decremento (+ e –).

Adicionalmente, cumpre-se a regra de tratamento de exceções estatísticas: o rodapé da aplicação adverte explicitamente o utilizador de que a ausência de atribuição de nota submeterá o registo sob o estado lógico de '*Não Avaliado*', salvaguardando a integridade matemática dos cálculos e médias longitudinais da base associativa contra distorções provocadas por omissões humanas.

2.3. Arquitetura do Sistema e Decisões Técnicas

Nesta secção documentam-se as decisões arquiteturais fundamentais tomadas pela equipa para sustentar a implementação do SIGD, assegurando coerência entre os requisitos funcionais, as restrições tecnológicas e os constrangimentos operacionais do projeto.

ID	ADR-01: Padrão Arquitetural
Contexto	O SIGD é um sistema de gestão com regras de negócio financeiras fortes, controlo de acessos (RBAC) rigoroso e necessidade de segregação de dados entre Clube e SAD.
Opções Consideradas	Monólito Tradicional, Monólito Modular, Arquitetura de Microserviços.
Decisão	Layered Architecture (Apresentação → Serviço → Persistência) encapsulada num Monólito Modular organizado por domínio em packages Spring Boot.
Justificação	Com uma equipa de 4 elementos, os microserviços trariam um <i>overhead</i> de infraestrutura (<i>over-engineering</i>) insustentável. O Monólito Modular garante a coesão necessária e permite separar logicamente as áreas (Tesouraria, Clínica, Equipa Técnica) sem a complexidade de redes distribuídas.

Tabela 2.2.: Especificação do ADR-01.

ID	ADR-02: Stack Tecnológica Base
Contexto	Necessidade de desenvolver uma solução acessível tanto em desktop (Secretaria/Direção) como em mobile (Treinadores/Atletas no relvado), sustentada por um backend robusto.
Decisão	Frontend: React Native (<i>Cross-platform</i> com Expo) para abranger web, iOS e Android com a mesma base de código. Backend: Java Spring Boot (REST API). Base de Dados: MySQL. ORM: JPA/Hibernate via Spring Data.
Justificação	O React Native garante o requisito <i>mobile-first</i> exigido pelas operações de campo. O ecossistema Spring Boot + MySQL é o standard da indústria para sistemas ERP, oferecendo maturidade e vasta documentação.

Tabela 2.3.: Especificação do ADR-02.

ID	ADR-03: Autenticação e Autorização
Contexto	A plataforma possui atores com níveis de acesso drasticamente distintos, desde o Portal do Utilizador até à Direção Executiva.
Decisão	Implementação de autenticação <i>stateless</i> com JWT (<i>Access Token</i> + <i>Refresh Token</i>) e autorização via Spring Security. As <i>roles</i> serão embutidas no <i>payload</i> do JWT.
Justificação	O JWT elimina a necessidade de gestão de sessões no servidor, facilitando a escalabilidade. As <i>roles</i> estritas (ROLE_ADMIN, ROLE_CEO, ROLE_CFO, ROLE_SECRETARIA, ROLE_TREINADOR, ROLE_DT, ROLE_MEDICO, ROLE_UTILIZADOR) mapeiam diretamente os atores identificados nos Casos de Uso, garantindo um controlo RBAC direto nas rotas da API.

Tabela 2.4.: Especificação do ADR-03.

ID	ADR-04: Segregação Societária e Financeira (Clube vs. SAD)
Contexto	O requisito RNF-27 obriga à separação rigorosa de dados e capitais entre a Associação Desportiva e a SAD. Estando a base de dados assente em MySQL.
Decisão	Adição de uma coluna discriminadora <i>entidade_juridica</i> (Enum: CLUBE, SAD) nas entidades relevantes. A segregação será forçada na camada de aplicação através de filtros do Hibernate (@Filter) e validações na camada de <i>Service</i> .
Justificação	Sendo a stack em MySQL, a filtragem ao nível do ORM/aplicação é a alternativa mais viável. O uso de @Filter no Hibernate permite aplicar automaticamente a cláusula WHERE <i>entidade_juridica</i> = ... em todas as <i>queries</i> dessa sessão, mitigando o risco de um programador se esquecer de colocar a condição manual nos repositórios.

Tabela 2.5.: Especificação do ADR-04.

ID	ADR-05: Registo de Auditoria (Audit Trail)
Contexto	O requisito US41 exige um histórico inalterável de ações críticas, como altas médicas ou anulações financeiras.
Decisão	Implementação de uma tabela <i>append-only</i> (<code>audit_log</code>) populada através de JPA <code>@EntityListeners</code> (<code>@PrePersist</code> , <code>@PreUpdate</code> , <code>@PreRemove</code>). Nenhuma operação de DELETE ou UPDATE é permitida nesta tabela.
Justificação	O uso de <i>Entity Listeners</i> do JPA é declarativo, acoplado diretamente às entidades que precisam de auditoria, reduzindo a complexidade face a soluções baseadas em Spring AOP.

Tabela 2.6.: Especificação do ADR-05.

ID	ADR-06: Notificações e Tarefas Automáticas
Contexto	O sistema necessita de executar rotinas automáticas como verificar a caducidade de Exames Médico-Desportivos ou fechar estatísticas de jogo 24h após o evento.
Decisão	Utilização do agendador nativo do Spring Boot, através de <i>Cron Jobs</i> com <code>@Scheduled</code> .
Justificação	Trata-se de uma solução <i>out-of-the-box</i> e de baixo atrito arquitetural. Não requer filas de mensagens externas, como RabbitMQ ou Kafka, mantendo a infraestrutura simples e alinhada com os recursos e a dimensão da equipa.

Tabela 2.7.: Especificação do ADR-06.

ID	ADR-07: Versionamento e Contrato da API
Contexto	Existe a necessidade de padronizar a comunicação entre a aplicação React Native e o backend Spring Boot.
Decisão	Prefixo universal <code>/api/v1/</code> para todas as rotas; formato de comunicação estrito em <code>application/json</code> ; paginação via <i>query parameters</i> (ex.: <code>?page=0&size=20</code>); padronização de erros seguindo a RFC 7807 (<i>Problem Details for HTTP APIs</i>) ou formato standard equivalente (<code>status</code> , <code>error</code> , <code>message</code>).
Justificação	Esta decisão estabelece um contrato sólido desde o início do projeto, facilita o tratamento de erros no <i>frontend</i> e previne problemas de <i>parsing</i> e integração entre as várias componentes da solução.

Tabela 2.8.: Especificação do ADR-07.

2.4. Modelação e Arquitetura do Sistema

A fase de modelação visual constitui o alicerce arquitetural prévio à implementação técnica. Com o objetivo de evitar uma transição direta e não planeada da especificação de requisitos para a escrita de código fonte, adotou-se uma abordagem prescritiva baseada em modelos (UML) para mapear o domínio, perspetivar as interações sistémicas e estipular as fronteiras dos componentes. Os diagramas seguintes foram concebidos na fase de desenho da solução para definir as regras estruturais e garantir a total rastreabilidade entre a lógica de negócio desportiva e a futura infraestrutura tecnológica.

2.4.1. Diagrama de Classes

O diagrama de classes especifica o modelo de domínio estático projetado para o sistema, desenhado em estrita conformidade com as diretrizes de *Domain-Driven Design* (DDD). Este modelo não atua apenas como documentação funcional, mas dita a taxonomia formal que guiará o futuro mapeamento relacional (*Object-Relational Mapping* - ORM) a implementar no backend.

A estruturação gráfica propõe a separação lógica das entidades nos eixos nucleares do clube:

- **Módulo Administrativo e Tesouraria:** Estipula os relacionamentos de dependência e herança funcional entre Utilizador, EncarregadoEducacao e a ramificação de ObrigacaoFinanceira.
- **Módulo Desportivo:** Define a taxonomia principal com a agregação de entidades Atleta num Plantel ou Equipa, e a respetiva alocação operacional a eventos dinâmicos planeados, como SessaoTreino ou FichaJogo.
- **Módulo Clínico:** Estabelece o mapeamento da OcorrenciaClinica e a sua dependência existencial obrigatória face ao perfil biográfico do atleta, suportando a lógica planeada para o semáforo de aptidão.

2.4.2. Diagramas de Sequência

Para planear o comportamento dinâmico e o ciclo de vida das transações do sistema, modelaram-se diagramas de sequência focados nos fluxos críticos. A topologia visual prescreve a arquitetura em camadas a adotar no backend (*N-Tier Architecture*), segregando as responsabilidades de forma atómica: a interface projetada com o cliente (Frontend), a porta de entrada da API REST (*Controller*), a lógica de negócio orquestrada (*Service*) e o acesso persistente a dados (*Repository/DB*).

A ?? perspetiva a mecânica central do fluxo de Autenticação e Autorização. O diagrama

clarifica a submissão de credenciais esperada pelo ator, a delegação da validação criptográfica para a camada de serviços e a interação pretendida com a base de dados. Em caso de sucesso, dita a subsequente emissão de um *JSON Web Token (JWT) stateless* para retenção no cliente mobile. O recurso a fragmentos de interação *alt* garante a definição prévia de desvios de exceção estritos (como tentativas de acesso com credenciais inválidas).

2.4.3. Diagrama de Componentes

O diagrama de componentes abstrai a micro-lógica interna para fornecer o planeamento macroscópico da topologia de *deployment* e da organização modular. Este artefacto dita como os futuros blocos de software deverão interoperar no ecossistema final, justificando as decisões de separação tecnológica tomadas na fase de arquitetura.

A modelação estipula três grandes blocos concêntricos em rede:

1. **Camada de Apresentação (Client-Side):** A aplicação a desenvolver sob o ecossistema Expo/React Native, planeada para assumir o *routing* do lado do cliente, a renderização de interfaces adaptativas e a gestão de estado de sessão.
2. **Camada Lógica (Server-Side):** O servidor central a construir em Spring Boot (Java), focado na exposição de interfaces consumíveis (*Endpoints REST*) que deverão ser protegidas pelos filtros de segurança.
3. **Camada de Dados (Persistence):** A infraestrutura de dados projetada para o MySQL e os respetivos volumes persistentes, cujo acesso será isolado e consumido exclusivamente através do padrão *Data Access Object* do servidor principal.

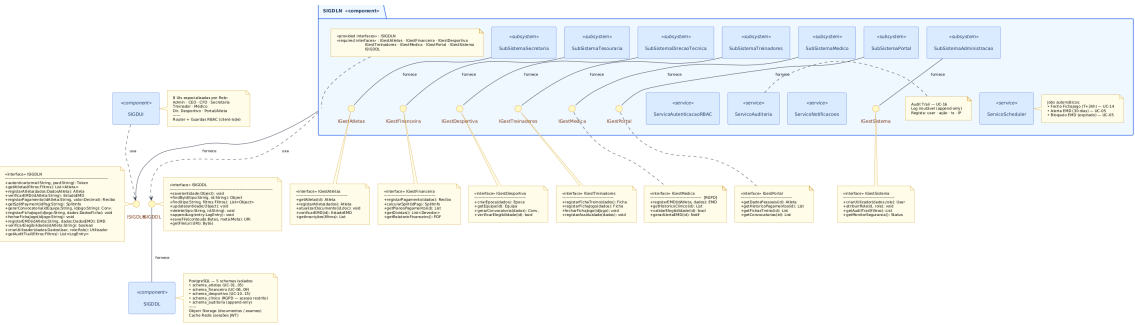


Figura 2.5.: Diagrama de componentes planejado para o SIGD, ilustrando o encapsulamento modular estipulado e a futura topologia de integração e persistência.

Capítulo 3 - Implementação e Desenvolvimento Assistido por LLM

A terceira etapa do projeto materializou a transição do plano conceptual para o sistema funcional. Com base na arquitetura definida na Etapa 2 e nos requisitos especificados no SRS, a equipa procedeu à implementação incremental do SIGD, adotando uma metodologia de desenvolvimento assistido por LLM que permeou todas as fases do processo — desde a configuração do ambiente até à produção da documentação técnica de suporte às etapas de verificação e validação.

3.1. Configuração do Ambiente de Trabalho

A primeira tarefa da fase de implementação consistiu na definição e estabilização do ambiente de desenvolvimento partilhado pela equipa. Esta decisão não foi meramente operacional: a homogeneidade do ambiente entre os diferentes postos de trabalho revelou-se uma condição crítica para a reprodutibilidade dos resultados e para a eliminação de erros de integração decorrentes de divergências de configuração.

3.1.1. Stack Tecnológica e Justificação

A *stack* adotada resultou diretamente das decisões de arquitetura da Etapa 2, tendo sido validada pela sua maturidade ecossistémica, pela disponibilidade de documentação e pela adequação aos requisitos não-funcionais especificados no SRS. A Tabela 3.1 sintetiza os componentes principais e a sua função no sistema.

Tabela 3.1.: Componentes principais da *stack* tecnológica adotada no SIGD.

Componente	Tecnologia	Versão	Função
Linguagem Backend	Java	21 (LTS)	Lógica de negócio e API REST
Framework Backend	Spring Boot	3.4.x	Injeção de dependências, segurança, ORM
Build	Maven	3.9.6	Gestão de dependências e compilação
ORM / Validação	Hibernate	6.6.2	Mapeamento objeto-relacional
Migrações de BD	Flyway	integrado	Versionamento evolutivo do esquema
Base de Dados	MySQL	8	Persistência relacional
Containerização	Docker / Compose	27+ / 2.0+	Orquestração de serviços
Framework Frontend	React Native + Expo	SDK 51	Interface multiplataforma (web + mobile)
Linguagem Frontend	TypeScript	5.x	Tipagem estática e manutenibilidade
Gestor de Pacotes	npm	10.x	Dependências frontend

A escolha do Java 21 LTS com Spring Boot 3.4.x garantiu estabilidade a longo prazo e compatibilidade nativa com os mecanismos de segurança exigidos pelo RNF-22 (autenticação JWT e controlo de acessos por RBAC). A opção por React Native com Expo decorreu da necessidade de suportar tanto *browsers* web como dispositivos móveis a partir de uma única base de código, respondendo diretamente ao RF-41 (acessibilidade multiplataforma) sem duplicação de esforço de desenvolvimento.

3.1.2. Orquestração de Serviços com Docker

Para garantir a paridade de ambiente entre os postos de trabalho da equipa — com sistemas operativos e configurações distintas — todos os serviços de infraestrutura foram containerizados com Docker Compose. A solução encapsula três serviços orquestrados numa rede privada isolada: o `sigd-mysql`, uma instância MySQL 8 com volume persistente, exposta na porta 3306, com verificação de saúde (*healthcheck*) integrada que bloqueia o arranque dos serviços dependentes até à disponibilidade total da base de dados; o `sigd-phpmyadmin`, interface gráfica de gestão da base de dados exposta na porta 8081, utilizada para inspeção e intervenção manual durante o desenvolvimento; e a rede privada `backend_sigd-network`, que isola os serviços de infraestrutura da rede do *host*, eliminando conflitos de porta entre projetos paralelos.

3.1.3. Automatização do Arranque — Script `start.ps1`

Um dos problemas identificados precocemente foi a fricção operacional associada ao arranque manual do ambiente em cada sessão de trabalho e, em particular, a dificuldade em garantir que todos os membros da equipa partilhavam sempre o mesmo estado da base de dados antes de uma sessão de testes ou de uma demonstração.

Para eliminar esta fonte de erro humano, foi desenvolvido um script PowerShell unificado (`start.ps1`) que automatiza a totalidade do ciclo de arranque. A sua execução segue uma sequência determinística e tolerante a falhas: terminação e remoção de contentores Docker anteriores, garantindo um estado limpo; criação e arranque da rede e contentores; ciclo de espera ativa com verificação do *healthcheck* do MySQL (até 30 tentativas com intervalo de 5 segundos), abortando com mensagem de erro em caso de *timeout*; pergunta interativa ao operador sobre a reposição do conjunto de dados de demonstração, injetando o `seed_master_demo.sql` directamente no contentor em caso afirmativo; e arranque do backend Spring Boot e do frontend Expo em janelas de terminal independentes.

Esta abordagem tornou a reposição do ambiente de demonstração num processo de um único comando, eliminando erros de configuração entre elementos da equipa e garantindo a reprodutibilidade total dos cenários de teste.

3.1.4. Estratégia de Configuração e Perfis

A configuração do backend foi segmentada em perfis Spring (`application.yml` e `application-dev.yml`), separando os parâmetros de produção dos de desenvolvimento. Uma decisão técnica com impacto direto na qualidade do processo foi a configuração da propriedade `spring.jpa.hibernate.ddl-auto: validate` no perfil de desenvolvimento.

Esta configuração instrui o Hibernate a validar o esquema da base de dados contra o modelo de entidades Java no arranque, falhando imediatamente se existirem divergências — tabelas em falta, colunas com tipos incorretos ou restrições violadas. A alternativa `update`, que cria ou altera tabelas automaticamente, foi deliberadamente rejeitada: ao obrigar a equipa a manter as migrações Flyway sempre atualizadas e a aplicá-las explicitamente, o modo `validate` funcionou como mecanismo de deteção precoce de inconsistências, prevenindo que erros de esquema chegassem silenciosamente às fases de teste.

3.2. Estratégia de Desenvolvimento Modular

3.2.1. Decomposição do Sistema em Módulos Funcionais

A estrutura do SIGD foi organizada em seis módulos funcionais independentes, cada um correspondendo a um perfil de utilizador e a um domínio de negócio delimitado. Esta decomposição, estabelecida na fase de arquitetura, foi preservada e reforçada durante a implementação, traduzindo-se numa separação física de pacotes no backend e de ecrãs no frontend. A Tabela 3.2 sintetiza o mapeamento entre módulos, roles e domínios.

Tabela 3.2.: Mapeamento dos módulos funcionais e domínios de negócio do SIGD.

Módulo	Role Principal	Domínio de Negócio
Secretaria	ROLE_SECRETARIA	Gestão de atletas, encargados e documentação
Clínica	ROLE_MEDICO	Ocorrências, EMD e deliberações de aptidão
Treinador	ROLE_TREINADOR	Sessões, convocatórias, presenças e fichas de jogo
Portal EE	ROLE_EE	Consulta de obrigações, agenda e estado clínico
Executivo	ROLE_CEO, ROLE_CFO, ROLE_DIRETOR_TECNICO	Dashboards, KPIs financeiros e desportivos
Admin	ROLE_ADMIN	Gestão de utilizadores, auditoria e configuração

A correspondência direta entre módulos e *roles* de utilizador não foi apenas uma decisão organizacional — foi a base estrutural do mecanismo de controlo de acessos (RBAC) implementado com Spring Security. Cada *endpoint* da API foi anotado com a *role* mínima necessária para o seu acesso, garantindo que a segregação de funcionalidades no frontend fosse reforçada igualmente ao nível da camada de serviço.

3.2.2. Ordem de Implementação e Gestão de Dependências

A ordem de implementação dos módulos foi determinada por duas variáveis: as dependências técnicas entre domínios e a prioridade de negócio estabelecida pelos requisitos. Não sendo possível implementar convocatórias sem atletas, nem fichas de jogo sem eventos desportivos, a sequência respeitou um grafo de dependências implícito no modelo de dados.

A implementação seguiu a seguinte progressão lógica. Em primeiro lugar, a infraestrutura

transversal — autenticação JWT, RBAC, tratamento global de exceções e auditoria — constituiu a fundação sem a qual nenhum módulo de negócio poderia ser validado com segurança. Em segundo lugar, o módulo de Secretaria foi implementado por ser a fonte de dados primária (atletas, encarregados, equipas) de que todos os outros dependem. O módulo Clínico seguiu-se por ser dependente dos atletas da Secretaria e por definir os estados de elegibilidade que condicionam o módulo do Treinador. Este último, o mais extenso em termos de fluxos, foi implementado após a estabilização dos dois módulos anteriores. O Portal EE, como consumidor de dados dos três módulos precedentes sem lógica de escrita complexa, foi o quinto a ser desenvolvido. Os módulos Executivos foram implementados por último, por dependerem da existência de dados reais gerados pelos módulos operacionais. O módulo Admin foi desenvolvido em paralelo com os restantes, por fornecer ferramentas de suporte transversal.

3.2.3. Ciclo de Desenvolvimento por Módulo

Para cada módulo, a equipa adotou um ciclo de desenvolvimento incremental composto por quatro fases sequenciais, garantindo que cada camada era validada antes de a superior ser construída. A primeira fase, de modelação e migração, consistiu na definição das entidades JPA e na criação do ficheiro de migração Flyway correspondente, aplicado manualmente e validado pelo modo `validate` do Hibernate no arranque. A segunda fase implementou a camada de serviço e repositório, com cobertura de testes unitários gerada com assistência do LLM. A terceira fase expôs os *endpoints* REST com anotações de segurança RBAC e validação de *inputs*, coberta por testes de integração com `MockMvc`. A quarta e última fase implementou os ecrãs de frontend e a integração com a API, com validação manual dos fluxos de utilizador contra os critérios de aceitação das *User Stories*.

Esta disciplina de implementação em camadas foi o principal fator que permitiu à equipa manter a cobertura de testes e a estabilidade do sistema ao longo de um processo de desenvolvimento com 48 *bugs* corrigidos e 15 funcionalidades adicionadas após a entrega inicial.

3.3. Geração de Código Assistida por LLM

3.3.1. Papel do LLM no Processo de Implementação

Ao contrário das etapas anteriores, onde o LLM assumiu predominantemente um papel de extração e modelação de conhecimento, na fase de implementação o seu papel expandiu-se para três funções distintas e complementares: gerador, revisor e depurador.

Enquanto gerador, o modelo foi utilizado para produzir código estrutural repetitivo — entidades JPA, repositórios Spring Data, DTOs, ficheiros de migração Flyway e componentes React Native — libertando a equipa para se concentrar nas decisões de lógica de negócio não triviais. Enquanto revisor, foi utilizado para auditar código existente antes de o modificar, identificando

efeitos colaterais e dependências ocultas. Enquanto depurador, foi o primeiro recurso na análise de erros de compilação, falhas de testes e comportamentos inesperados em *runtime*, acelerando significativamente os ciclos de correção.

É importante sublinhar que o LLM não substituiu o julgamento técnico da equipa — funcionou como um amplificador de capacidade. Todas as sugestões geradas foram revistas, validadas e, em vários casos, corrigidas antes de serem integradas na base de código. Esta supervisão humana ativa foi particularmente crítica em decisões de segurança e em lógica de negócio sensível, como o cálculo de estados de elegibilidade e a segregação financeira Clube/SAD.

3.3.2. Metodologia de Interação — Engenharia de Prompts

A experiência acumulada nas etapas anteriores do projeto permitiu à equipa desenvolver um conjunto de padrões de interação adaptados especificamente ao contexto de desenvolvimento de software. A principal lição incorporada foi a de que a qualidade do *output* do LLM é diretamente proporcional à quantidade de contexto fornecido e à especificidade das restrições impostas.

Foram adotadas três regras transversais a todos os *prompts* de implementação. A primeira, de leitura obrigatória antes de qualquer escrita, determinava que todos os *prompts* que implicavam a modificação de ficheiros existentes fossem iniciados com uma instrução explícita de leitura dos ficheiros relevantes, eliminando uma classe inteira de erros em que o modelo gerava código inconsistente com interfaces já definidas:

```
"ANTES DE ESCREVER QUALQUER CODIGO:
1. Le COMPLETO: [ficheiro A].
2. Le COMPLETO: [ficheiro B].
3. Lista: [directorio X]."
```

A segunda regra impunha restrições negativas explícitas. Para além de especificar o que deveria ser feito, os *prompts* incluíam sistematicamente instruções sobre o que não deveria ser alterado, prevenindo modificações acidentais em código estável em módulos com elevada interdependência:

```
"Nao alteres STATUS.md nem AUDIT.md. Nao modifique o
SecurityConfig.java. Nao alteres ficheiros fora do pacote indicado."
```

A terceira regra introduzia instruções de paragem e validação em sequências de tarefas complexas, garantindo que erros numa etapa não se propagavam silenciosamente para as seguintes:

```
"Apos implementar, compila com mvn compile e cola o output. Escreve
'ETAPA X CONCLUIDA - aguardo confirmacao para avancar.' Nao passes
a etapa seguinte sem instrucao explicita."
```

3.3.3. Padrões de Prompt por Tipo de Tarefa

A equipa identificou quatro categorias recorrentes de tarefas de desenvolvimento, para cada uma das quais foi estabilizado um *template* de instrução. Estes *templates* encontram-se documentados integralmente nos Anexos X a XIII. De forma a ilustrar a metodologia, apresenta-se de seguida um exemplo representativo da categoria de maior complexidade — a implementação de um novo módulo funcional completo. O *prompt* seguinte foi utilizado para iniciar a implementação do Módulo do Treinador, após a estabilização dos módulos de Secretaria e Clínica:

```
"ANTES DE ESCREVER QUALQUER CODIGO:
1. Le COMPLETO: src/main/java/com/sigd/core/model/Atleta.java
2. Le COMPLETO: src/main/java/com/sigd/core/model/Equipa.java
3. Lista: src/main/java/com/sigd/treinador/
CONTEXTO - Modulo Treinador: Este modulo serve o utilizador com
ROLE_TREINADOR. As suas responsabilidades centrais sao: (1) gestao
de sessoes de treino com registo de presencas e avaliacoes de
rendimento; (2) criacao e publicacao de convocatorias para eventos
desportivos; (3) submissao de fichas de jogo com resultado e
observacoes.
TAREFA 1 - Criar entidade SessaoTreino com os campos: id, equipa
(FK), data, horaInicio, horaFim, tipo [TREINO|TATICO|FISICO|
AQUECIMENTO], estado [PLANEADA|CONCLUIDA|CANCELADA], criadoEm.
TAREFA 2 - Criar SessaoTreinoRepository com: findByEquipaId,
findByEquipaIdAndData, findByEquipaIdAndEstado.
TAREFA 3 - Criar SessaoTreinoService com: listarPorEquipa,
obterDetalhe, concluir.
TAREFA 4 - Criar SessaoTreinoController com endpoints GET /sessoes
e PATCH /sessoes/{id}/concluir, ambos com
@PreAuthorize("hasRole('ROLE_TREINADOR')").
TAREFA 5 - Compilar com mvn compile e colar o output.
Nao alteres ficheiros fora do pacote treinador.
Nao alteres o SecurityConfig.java."
```

O catálogo integral de *prompts* utilizados ao longo da implementação encontra-se disponível no repositório do projeto através da ligação indicada no Anexo XIII.

3.3.4. Limitações Identificadas e Estratégias de Mitigação

A utilização intensiva do LLM ao longo de vários meses de desenvolvimento permitiu à equipa identificar e documentar um conjunto de limitações recorrentes, para cada uma das quais foram adotadas estratégias de mitigação específicas.

A deriva de contexto em conversas longas (*context drift*) foi a limitação mais frequente: em sessões de desenvolvimento prolongadas, o modelo tendia a perder coerência com decisões

tomadas no início da conversa, reintroduzindo padrões de código já corrigidos ou ignorando restrições anteriormente estabelecidas. A mitigação adoptada foi a compactação periódica do contexto — no início de cada nova sessão, o modelo era realimentado com um sumário estruturado do estado actual do sistema, preservando a continuidade sem depender da memória da sessão anterior.

A alucinação de nomes de métodos e APIs manifestava-se através da geração de código que referenciava métodos inexistentes em repositórios ou versões de APIs entretanto deprecadas. A Regra 1 descrita na secção anterior — leitura obrigatória dos ficheiros antes de qualquer geração — garantia que o modelo operava sobre o estado real do código.

A tendência para sobre-engenharia levava o modelo, sem restrições explícitas, a introduzir abstrações desnecessárias (camadas de *mapper*, interfaces redundantes, configurações de *cache*) que aumentavam a complexidade sem acrescentar valor funcional. A restrição “Zero Over-Engineering”, aplicada sistematicamente nos *prompts*, conteve este comportamento.

Por fim, a modificação de ficheiros fora do escopo ocorria quando, em tarefas de correção de *bugs*, o modelo propunha alterações em ficheiros não relacionados com o problema, introduzindo regressões em funcionalidades estáveis. As restrições negativas explícitas e a instrução de compilação após cada tarefa funcionaram como rede de segurança.

3.4. Implementação do Backend

3.4.1. Estrutura de Pacotes e Arquitetura em Camadas

A estrutura interna do backend seguiu uma organização em camadas horizontais dentro de módulos verticais, combinando os benefícios do isolamento por domínio de negócio com a clareza da separação de responsabilidades técnicas. Cada módulo funcional foi implementado como um pacote autónomo com estrutura interna idêntica, contendo as subcamadas de *controller*, *service* e DTOs. Os modelos e repositórios partilhados entre módulos residem no pacote *core*, garantindo que as dependências entre módulos fluam sempre através desta camada central e nunca directamente entre módulos de negócio — um princípio de arquitectura limpa que facilitou a testabilidade isolada de cada componente.

3.4.2. Gestão Evolutiva do Esquema com Flyway

O versionamento da base de dados foi gerido através de migrações Flyway, armazenadas em ficheiros SQL numerados sequencialmente. Esta abordagem garantiu que qualquer membro da equipa, ao clonar o repositório e iniciar o ambiente, obtinha exactamente o mesmo esquema de base de dados. A Tabela 3.3 detalha a evolução do esquema ao longo do projeto.

Tabela 3.3.: Mapeamento da evolução do esquema de base de dados através das migrações Flyway.

Migração	Conteúdo	Contexto
V1	Esquema base (utilizador, atleta, equipa, escalão, encarregado)	Módulo Secretaria
V2	Tabela obrigacao_financeira	Módulo Financeiro
V3	Tabelas ocorrencia e ocorrencia_evolucao	Módulo Clínico
V4	Seed inicial de utilizadores de sistema	Configuração de ambiente
V5	Seed de dados de demonstração base	Validação de módulos
V6	Tabelas do módulo Treinador	Módulo Treinador
V7	Campos de <i>lockout</i> na tabela utilizador	Correção segurança RNF-25
V8	Campo federado na tabela atleta	Requisito CFO/DT
V9	Campo data_validade_emd na tabela atleta	Requisito Clínico
V10	Tabelas ficha_jogo_titular e evento_jogo	Expansão módulo Treinador

Um padrão operacional importante foi estabelecido durante o desenvolvimento: em ambiente de desenvolvimento com `ddl-auto: validate`, cada nova migração tinha de ser aplicada manualmente ao contentor MySQL antes do arranque do backend. O script de arranque automatizado foi posteriormente estendido para incorporar esta etapa, eliminando a fricção associada à aplicação manual.

3.4.3. Segurança — Autenticação, Autorização e Proteções Transversais

A implementação da camada de segurança constituiu uma das fases mais críticas do backend, dada a natureza sensível dos dados processados pelo sistema — incluindo dados clínicos de menores de idade e informação financeira. A arquitectura de segurança assentou em quatro pilares.

O primeiro é a autenticação por JWT: cada pedido autenticado transporta um token JWT assinado com chave HMAC-SHA256, gerado no momento do *login* e validado por um filtro Spring Security (`JwtAuthenticationFilter`) interposto antes de qualquer lógica de negócio, encapsulando o identificador do utilizador e o seu *role* e eliminando a necessidade de consultas à base de dados em cada pedido. O segundo é o controlo de acessos por RBAC: todos os *endpoints* foram anotados com `@PreAuthorize`, definindo explicitamente o *role* mínimo necessário para o seu acesso, com uma política de *deny-by-default* que rejeita com HTTP 403 qualquer *endpoint* não explicitamente autorizado. O terceiro é a proteção contra ataques de força bruta: em resposta ao RNF-25, foi implementado um mecanismo de *lockout* progressivo que, após

cinco tentativas de autenticação falhadas consecutivas, bloqueia a conta por um período configurável, com o estado persistido na base de dados através dos campos `tentativas_falhadas` e `bloqueado_ate` introduzidos na migração V7. O quarto é a sanitização de *inputs*: todos os campos de texto livre são processados através de um componente `SanitizadorHtml` baseado na biblioteca `Jsoup` antes da persistência, com validação estrutural dos *payloads* assegurada pelas anotações `Bean Validation` e um `GlobalExceptionHandler` que converte as violações em respostas HTTP 400 estruturadas.

3.4.4. Lógica de Negócio — Decisões Técnicas por Módulo

A implementação dos módulos de negócio implicou um conjunto de decisões técnicas não triviais. No módulo Clínico/Treinador, o estado de elegibilidade de um atleta (`APTO`, `CONDICIONADO`, `INAPTO`, `PENDENTE_EMD`) é calculado dinamicamente pelo `SemaforoService` com base no grau de restrição da ocorrência clínica ativa e na validade do EMD, com o estado resultante persistido no campo `estado_elegibilidade` da entidade `Atleta` para evitar *joins* complexos em *runtime*. No módulo do Treinador, o estado de um jogo (ficha pendente, prazo expirado, ficha submetida) é calculado dinamicamente no `FichaJogoService` com base na data e hora do evento, sem depender do campo estado da base de dados — decisão motivada pela constatação de que alterar manualmente a hora de um jogo não actualizava o estado persistido. No módulo Financeiro, a separação de obrigações por entidade jurídica (`CLUBE` vs. `SAD`) é implementada ao nível do modelo de dados através do campo `entidade_juridica`, com os *endpoints* do CFO a filtrar e agregar separadamente por este campo. Por fim, o cálculo de minutos jogados por atleta, necessário para as estatísticas individuais de utilização desportiva, é executado no `FichaJogoService` a partir dos eventos de substituição registados na ficha, sem persistência intermediária.

3.4.5. Tratamento Global de Exceções e Padronização de Respostas

Um dos requisitos de qualidade transversais ao backend foi a consistência das respostas de erro. Para concretizar este requisito, foi implementado um `GlobalExceptionHandler` com `@ControllerAdvice` que interceta e mapeia as exceções mais relevantes para os códigos HTTP semânticos correspondentes, conforme sintetizado na Tabela 3.4.

Tabela 3.4.: Mapeamento global de exceções do sistema para códigos de estado HTTP.

Exceção	HTTP	Contexto Típico
EntityNotFoundException	404	Recurso não encontrado por ID
IllegalArgumentException	400	<i>Input</i> inválido
IllegalStateException	409	Conflito de estado
AccessDeniedException	403	<i>Role</i> insuficiente
MethodArgumentNotValidException	400	Violação de Bean Validation
Exception (<i>fallback</i>)	500	Erro interno não antecipado

Todas as respostas de erro seguem uma estrutura JSON uniforme com os campos `timestamp`, `status`, `error` e `message`, facilitando o tratamento de erros no frontend e a rastreabilidade em *logs* de auditoria.

3.5. Implementação do Frontend

3.5.1. Estrutura de Ecrãs e Organização por Perfil

A implementação do frontend seguiu a organização modular estabelecida para o backend, com a estrutura de ecrãs espelhando diretamente os módulos funcionais e os perfis de utilizador. A directoria `src/` foi segmentada nas seguintes pastas principais: `screens/`, organizada por *role* (`auth`, `secretaria`, `treinador`, `clínica`, `portal`, `executivo` e `admin`); `services/`, com clientes HTTP por módulo; `navigation/`, com *stack navigators* por perfil de utilizador; `components/`, com componentes partilhados (`cards`, `modais`, `badges`); e `context/`, com o estado global de equipa seleccionada e utilizador autenticado.

Após o *login*, o sistema identifica o *role* do utilizador a partir do token JWT e redirige para o *navigator* correspondente, garantindo que utilizadores com *roles* distintos nunca têm acesso visual a ecrãs fora do seu domínio — complementando a protecção RBAC do backend com uma segunda linha de defesa no frontend.

3.5.2. Integração com a API REST

A comunicação com o backend foi encapsulada em ficheiros de serviço dedicados por módulo (`treinadorService.ts`, `secretariaService.ts`, etc.), centralizando a lógica de pedidos HTTP e evitando a duplicação de configuração em ecrãs individuais. Cada serviço segue um padrão uniforme: a URL base do backend está configurada num único ponto, facilitando a mudança de ambiente; um intercetor central anexa o *header* `Authorization: Bearer`

<token> a todos os pedidos autenticados; e respostas HTTP 401 desencadeiam automaticamente o *logout* e redireccionamento para o ecrã de *login*, enquanto respostas 403 apresentam uma mensagem de acesso negado sem expor detalhes técnicos ao utilizador.

Esta arquitectura garantiu que a adição de novos *endpoints* no backend implicava alterações circunscritas ao ficheiro de serviço correspondente, sem propagação de mudanças aos ecrãs consumidores.

3.5.3. Desafios de Compatibilidade e Resolução

O desenvolvimento com React Native e Expo num ambiente de múltiplos postos de trabalho introduziu uma classe específica de problemas de compatibilidade que não se manifestam em desenvolvimento web tradicional.

A actualização da versão do SDK Expo num posto de trabalho sem actualização correspondente na aplicação Expo Go instalada nos dispositivos de teste resultava em erros de carregamento silenciosos. A estratégia de resolução adoptada foi a fixação explícita da versão do SDK no `app.json`. Em caso de comportamento inesperado após actualizações, foi estabelecido um procedimento de *reset* nuclear da *cache*:

```
npx expo start --clear
# Em caso de persistencia:
rm -rf node_modules/.cache
npm install
npx expo start --web --port 8082
```

As diferenças de renderização entre web e mobile afectaram particularmente o módulo do Treinador: ao seleccionar determinadas equipas no ecrã de Plantel, os botões de filtro sobrepunham-se ao conteúdo da lista. A causa foi a colocação dos botões dentro de um `ScrollView` partilhado com a lista de atletas — a solução passou por mover os botões para fora do *scroll*, com a lista em `FlatList` com `contentContainerStyle` de *padding* adequado.

A gestão de estado entre abas exigia que a equipa seleccionada numa aba fosse preservada nas restantes (Plantel, Sessões, Jogos). A solução adoptada foi a criação de um `EquipaContext` em React Context API, disponibilizando o estado globalmente sem necessidade de *prop drilling* entre componentes.

3.5.4. Decisões de UX com Impacto Técnico

Algumas decisões de experiência de utilizador implicaram considerações técnicas não triviais na implementação. No fluxo de registo de presenças, a decisão de não incluir um botão de “seleccionar todos” — identificada durante os testes como fonte de comportamentos incorrectos — simplificou tanto a implementação como a experiência, obrigando o treinador a uma selec-

ção explícita por atleta que reduz erros de registo. A submissão da sessão apenas é activada quando todos os atletas aptos têm um estado atribuído, prevenindo submissões incompletas.

O fluxo de preenchimento da ficha de jogo foi estruturado em dois ecrãs distintos: o primeiro para selecção do onze inicial a partir da lista de convocados, o segundo para registo de eventos (golos, substituições, cartões) e observações. Esta separação implicou a gestão de estado temporário entre ecrãs — implementada através de parâmetros de navegação passados pelo React Navigation, sem persistência prematura na base de dados até à submissão final.

A lista de jogos do Treinador apresenta cards com coloração semântica baseada no estado `Ficha` calculado pelo backend: azul para jogos agendados, vermelho para fichas pendentes dentro do prazo, cinzento para fichas submetidas ou prazo expirado. O *countdown* de horas restantes é calculado no frontend a partir do campo `minutosRestantesParaSubmeter` retornado pela API, apresentado no formato `Xh Ym` e actualizado automaticamente sem necessidade de *re-fetch*.

3.6. Gestão da Base de Dados e Seed de Demonstração

3.6.1. Modelo Relacional Final

O modelo relacional do SIGD evoluiu ao longo das dez migrações Flyway para um total de dezoito tabelas, organizadas em quatro domínios lógicos. A Tabela 3.5 sintetiza a estrutura final, evidenciando as relações de dependência entre entidades.

Tabela 3.5.: Estrutura lógica e relacional das entidades do domínio do SIGD.

Domínio	Tabelas	Cardinalidade Chave
Institucional	modalidade, escalao, equipa, epoca_desportiva	Escalão pertence a Modalidade; Equipa pertence a Escalão
Pessoas	utilizador, encarregado_educacao, atleta	Atleta tem EE e pertence a Equipa; Utilizador EE liga-se por email ao EE
Clínico	ocorrencia, ocorrencia_evolucao	Evolução pertence a Ocorrência; Ocorrência tem Médico (FK para Utilizador)
Financeiro	obrigacao_financeira	Obrigação pertence a EE e pode referenciar Atleta
Desportivo	sessao_treino, registo_assiduidade, avaliacao_rendimento, evento_desportivo, convocatoria, convocatoria_atletas, ficha_jogo, ficha_jogo_titular, evento_jogo	Estrutura hierárquica: Evento → Convocatória → Ficha → Titulares/Eventos
Auditoria	audit_log	Registo imutável de acções com ator, entidade e <i>timestamps</i>

Uma decisão de esquema com impacto direto na *performance* das consultas mais frequentes foi a persistência do estado_elegibilidade diretamente na tabela atleta, em vez de o calcular dinamicamente por *join* com ocorrencia. Embora introduza redundância controlada, esta abordagem elimina *joins* complexos nas consultas de plantel e convocatória — operações executadas com elevada frequência durante sessões de treino e jogos, precisamente quando a latência seria mais penalizadora.

3.6.2. Seed Master Demo — Objetivos e Construção

A necessidade de um conjunto de dados de demonstração consistente, denso e reprodutível tornou-se evidente logo nas primeiras sessões de teste: dados insuficientes impediam a validação de fluxos completos, e dados inconsistentes entre postos de trabalho tornavam os resultados dos testes não comparáveis.

A resposta a este problema foi o desenvolvimento do `seed_master_demo.sql` — um ficheiro SQL de reposição total que, ao ser executado, limpa todos os dados existentes e reconstrói um estado de base de dados completo e coerente, independentemente do histórico de operações anteriores. A Tabela 3.6 detalha a estrutura e volumetria do ficheiro.

Tabela 3.6.: Estrutura e volumetria do ficheiro de dados de demonstração (*Seed Master Demo*).

Secção	Conteúdo	Registos
0	TRUNCATE de todas as tabelas + <i>reset</i> de utilizadores de sistema	—
1	Modalidades, Escalões, Equipas, Época Desportiva	16
2	Encarregados de Educação + Utilizadores (sistema e EE)	37
3	Atletas distribuídos por 6 equipas com estados variados	50
4	Sessões de treino (passadas, hoje, futuras)	29
5	Eventos desportivos (jogos passados e agendados)	24
6	Fichas de jogo + Convocatórias publicadas	8
7	Registos de assiduidade + Avaliações de rendimento	265
8	Ocorrências clínicas + Evoluções	17
9	Obrigações financeiras (CLUBE e SAD)	95
10	<i>Audit log</i> com acções representativas de todos os <i>roles</i>	44
11	SELECT de verificação com contagem de todas as tabelas	—

A densidade e variedade dos dados foram desenhadas especificamente para cobrir os cenários de teste de todos os módulos: atletas em todos os estados de elegibilidade (APTO, CONDICIONADO, INAPTO, PENDENTE_EMD); obrigações em todos os estados financeiros (PAGO, PENDENTE, EM_ATRASO); jogos passados com e sem ficha submetida para testar o mecanismo de prazo; sessões de treino em datas relativas (CURDATE(), DATE_SUB, DATE_ADD) para garantir que os dados se mantêm temporalmente relevantes independentemente da data de execução.

3.6.3. O *Seed* como “Dono da Verdade”

Uma regra operacional estabelecida e seguida pela equipa ao longo de todo o desenvolvimento foi a de que qualquer alteração ao esquema da base de dados que implicasse novos dados de teste tinha de ser reflectida no *seed*. Injecções manuais de dados de teste directamente no contentor MySQL eram consideradas temporárias e descartáveis — o *seed* era o único estado de referência permanente.

Esta disciplina teve duas consequências práticas. Primeiro, o *seed* evoluiu a par com o esquema: cada nova migração Flyway era acompanhada da actualização do *seed* com dados representativos para as novas tabelas ou campos. Segundo, a reposição do ambiente de demonstração tornou-se um procedimento de um único comando — eliminando a necessidade de qualquer documentação adicional sobre o estado esperado da base de dados.

Capítulo 4 - Verificação, Validação e Avaliação da Qualidade do Software Produzido

A quarta etapa do projecto centrou-se na verificação sistemática de que o sistema implementado cumpria os requisitos especificados no SRS e satisfazia os padrões de qualidade definidos para o SIGD. Ao contrário de uma abordagem de qualidade sequencial — onde os testes são realizados apenas após a conclusão da implementação —, a estratégia adoptada integrou actividades de verificação ao longo de todo o ciclo de desenvolvimento, culminando nesta fase numa avaliação formal e abrangente que cobriu testes automatizados, validação de aceitação e métricas de qualidade segundo a norma ISO/IEC 25010.

O LLM desempenhou um papel activo e documentado nesta etapa, não apenas como gerador de casos de teste, mas como instrumento de análise de cobertura, detecção de inconsistências entre a implementação e o SRS, e apoio à redacção dos relatórios de teste formais. A presente secção descreve a metodologia adoptada, os resultados obtidos e as decisões tomadas ao longo deste processo.

4.1. Estratégia de Verificação e Validação

4.1.1. Distinção entre Verificação e Validação

A estratégia de qualidade adoptada no SIGD estabeleceu uma distinção operacional clara entre duas actividades complementares, frequentemente confundidas na prática de desenvolvimento. A verificação — resumida na questão “Estamos a construir o sistema correctamente?” — focou-se em garantir que a implementação era internamente consistente e conformante com as especificações técnicas, materializando-se nos testes unitários e de integração automatizados, na análise de cobertura JaCoCo e na verificação formal do SRS. A validação — resumida na questão “Estamos a construir o sistema correcto?” — focou-se em confirmar que o sistema satisfazia as necessidades reais dos utilizadores e os critérios de aceitação definidos nas *User Stories*, materializando-se nos testes de sistema (T3), nos testes de aceitação (T4) e na avaliação de qualidade ISO 25010.

Esta distinção orientou a organização do trabalho desta etapa: as actividades de verificação

foram executadas em ambiente controlado com ferramentas automatizadas, enquanto as atividades de validação exigiram a execução manual de cenários de utilização real contra o sistema em funcionamento.

4.1.2. Pirâmide de Testes Adoptada

A distribuição dos esforços de teste seguiu o modelo da pirâmide de testes, adaptado às características específicas do SIGD. A base da pirâmide foi ocupada pelos testes unitários, com maior volume, execução mais rápida e isolamento total de dependências externas, cobrindo a lógica de negócio de cada serviço individualmente com dependências substituídas por *mocks* Mockito e constituindo a primeira linha de detecção de regressões em ciclos de desenvolvimento rápidos. O nível intermédio correspondeu aos testes de integração, com volume médio, verificando o comportamento da API REST de ponta a ponta com base de dados H2 em memória e validando contratos de API, segurança de *endpoints*, serialização/deserialização de DTOs e consistência de respostas HTTP. O topo da pirâmide foi ocupado pelos testes de sistema e aceitação, com menor volume, execução manual e maior fidelidade ao ambiente real, verificando fluxos completos de utilizador contra o sistema em execução com dados de demonstração reais.

Esta estrutura garantiu que a detecção de problemas ocorria o mais cedo possível no ciclo de desenvolvimento — os testes unitários corriam a cada compilação, os de integração em cada *commit* significativo, e os de sistema e aceitação nas fases de revisão de módulo e entrega.

4.1.3. Ferramentas e Infraestrutura de Testes

A infraestrutura de testes foi configurada para executar de forma completamente autónoma, sem dependências de serviços externos ou estado da base de dados de desenvolvimento. A Tabela 4.1 sintetiza as ferramentas utilizadas.

Tabela 4.1.: Ferramentas e versões utilizadas na infraestrutura de testes do SIGD.

Ferramenta	Versão	Função
JUnit 5	5.10.x	<i>Framework</i> de testes unitários e de integração
Mockito	5.x	Criação de <i>mocks</i> para isolamento de dependências
MockMvc	integrado Spring	Simulação de pedidos HTTP em testes de integração
H2 Database	2.x	Base de dados em memória para testes de integração
JaCoCo	0.8.11	Análise e reporte de cobertura de código
Maven Surefire	3.x	Execução automatizada dos testes no ciclo de <i>build</i>

A configuração do perfil de testes (`application-test.yml`) estabeleceu a base de dados H2 com modo `create-drop` — o esquema é criado de raiz antes de cada execução de testes e destruído no final, garantindo isolamento total entre *suites* de teste e eliminando dependências de estado persistido.

4.1.4. Papel do LLM na Estratégia de Qualidade

O LLM foi integrado na estratégia de qualidade em três vertentes distintas, cada uma com um nível de autonomia diferente e um grau correspondente de supervisão humana. Na geração de casos de teste, o modelo foi utilizado para gerar a estrutura base dos testes, cobrindo os cenários mais comuns, com a supervisão humana focada na validação da correcção lógica dos cenários gerados e na adição de casos de fronteira específicos do domínio do BFC. Na análise de inconsistências, o modelo foi utilizado para cruzar a implementação com o SRS, identificando requisitos sem cobertura de teste ou com implementação divergente da especificação — uma análise que seria proibitivamente demorada se realizada manualmente sobre um catálogo de 46 requisitos funcionais e 25 não-funcionais. Na redacção de relatórios, o modelo foi utilizado para estruturar e redigir os documentos de teste formais (T3, T4, T5), seguindo a mesma lógica de *ditado estruturado* descrita na Etapa 3.

4.2. Geração de Testes Assistida por LLM

4.2.1. Metodologia de Geração

A geração de testes assistida por LLM seguiu uma abordagem em duas fases para cada módulo funcional, inspirada na estratégia divergente-convergente adoptada na fase de engenharia de requisitos.

Na fase divergente, o modelo era alimentado com a classe de serviço a testar, os requisitos do SRS que essa classe implementava e os critérios de aceitação das *User Stories* correspondentes. A instrução solicitava a identificação exaustiva de cenários a testar — sem ainda gerar código — produzindo uma lista comentada de casos de teste candidatos, agrupados por método e categoria (caminho feliz, validação de negócio, caso de erro).

Na fase convergente, a lista de cenários era revista e aprovada pela equipa, que eliminava redundâncias, adicionava casos de fronteira específicos do domínio e confirmava a prioridade de cada cenário. Apenas após esta aprovação o modelo gerava o código de teste correspondente, seguindo os padrões definidos no Template VII-C (Anexo XII).

Esta separação entre identificação de cenários e geração de código revelou-se fundamental para a qualidade dos testes produzidos. Quando o modelo gerava directamente o código sem a fase de planeamento intermediária, tendia a produzir testes que cobriam os caminhos mais

óbvios mas omitiam sistematicamente casos de fronteira críticos — listas vazias, valores nulos em campos opcionais, transições de estado inválidas.

4.2.2. Cobertura por Categorias de Teste

A geração assistida foi aplicada transversalmente a todos os módulos, produzindo testes nas três categorias definidas na estratégia. Os testes de caminho feliz verificam que a operação correcta produz o resultado esperado e foram os mais simples de gerar — o modelo produzia-os com elevada fiabilidade desde que a classe alvo estivesse bem estruturada e os DTOs claramente definidos. Os testes de validação de negócio verificam que as regras do SRS são correctamente aplicadas e requereram maior intervenção humana na fase divergente, dado que o modelo nem sempre identificava todas as regras implícitas de um domínio específico como o desportivo e clínico; exemplos representativos incluem a verificação de que um atleta com `PENDENTE_EMD` não pode ser convocado, que golos negativos são rejeitados e que o último administrador activo não pode ser desactivado. Os testes de casos de erro e fronteira verificam o comportamento do sistema perante *inputs* inválidos, estados inconsistentes e condições excepcionais, tendo sido os que mais beneficiaram da abordagem divergente-convergente — a revisão humana identificou sistematicamente casos que o modelo omitia, como a tentativa de submeter uma ficha de jogo para um evento já com ficha registada ou o cálculo de médias de avaliação quando nenhuma sessão tinha sido concluída.

4.2.3. Detecção de Inconsistências entre Implementação e SRS

Uma aplicação particularmente valiosa do LLM nesta fase foi a análise cruzada entre o código implementado e o SRS, com o objectivo de identificar requisitos sem cobertura de teste ou com implementação divergente da especificação. O processo consistia em fornecer ao modelo, em simultâneo, a especificação de um requisito do SRS e a implementação correspondente no serviço, solicitando a identificação de divergências. Esta análise identificou um conjunto de inconsistências que tinham escapado à revisão manual durante a implementação, das quais se destacam:

- O RF-03 especificava que a média de avaliação de rendimento deveria ser calculada sobre as últimas 5 sessões com presença validada, mas a implementação calculava sobre todas as sessões concluídas sem filtro de presença — corrigido após detecção;
- O RNF-04 especificava um *debounce* mínimo de 300ms na pesquisa de atletas, mas a implementação *frontend* não tinha qualquer mecanismo de *debounce* — corrigido com implementação de `setTimeout` de 300ms;
- O RF-22 especificava que o bloqueio de conta por tentativas falhadas deveria persistir na base de dados e sobreviver a reinícios do servidor, mas a implementação original armazenava o estado apenas em memória — corrigido com a migração V7.

Esta capacidade de análise cruzada automatizada sobre um catálogo extenso de requisitos representou um ganho de eficiência significativo face a uma revisão manual exaustiva, que seria inviável dentro das restrições temporais do projecto.

4.3. Testes Unitários

4.3.1. Âmbito e Organização

Os testes unitários constituíram a base da pirâmide de testes do SIGD, cobrindo a lógica de negócio implementada nas camadas de serviço de cada módulo funcional. A decisão de focar os testes unitários exclusivamente na camada de serviço decorreu de um princípio de responsabilidade: é nos serviços que reside a lógica de negócio crítica que o SRS especifica, sendo essa a camada cujo comportamento incorrecto tem maior impacto funcional.

A organização dos testes espelhou a estrutura de pacotes do *backend*, com uma classe de teste por classe de serviço, distribuída pelos seguintes pacotes: *auth/* (*AuthServiceTest.java*); *secretaria/* (*AtletaServiceTest.java*, *EncarregadoServiceTest.java*, *UtilizadorAdminServiceTest.java*); *clinica/* (*OcorrenciaServiceTest.java*, *SemaforoServiceTest.java*); *treinador/* (*SessaoTreinoServiceTest.java*, *ConvocatoriaServiceTest.java*, *FichaJogoServiceTest.java*); *portal/* (*PortalServiceTest.java*); e *executivo/* (*CeoServiceTest.java*, *CfoServiceTest.java*).

4.3.2. Padrões de Implementação

Todos os testes unitários seguiram um conjunto de padrões técnicos uniformes, estabelecidos no Template VII-C e aplicados consistentemente ao longo de todos os módulos. A configuração base de cada classe de teste seguiu o padrão:

```
@ExtendWith(MockitoExtension.class)
class AtletaServiceTest {

    @Mock
    private AtletaRepository atletaRepository;

    @Mock
    private EncarregadoRepository encarregadoRepository;

    @InjectMocks
    private AtletaService atletaService;
}
```

A convenção de nomenclatura adoptada seguiu o padrão `[metodoTestado]_[estadoInicial]_[resultadoEsperado]` tornando o propósito de cada teste imediatamente legível sem necessidade de ler o corpo:

```

@Test
void registrarAtleta_comNifDuplicado_lancaIllegalArgumentException()

@Test
void registrarAtleta_comDadosValidos_retornaAtletaDTO()

@Test
void desactivarUtilizador_quandoUltimoAdmin_lancaIllegalStateException()

```

A estrutura interna AAA (Arrange-Act-Assert) foi aplicada rigorosamente em cada teste, com separação visual entre as três fases através de comentários *inline*:

```

@Test
void calcularElegibilidade_comGrauVermelho_retornaInapto() {
    // Arrange
    Atleta atleta = Atleta.builder().id(1L).build();
    Ocorrencia ocorrencia = Ocorrencia.builder()
        .atleta(atleta)
        .grauRestricao("VERMELHO")
        .estado("ATIVA")
        .build();
    when(ocorrenciaRepository.findTopByAtletaIdAndEstado(1L, "ATIVA"))
        .thenReturn(Optional.of(ocorrencia));

    // Act
    String resultado = semaforoService.calcularElegibilidade(atleta);

    // Assert
    assertThat(resultado).isEqualTo("INAPTO");
}

```

4.3.3. Cenários Cobertos por Módulo

A geração assistida por LLM, combinada com a revisão humana da fase divergente, produziu uma cobertura de cenários estruturada e deliberada. A Tabela 4.2 sintetiza os cenários mais representativos por módulo.

Tabela 4.2.: Cenários de teste unitário cobertos por módulo funcional.

Módulo	Caminho Feliz	Validação de Negócio	Casos de Erro
Secretaria	Registo de atleta válido; associação a EE; validação de documentos	NIF duplicado; email de EE duplicado; último admin indesactivável	Atleta não encontrado; EE não encontrado
Clínica	Registo de ocorrência; evolução de grau; deliberação EMD	Diagnóstico vazio rejeitado; grau inválido rejeitado	Médico sem permissão; ocorrência não encontrada
Semáforo	APTO sem ocorrência activa; CONDICIONADO com AMARELO; INAPTO com VERMELHO	PENDENTE_EMD sem data válida; prioridade de ocorrência activa	Atleta sem ocorrências
Treinador	Criar sessão; registar presença; submeter avaliação	Golos negativos rejeitados; nota fora de [1.0, 5.0]; sessão já concluída	Sessão não encontrada; atleta não pertence à equipa
Convocatória	Criar convocatória; adicionar atleta; publicar	INAPTO não convocável; PENDENTE_EMD não convocável; tecto excedido	Evento não encontrado; convocatória já publicada
Portal EE	Listar obrigações; filtrar por estado; consultar agenda	—	EE sem atletas associados
Financeiro	Calcular rácio de liquidez; agrupar por rubrica	—	Dados insuficientes para cálculo

4.3.4. Casos de Fronteira e Testes Negativos

A análise da fase divergente de geração assistida identificou um conjunto de casos de fronteira que não resultavam directamente da leitura dos requisitos do SRS, mas da análise combinatória dos estados possíveis do sistema. Estes casos revelaram-se os mais valiosos do conjunto de testes, por terem detectado *bugs* reais que escaparam à revisão durante a implementação.

O primeiro caso de relevo foi o cálculo de média com histórico insuficiente. O RF-03 especifica que a média de avaliação é calculada sobre as últimas 5 sessões. O caso de fronteira de um atleta com menos de 5 sessões concluídas — situação inevitável no início de cada época — não estava tratado na implementação original, resultando numa divisão por zero. O teste que detectou este *bug* foi:

```
@Test
```

```

void calcularMediaAvaliacao_semSessoesDisponiveis_retornaZeroSemExcecao()
{
    when(avaliacaoRepository.findTopNByAtletaId(anyLong(), anyInt()))
        .thenReturn(Collections.emptyList());

    Double media = sessaoService.calcularMediaAvaliacao(1L);

    assertThat(media).isEqualTo(0.0);
}

```

O segundo caso foi a submissão de ficha para evento já com ficha: a tentativa de submeter uma segunda ficha de jogo para um evento que já tinha ficha registada não estava validada no serviço original, permitindo duplicação de registos. O teste correspondente verificou que a excepção `IllegalStateException` era lançada com a mensagem correcta.

O terceiro caso foi o bloqueio de conta com tentativas distribuídas: o mecanismo de *logout* contava tentativas falhadas consecutivas, mas não tratava correctamente o caso em que um *login* bem sucedido intercalado devia repor o contador. O teste que detectou esta lacuna verificou que após *login* válido o contador era efectivamente reposto para zero.

4.3.5. Resultados Finais

O conjunto final de testes unitários compreendeu 175 testes, todos com resultado de aprovação. A Tabela 4.3 reflecte a evolução ao longo do desenvolvimento.

Tabela 4.3.: Evolução dos resultados dos testes unitários ao longo do desenvolvimento.

Fase	Aprovados	Falhados	Total
Após implementação inicial	148	27	175
Após correcção Módulo Segurança	163	12	175
Após correcção Módulos Clínica + Treinador	170	5	175
Estado final	175	0	175

4.4. Testes de Integração

4.4.1. Âmbito e Configuração

Os testes de integração cobriram o comportamento da API REST de ponta a ponta, desde a recepção do pedido HTTP até à resposta serializada, passando pela autenticação, autorização,

lógica de negócio e persistência numa base de dados H2 em memória. O seu propósito foi verificar que as camadas do sistema funcionavam correctamente em conjunto — um aspecto que os testes unitários, por natureza, não conseguem garantir.

A configuração do perfil de testes estabeleceu o contexto de execução:

```
# application-test.yml
spring:
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;MODE=MySQL
    driver-class-name: org.h2.Driver
  jpa:
    hibernate:
      ddl-auto: create-drop
      database-platform: org.hibernate.dialect.H2Dialect
    flyway:
      enabled: false
```

A desactivação do Flyway no perfil de testes e a utilização do modo create-drop do Hibernate garantiu que cada *suite* de testes partia de um esquema limpo e consistente, sem dependências do histórico de migrações. Os dados necessários para cada teste eram criados programaticamente nos métodos `@BeforeEach`, garantindo o isolamento total entre testes.

4.4.2. Padrões de Implementação

Os testes de integração utilizaram `@SpringBootTest` com `@AutoConfigureMockMvc`, carregando o contexto Spring completo e permitindo a simulação de pedidos HTTP através do `MockMvc` sem necessidade de um servidor real em execução:

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class AtletaControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private AtletaRepository atletaRepository;

    @Autowired
    private JwtService jwtService;

    private String tokenSecretaria;
    private String tokenTreinador;
```

```

@BeforeEach
void setup() {
    atletaRepository.deleteAll();
    tokenSecretaria = gerarToken("secretaria", "ROLE_SECRETARIA");
    tokenTreinador = gerarToken("treinador", "ROLE_TREINADOR");
}
}

```

Um padrão crítico nos testes de integração foi a verificação explícita da segurança de cada *endpoint* — não apenas que o pedido autenticado com o *role* correcto funcionava, mas também que pedidos sem *token* ou com *role* insuficiente eram correctamente rejeitados:

```

@Test
void registrarAtleta_semAutenticacao_retorna401() throws Exception {
    mockMvc.perform(post("/api/v1/secretaria/atletas")
        .contentType(MediaType.APPLICATION_JSON)
        .content(atletaJsonValido))
        .andExpect(status().isUnauthorized());
}

@Test
void registrarAtleta_comRoleTreinador_retorna403() throws Exception {
    mockMvc.perform(post("/api/v1/secretaria/atletas")
        .header("Authorization", "Bearer_" + tokenTreinador)
        .contentType(MediaType.APPLICATION_JSON)
        .content(atletaJsonValido))
        .andExpect(status().isForbidden());
}

@Test
void registrarAtleta_comRoleSecretaria_retorna201() throws Exception {
    mockMvc.perform(post("/api/v1/secretaria/atletas")
        .header("Authorization", "Bearer_" + tokenSecretaria)
        .contentType(MediaType.APPLICATION_JSON)
        .content(atletaJsonValido))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.nomeCompleto").value("Tomas_Silva"))
        .andExpect(jsonPath("$.estadoElegibilidade").value("APTO"));
}

```

4.4.3. Cobertura de Segurança e Contratos de API

Os testes de integração foram particularmente focados em dois aspectos que os testes unitários não conseguem verificar: a correcta aplicação das regras de segurança RBAC e a conformidade dos contratos de API com o esperado pelo *frontend*.

Para a cobertura de segurança, cada *endpoint* foi testado com pelo menos três variantes: sem autenticação (HTTP 401 esperado), com *role* insuficiente (HTTP 403 esperado) e com *role* correcto (HTTP 2xx esperado). Esta cobertura sistemática garantiu que nenhum *endpoint* ficou inadvertidamente exposto sem autenticação — uma classe de vulnerabilidade particularmente crítica num sistema que processa dados clínicos de menores.

Para a conformidade de contratos de API, os testes verificaram a estrutura exacta das respostas JSON, incluindo a presença e ausência de campos específicos. A verificação de ausência foi particularmente importante para garantir que dados sensíveis não eram inadvertidamente incluídos nas respostas:

```
@Test
void listarPlantel_comRoleTreinador_naoExpoePasswordHash()
    throws Exception {
    mockMvc.perform(get("/api/v1/treinador/plantel?equipaId=1")
        .header("Authorization", "Bearer_" + tokenTreinador))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.passwordHash").doesNotExist())
        .andExpect(jsonPath("$.diagnostico").doesNotExist());
}
```

4.4.4. Resultados Finais

O conjunto final de testes de integração compreendeu 32 testes, todos com resultado de aprovação. A Tabela 4.4 reflecte a evolução ao longo do desenvolvimento.

Tabela 4.4.: Evolução dos resultados dos testes de integração ao longo do desenvolvimento.

Fase	Aprovados	Falhados	Total
Após implementação inicial	21	11	32
Após correcção de contratos de API	26	6	32
Após correcção de segurança RBAC	30	2	32
Estado final	32	0	32

As falhas mais frequentes nos testes de integração foram de duas naturezas: divergências entre a estrutura dos DTOs de resposta e o esperado pelo teste (resolvidas alinhando os DTOs com os contratos definidos), e falhas de segurança onde *endpoints* deviam retornar 403 mas retornavam 404 por o *handler* de autorização ser executado após a lógica de negócio — corrigido reordenando os filtros do *SecurityConfig*.

Conclusões e Trabalho Futuro

O presente relatório documentou o percurso integral de desenvolvimento do Sistema Integrado de Gestão Desportiva do Boavista Futebol Clube — o SIGD —, desde a concepção dos requisitos até à verificação formal da qualidade do software produzido. Ao longo das quatro etapas, a equipa adoptou uma metodologia de desenvolvimento assistido por Inteligência Artificial que permeou todas as fases do ciclo de vida do software.

O SIGD concretizou uma plataforma de gestão desportiva funcional e operacional, integrando seis módulos autónomos que servem perfis de utilizador distintos. Os resultados quantitativos confirmam a maturidade técnica do produto: 175 testes unitários e 32 testes de integração aprovados na totalidade, cobertura JaCoCo de 73.4% e classificação média de 3.5/4 segundo a norma ISO/IEC 25010, com excelência nas características de Segurança, Manutenibilidade e Portabilidade.

Do ponto de vista metodológico, a experiência acumulada permite formular três conclusões centrais sobre a utilização de LLMs no desenvolvimento de software. Primeiro, o LLM funciona como amplificador de capacidade e não como substituto do julgamento técnico — a qualidade do *output* foi consistentemente proporcional à qualidade do contexto fornecido. Segundo, a supervisão humana é insubstituível em decisões de domínio: em múltiplos momentos, o modelo gerou código tecnicamente correcto mas logicamente errado face às regras de negócio específicas do domínio desportivo e clínico. Terceiro, a engenharia de *prompts* revelou-se uma competência técnica de primeira ordem, tão determinante para o sucesso do projecto quanto as competências tradicionais de programação.

O estado actual do SIGD constitui uma base funcional sólida com um conjunto identificado de oportunidades de evolução. No plano das **funcionalidades descoped**, destacam-se o sistema de notificações *push* e SMS para alertas de caducidade de EMD e publicação de convocatórias, a geração automática de PDF de convocatória, a conclusão do fluxo de ficha de jogo com onze inicial e eventos por minuto, e o mecanismo de *reset* de *password* por email.

No plano das **melhorias técnicas**, as prioridades são a implementação de testes de carga para verificação formal dos requisitos de desempenho, a configuração de um *pipeline* CI/CD para execução automática dos testes a cada *commit*, e a adopção de uma solução de monitorização em produção com *health checks* e alertas automáticos.

No plano da **expansão do âmbito**, a publicação de uma aplicação móvel nativa nas *app stores*, a integração com as APIs da Federação Portuguesa de Futebol para validação automática do estatuto federativo, e o desenvolvimento de um portal de autoatendimento financeiro para

sócios — identificado como prioridade absoluta por 79% dos inquiridos — constituem os vectores de maior impacto na adopção e utilidade institucional da plataforma.

O SIGD não é um protótipo — é uma plataforma funcional, testada e documentada, pronta para uma fase de testes piloto com utilizadores reais do Boavista FC. O trabalho futuro identificado representa o roteiro natural de evolução de um produto com fundações técnicas sólidas para crescer.

Referências Bibliográficas

- Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley.
- Chen, J., Wang, S., Chen, X., et al. (2021). Software Engineering for AI-Based Systems: A Survey. *ACM Transactions on Software Engineering and Methodology*, 31(2), Article 26. <https://doi.org/10.1145/3472810>
- Feldt, R., Torkar, R., Afzal, W., et al. (2018). Artificial Intelligence in Software Engineering: A Systematic Mapping Study. *IEEE Transactions on Software Engineering*, 44(6), 1–31. <https://doi.org/10.1109/TSE.2018.2793609>
- Forsberg, K., Mooz, H., & Cotterman, H. (2005). *Visualizing Project Management* (3ª ed.). John Wiley & Sons.
- Institute of Electrical and Electronics Engineers. (2018). Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. <https://ieeexplore.ieee.org/document/8559686>
- International Organization for Standardization & International Electrotechnical Commission. (2023). Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models. <https://www.iso.org/standard/78175.html>
- Kim, G., Debois, P., Willis, J., & Humble, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.
- Leite, L., Rocha, C., Kon, F., Milojicic, D., & Meirelles, P. (2019). A Survey of DevOps Concepts and Challenges. *ACM Computing Surveys*, 52(6), Article 127. <https://doi.org/10.1145/3359981>
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9ª ed.). McGraw-Hill Education.
- Royce, W. W. (1970). Managing the Development of Large Software Systems. *Proceedings of IEEE WESCON*, 26(8), 1–9. <https://doi.org/10.1109/WESCON.1970.4954670>
- Sculley, D., Holt, G., Golovin, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28, 2503–2511.
- Sommerville, I. (2016). *Software Engineering* (10ª ed.). Pearson Education.
- Walls, C. (2022). *Spring Boot in Action* (2ª ed.) [Referência de suporte técnico para o ecossistema Spring Boot]. Manning Publications.

Lista de Siglas e Acrónimos

ACID Atomicidade, Consistência, Isolamento e Durabilidade.

APA American Psychological Association.

BFC Boavista Futebol Clube.

CEO Chief Executive Officer.

CFO Chief Financial Officer.

EMD Exame Médico-Desportivo.

IA Inteligência Artificial.

IEC International Electrotechnical Commission.

IEEE Institute of Electrical and Electronics Engineers.

ISO International Organization for Standardization.

IT Information Technology.

LLM Modelos de Linguagem de Grande Escala.

RE Requisito Estratégico.

RF Requisito Funcional.

RGPD Regulamento Geral sobre a Proteção de Dados.

RO Requisito Operacional.

ROI Return on Investment.

SAD Sociedade Anónima Desportiva.

SGBD Sistema de Gestão de Bases de Dados.

SIGD Sistema Integrado de Gestão Desportiva.

SRS Software Requirements Specification.

UML Unified Modeling Language.

US User Story.

Anexos

Índice de Anexos

Anexo I. Simulações de Entrevistas de Eliciação	85
Anexo II. Dados do Inquérito por Questionário	86
1. Estrutura do Questionário e Resultados Quantitativos	86
Anexo III. Atas de Reunião e Decisões Técnicas	90
Anexo IV. Catálogo de <i>User Stories</i>	92
Anexo V. Catálogo de Requisitos Iniciais	94
Anexo VI. Documento SRS	96
Anexo VII. Catálogo de Casos de Uso	98
Anexo VIII. Modelo de Domínio Conceptual	102
Anexo IX. Prototipagem Assistida	104

Anexo I. Simulações de Entrevistas de Eliciação

Neste anexo encontram-se descritas as metodologias aplicadas nas simulações de entrevistas realizadas com os diversos *stakeholders*, suportadas por Inteligência Artificial (LLM). Por forma a garantir o rigor académico e a validade da eliciação de requisitos, todas as simulações seguiram um padrão transversal de parametrização, assente em três diretrizes principais:

1. **Isolamento de Domínio:** Em todas as interações iniciais, a *persona* foi explicitamente proibida de abordar temas logísticos, financeiros ou arquiteturais fora do escopo do produto, forçando-a a focar-se apenas na sua "dor" operacional.
2. **Restrição de Formato:** O modelo foi bloqueado de gerar listas estruturadas ou jargão técnico (*APIs*, bases de dados), obrigando a respostas em texto corrido e coloquial, mimetizando desabafos e conversas reais no terreno.
3. **Eliciação em Funil:** Cada entrevista foi fatiada em três interações lógicas. Uma primeira fase de diagnóstico de dores gerais, seguida de interações de aprofundamento para extrair fluxos de trabalho e, por fim, regras de negócio restritas.

Deste modo, a documentação integral das transcrições realizadas, contendo os diálogos resultantes de cada simulação com os respetivos *stakeholders*, encontra-se disponível para consulta externa através da seguinte ligação: <https://tinyurl.com/y5eemjns>.

Anexo II. Dados do Inquérito por Questionário

O instrumento de recolha de dados, bem como a base de dados em bruto com os resultados consolidados, encontram-se disponíveis para consulta através da seguinte ligação: <https://tinyurl.com/3f8yazwy>.

1. Estrutura do Questionário e Resultados Quantitativos

A tabela seguinte apresenta, de forma consolidada, as questões formuladas, as respetivas opções de resposta disponibilizadas aos utilizadores e a distribuição estatística das respostas obtidas.

Questão	Opção de Resposta	%
1. Qual é a sua principal relação com o Boavista FC? (Pode escolher mais do que uma)	Encarregado de Educação / Adepto	51%
	Encarregado de Educação de um atleta da Formação	39%
	Atleta (Escalões de Formação ou Modalidades Amadoras)	22%
	Outro	0%
2. Em que faixa etária se insere?	36 - 50 anos	39%
	18 - 35 anos	22%
	51 - 65 anos	17%
	Menos de 18 anos	11%
	Mais de 65 anos	11%
3. Que tipo de dispositivo prefere utilizar para aceder aos serviços digitais do seu dia a dia (ex: homebanking, portais escolares, clubes)?	Telemóvel (Smartphone)	55%
	Computador (Portátil ou Fixo)	35%
	Tablet	10%
4. Com que frequência necessita de se deslocar fisicamente à Secretaria do Boavista FC APENAS para tratar de burocracias (saber valores em dívida, pedir recibos, entregar papéis)?	1 a 2 vezes por mês	45%
	Raramente (1 a 2 vezes por época)	31%
	Nunca / Não se aplica	12%
	1 a 2 vezes por semana	11%
5. Atualmente, se precisar de atualizar os seus dados pessoais (ex: mudar de morada, telefone ou email), como costuma fazê-lo?	Desloco-me presencialmente à secretaria	36%
	Raramente atualizo porque o processo é pouco prático	33%
	Tento ligar ou enviar um email para o clube	31%
Continua na página seguinte...		

Continuação da tabela anterior

Questão	Opção de Resposta	%
6. Qual é a sua maior frustração quando precisa de regularizar a sua situação administrativa presencialmente na Secretaria?	Filas de espera morosas devido à lentidão do sistema	39%
	O facto de o clube e o futebol de formação usarem processos separados	32%
	Não sinto dificuldades / Não se aplica	18%
	Dificuldade em obter um recibo único e claro	11%
7. Atualmente, sabe dizer com exatidão, e de forma autónoma (sem ter de ligar ou ir à secretaria), até que mês tem as suas quotas ou mensalidades regularizadas?	Não, dependo sempre de perguntar à secretaria	39%
	Tenho uma ideia por alto, mas não tenho a certeza absoluta	34%
	Sim, tenho um controlo pessoal rigoroso	28%
8. Caso tenha mais do que um familiar ou educando inscrito no clube, como descreve a experiência de gerir os pagamentos de todos?	Só tenho um familiar inscrito / Não se aplica	61%
	Trabalhosa, a secretaria trata cada processo separadamente	39%
	Simples, consigo tratar de tudo num único momento	0%
9. Quando o atleta falta a um treino, que método utiliza atualmente para justificar a ausência?	Não se aplica	40%
	Envio mensagem informal para o WhatsApp pessoal do Treinador	22%
	Ligo para a secretaria do clube	17%
	Muitas vezes não consigo avisar atempadamente	10%
	Peço a um colega de equipa para dar o recado	10%
10. Como recebe habitualmente as informações oficiais de última hora (ex: cancelamento de treinos devido ao mau tempo, alterações de convocatória)?	Não se aplica	40%
	Grupos informais de WhatsApp (treinadores/pais)	33%
	Contacto telefónico direto da equipa técnica	17%
	Apenas descubro quando chego às instalações	10%
11. Se tivesse acesso a uma "Área do Atleta" no telemóvel, que tipo de informação desportiva mais valorizaria poder consultar de forma oficial? (Escolha até 2)	Não se aplica	40%
	Calendário completo das convocatórias	34%
	Minutos de jogo oficiais e golos marcados	33%
<i>Continua na página seguinte...</i>		

Continuação da tabela anterior

Questão	Opção de Resposta	%
	Assiduidade aos treinos (presenças/faltas)	28%
12. Atualmente, sente que tem facilidade em acompanhar essas estatísticas do percurso desportivo do atleta sem ter de perguntar à equipa técnica?	Não, sinto que não tenho visibilidade sobre os dados oficiais	60%
	Não se aplica	40%
	Mais ou menos, a informação podia estar mais acessível	0%
	Sim, a informação flui bem e de forma transparente	0%
13. Já alguma vez o seu educando (ou o próprio) ficou impedido de treinar ou de ser convocado devido à caducidade do Exame Médico-Desportivo (EMD) sem que se tivesse apercebido com antecedência?	Sim, já fomos apanhados de surpresa	40%
	Não se aplica	40%
	Não, costumo controlar bem a data sozinho	21%
14. Como preferiria ser avisado de que o Exame Médico do atleta está prestes a caducar?	Não se aplica	40%
	Alerta automático na aplicação do clube (30 dias antes)	33%
	Através de um Email oficial do clube	17%
	Prefiro que o treinador me avise pessoalmente	10%
15. Qual o seu grau de conforto com a submissão digital da documentação médica (ex: foto do Exame) através de uma plataforma restrita, sabendo que apenas o Médico do clube a poderá visualizar?	Muito confortável	55%
	Confortável	34%
	Desconfortável (prefiro papel na Secretaria)	11%
16. Com que frequência se esquece (ou não sabe onde tem) o seu Cartão de Encarregado de Educação ou de Atleta físico quando precisa de o apresentar?	Raramente	41%
	Frequentemente	40%
	Não possuo cartão físico	14%
	Nunca	5%
17. Se o Boavista FC modernizasse os seus serviços digitais, em que formato preferiria consultar os seus recibos de pagamento?	Disponíveis na App/Portal para consulta e download	44%
	Recebê-los automaticamente por Email	38%
	Continuo a preferir o papel impresso na secretaria	18%
18. Face ao que respondeu, quais destas 3 áreas considera mais urgentes para melhorar a sua experiência com o clube? (Escolha 3)	Cartão de Encarregado de Educação / Atleta 100% digital no telemóvel	79%
	Area pessoal para consulta autónoma de quotas e recibos	62%
	Receção de notificações oficiais (treinos e exames)	57%
<i>Continua na página seguinte...</i>		

Continuação da tabela anterior

Questão	Opção de Resposta	%
	Atualização de dados e submissão de documentos online	54%
	Área para consulta de dados desportivos (assiduidade, jogos, convocatórias)	48%
19. Numa escala de 1 a 5, qual a urgência geral que atribui à modernização digital dos serviços do Boavista FC?	Nível 5 (Máxima)	34%
	Nível 4	33%
	Nível 3	16%
	Nível 2	17%
	Nível 1 (Mínima)	0%

Tabela II.1.: Resultados quantitativos do inquérito de levantamento de necessidades (258 respostas).

Anexo III. Atas de Reunião e Decisões Técnicas

A operacionalização das sessões conjuntas de alinhamento foi conduzida em ambiente simulado através de um modelo de LLM. Para garantir a fiabilidade empírica e o realismo do debate, sem enviesamento prévio das soluções de *software*, implementou-se uma arquitetura estruturada de *prompting* dividida em duas fases.

Numa primeira etapa de injeção de contexto, o modelo foi instruído a incorporar isoladamente os perfis, as dores operacionais e as limitações de cada *persona*, utilizando como base de conhecimento exclusiva as transcrições das entrevistas individuais.

Numa segunda etapa, induziu-se o cenário de impasse operacional. O modelo foi parametrizado para atuar como mediador num debate orgânico entre as partes, forçando-as a negociar na sua linguagem de negócio (sem jargão técnico) até convergirem numa solução de processo. Este método garantiu que as regras funcionais fossem derivadas de um consenso lógico e não impostas artificialmente, servindo as transcrições geradas como registo de auditoria para a redação formal das Atas apresentadas nas subsecções seguintes.

Template do Prompt 1: Assimilação de Contexto e Personas

Ação Exigida: Leitura e assimilação de contexto. Não geres nenhum texto de resposta além de confirmares que compreendeste as personas e aguardares a minha próxima instrução.

Contexto do Projeto: Estamos a desenhar o novo sistema de informação para o Boavista FC. Eu serei o Analista de Requisitos. Tu vais assumir duas personas distintas com base nas entrevistas individuais que realizámos previamente. Quero que absorvas o tom de voz, as preocupações operacionais e os limites de cada um.

Persona 1: [Cargo do Stakeholder 1]

Foco: [Principais dores e perspetivas extraídas da entrevista].

[Transcrições ou Notas da Entrevista 1]

Persona 2: [Cargo do Stakeholder 2]

Foco: [Principais dores e perspetivas extraídas da entrevista].

[Transcrições ou Notas da Entrevista 2]

Instrução Final: Confirma apenas com "Contexto assimilado. As personas estão prontas. Aguardo instruções para iniciar a reunião."

Template do Prompt 2: Simulação de Debate e Resolução de Conflitos

Ação Exigida: Simulação de uma Reunião de Alinhamento de Processos.

Cenário: Estamos numa sala de reuniões nas instalações do Boavista FC. Estão presentes o [Stakeholder 1], o [Stakeholder 2] e [Membro Omni], membro da equipa OmniSystema (atuando como mediador e analista de processos).

O Conflito a Resolver: [Descrição detalhada da contradição entre as necessidades do Stakeholder 1 e do Stakeholder 2].

A tua Tarefa:

Gera a transcrição direta desta reunião em formato de guião (ex: [Membro] (OmniSystema): ... ; [Stakeholder 1]: ...).

Regras Estritas para a Simulação:

- 1. Linguagem Natural e Zero Jargão Técnico: Os intervenientes NÃO percebem de desenvolvimento de software. Devem debater usando a linguagem da sua profissão.*
- 2. O Debate Dinâmico: Põe os intervenientes a debaterem de forma realista. Expondo as suas frustrações operacionais.*
- 3. A Construção da Solução: A meio da discussão, o membro da OmniSystema deve ouvir ambos os lados e propor uma solução de processo. Deixa-os discutir a viabilidade dessa proposta até concordarem com um fluxo de trabalho seguro e rápido.*

Escreve um diálogo fluído, maduro, com um tom realista, profissional e conciso (cerca de 1 a 2 páginas).

As atas integrais, contendo o detalhe dos conflitos debatidos e as soluções de compromisso acordadas, encontram-se disponíveis para consulta externa através da seguinte ligação: <https://tinyurl.com/32nnvf8b>.

Anexo IV. Catálogo de *User Stories*

Neste anexo encontra-se a referência ao catálogo integral das 41 *User Stories* que compõem o escopo funcional do Sistema Integrado de Gestão Desportiva (SIGD).

A engenharia de *prompting* trancou o *output* à estrutura canónica de escrita (*Como, Quero, Para*) e impôs a aplicação estrita de lógica condicional *Behavior-Driven Development* (*Cenário, Dado que, Quando, Então, E*) na definição dos critérios de aceitação, garantindo a validação direta por arquitetos de *software* e equipas de QA.

O bloco de instrução padronizado que governou esta fase de levantamento e permitiu o mapeamento das histórias estruturadas por *stakeholder* é apresentado detalhadamente abaixo.

Template de Instrução: Extração de User Stories e Critérios de Aceitação

Contexto e Missão:

Atuas como Engenheiro de Requisitos Sénior e Scrum Master. O nosso objetivo é consolidar o escopo de um ERP desportivo (SIGD). Tens de processar as fontes brutas anexadas (transcrições de entrevistas e atas de reuniões) e extrair o catálogo de User Stories.

Fontes de Informação Fornecidas:

[INSERIR TRANSCRICÕES DE ENTREVISTAS E ATAS DE REUNIÃO DE ALINHAMENTO]

Instruções Estritas de Processamento:

- 1. Mapeamento por Stakeholder: Divide e organiza as histórias de forma sequencial com base no nome do profissional ou utilizador entrevistado que originou a necessidade (ex: Miguel Santos).*
- 2. Sintaxe Padrão: Cada história deve seguir obrigatoriamente a estrutura clássica de três linhas, preenchendo as variáveis contextuais: Como [ator do sistema], Quero [ação ou funcionalidade no sistema], Para [benefício real de negócio retirado].*
- 3. Validação Comportamental (Critérios de Aceitação): Para cada história extraída, deves redigir cenários de teste detalhados aplicando de forma estrita o formato BDD (Behavior-Driven Development). É obrigatório o recurso exclusivo aos marcadores sintáticos: Cenário:, Dado que, Quando, Então e a conjunção aditiva E.*
- 4. Abstração de Implementação: Foca-te estritamente na necessidade funcional e na captura do valor operacional, abstendo-te de citar escolhas de engenharia, sistemas de base de dados ou frameworks.*

Formato de Saída Exigido:

O resultado final deve ser apresentado de forma estruturada para indexação tabular direta, sem qualquer texto introdutório ou considerações subjetivas, respeitando o seguinte modelo por bloco:

[Nome do Stakeholder / Utilizador]

ID: US[Número Sequencial]

Título: [Resumo Descritivo da História]

Como: [Perfil]

Quero: [Necessidade]

Para: [Objetivo de Negócio]

Critérios de Aceitação:

Cenário: [Descrição do caso de teste]

Dado que [contexto prévio do sistema]

Quando [ação concreta executada pelo ator]

Então [resposta comportamental ou validação do sistema]

E [efeito colateral ou atualização colateral de estado]

Aplica este padrão de forma exaustiva a todas as dores e fluxos operacionais presentes nos documentos fornecidos.

O catálogo integral destas especificações encontra-se disponível para consulta na seguinte ligação: <https://tinyurl.com/4sfha7zw>.

Anexo V. Catálogo de Requisitos Iniciais

Este anexo documenta a metodologia de Engenharia de *Prompts* adotada para a extração e modelação dos requisitos do sistema. Para garantir a coerência lógica e evitar a perda de contexto inerente à utilização de LLMs, a equipa adotou uma estratégia de Segregação de Domínios. As fontes de dados primárias (*User Stories*, transcrições de entrevistas, atas de alinhamento e estatísticas dos inquéritos) foram agrupadas em macrossegmentos (e.g., Módulo Clínico/Desportivo e Módulo Financeiro/Administrativo).

Para cada um destes domínios, o processo foi executado em duas fases iterativas:

1. **Fase Divergente (Levantamento Macro):** Um *prompt* exploratório desenhado para varrer o contexto fornecido e inferir necessidades, originando um índice bruto de requisitos.
2. **Fase Convergente (Especificação Comportamental):** Um *prompt* restritivo que obrigou o modelo a converter o índice numa especificação técnica estrita, focada na lógica de estado, validações e tratamento de erros.

Abaixo apresentam-se os *templates* de instrução utilizados para orientar esta especificação. O catálogo integral de requisitos gerado, resultado do processo metodológico descrito, encontra-se disponível para consulta através da seguinte ligação: <https://tinyurl.com/47a9y4rm>.

Template do Prompt 1: Extração Macro de Requisitos (Fase Divergente)

Ação Exigida: Mapeamento e Identificação Global de Requisitos de Software. Atuas como Engenheiro de Requisitos Sénior, baseando a tua metodologia estritamente na norma IEEE Std 29148. O nosso objetivo é fazer um levantamento exaustivo de alto nível, identificando os requisitos macro sem desenvolver a sua especificação técnica.

Vou fornecer-te o contexto completo para o Módulo de [ex: Gestão Desportiva e Clínica]. Este contexto inclui: - As User Stories validadas desta área. - As Atas de Reunião referentes a conflitos de negócio. - As Entrevistas dos Stakeholders envolvidos. - Os resultados estatísticos dos Inquéritos.

A tua Tarefa:

Analisa estas fontes cruzadas e identifica TODOS os requisitos de software (explícitos e implícitos) necessários para a operação integral do Módulo.

Regras de Extração e Comportamento:

1. *Listagem Simples: Não geres tabelas. Quero apenas uma lista indexada (ex: REQ-01, REQ-02).*
2. *Nível de Abstração: Não fragmentes requisitos em micro-passos visuais (ex: não cries*

um requisito só para validar NIF).

3. *Necessidades de Engenharia*: Para além das regras de negócio, DEVES deduzir e listar os Requisitos Implícitos de Engenharia e Arquitetura que são obrigatoriamente necessários para o sistema funcionar.

Formato de Saída Exigido:

[REQ-XX] - [Título/Resumo claro do Requisito] (Fontes: ...)

[Contexto Inserido Aqui]

Template do Prompt 2: Especificação Rigorosa e Comportamental (Fase Convergente)

Ação Exigida: Especificação Técnica de Requisitos. Atuas como Engenheiro de Requisitos Sénior. Transforma os Requisitos Macro (identificados na Fase 1) numa especificação rigorosa que sirva de manual para a equipa de desenvolvimento, mantendo abstração total de código.

Diretrizes de Especificação (CRÍTICO):

1. *Foco Comportamental e Lógico*: Quero profundidade nas regras lógicas. Descreve de onde o utilizador parte, que dados exatos introduz (com limites), mensagens de erro e bloqueios. Sem jargão de programação.
2. *Zero Over-Engineering*: Assume que os dispositivos têm conectividade contínua. Não adiciones validações de rede, modos offline ou armazenamento em cache.
3. *Sem Texto Solto*: Não geres parágrafos fora do bloco da tabela.
4. *Relevância para a Aplicação*: Na última secção da tabela, escreve apenas um parágrafo denso. Foca-te estritamente na dependência sistémica e cruza o pedido do stakeholder com os dados do inquérito. Não repitas dores passadas.

Estrutura Exigida para CADA Requisito (Formato Tabular):

ID e Nome: [ID] - [Nome do Requisito]

Área do sistema: [Módulo lógico]

Descrição do Requisito:

1. [Ponto de entrada]
2. [Dados a fornecer]
3. [Validações e Restrições]
4. [Sucesso e Conclusão]

Relevância para a aplicação: [Justificação de negócio + Dado do Inquérito + Impacto Arquitetural].

Fontes de Origem: [Fontes da lista macro].

A tua Tarefa imediata:

Gera as especificações seguindo estritamente este formato para o Lote X.

Anexo VI. Documento SRS

Este anexo documenta as diretivas de instrução utilizadas para guiar a separação entre as regras de negócio e as restrições da infraestrutura, passo fundamental para a estruturação do documento SRS.

A aplicação destas diretivas decorreu numa mecânica de dois passos. Primeiramente, o modelo de linguagem foi alimentado com o catálogo de requisitos brutos e instruído a devolver um plano de ação detalhado, identificando com precisão cirúrgica os excertos de texto (métricas temporais, limites de ficheiros e regras de segurança) que deveriam ser podados. Posteriormente, com base nesse relatório de auditoria, a equipa executou as alterações, limpando os Requisitos Funcionais e originando a nova secção independente de Requisitos Não-Funcionais (RNF) que compõe a versão final do SRS.

Template de Instrução: Auditoria de Segregação e Planeamento de RNF

Contexto e Missão:

Atuas como um Arquiteto de Software Sénior e Especialista em Engenharia de Requisitos (baseado em Ian Sommerville e na norma IEEE 830). A nossa equipa desenvolveu um catálogo de requisitos para um Sistema Integrado de Gestão Desportiva.

Para cumprir a norma IEEE 830 sem cair em "Over-Engineering" documental (estilhaçar regras de negócio em dezenas de micro-IDs rastreáveis), decidimos adotar uma abordagem de Arquitetura Limpa:

Vamos manter os Requisitos Funcionais (RF) focados estritamente no "Quê".

Vamos podar/extrair o "Como/Quão Bem"(SLAs, tempos de resposta, métricas de hardware) de dentro desses requisitos.

Vamos mover essa "gordura não-funcional" para uma nova secção dedicada de Requisitos Não-Funcionais (RNF), tratados como "Grandes Leis Sistémicas" que regem a plataforma como um todo.

Em anexo encontra-se a nossa lista atual de requisitos.

A Tua Tarefa:

Realizar uma análise profunda ao nosso catálogo e estruturar um plano de ação detalhado, respondendo aos seguintes 4 pontos:

1. Análise de Classificação:

Avalia a lista e classifica os requisitos atuais. Identifica se existem requisitos que são puramente RNF (e que devem saltar inteiros para a nova secção), os que são puramente RF (intocáveis), e os que são mistos (que precisam de ser limpos).

2. Proposta de Extração:

Lista explicitamente que trechos devem ser removidos dos requisitos mistos atuais.

Sugestões da nossa equipa (avalia e expande esta lista consoante a tua análise): Remover métricas temporais (ex: "1,5 segundos", "30 segundos", "60 segundos"), limites de ficheiros ("5MB") e regras de firewall/bloqueio ("5 tentativas", "15 minutos").

3. Criação da Secção RNF:

Com base no que foi extraído no ponto anterior, propõe a nova secção de RNF Sistémicos agrupados pelas categorias da IEEE 830 (Desempenho, Segurança, Usabilidade, Fiabilidade, Restrições de Design).

Nota Importante: Para além da gordura extraída, identifica e propõe RNF cruciais que nos possam ter escapado e que são vitais.

Sugestões da nossa equipa (avalia e decide): Uptime do sistema, políticas de backup da BD, conformidade explícita com RGPD (encriptação em repouso), e a imposição da nossa stack (React Web, Spring Boot, MySQL, On-Premise).

Formato de Resposta Exigido:

Usa um tom técnico, pragmático e focado na ação. Apresenta o teu output bem estruturado (bullet points e tabelas, se necessário) para que possamos simplesmente aplicar as tuas decisões ao nosso projeto.

O documento final resultante desta metodologia, o *Software Requirements Specification (SRS)*, encontra-se disponível para consulta integral através da seguinte ligação: <https://tinyurl.com/2hy8ptyj>.

Anexo VII. Catálogo de Casos de Uso

Este anexo documenta a abordagem metodológica para a transição entre o levantamento de Requisitos Funcionais (RF) e a primeira modelação comportamental do sistema, consubstanciada no mapeamento de Casos de Uso (UC).

Para mitigar a arbitrariedade na granularidade dos artefactos e garantir o estrito alinhamento com os objetivos de negócio dos diferentes departamentos do clube, o modelo de linguagem foi parametrizado com um perfil de analista de negócio sénior. A instrução fundacional obrigou a ferramenta a agrupar logicamente as interações em estruturas macro denominadas "Famílias de Casos de Uso", normalizando a nomenclatura através de verbos no infinitivo e garantindo a rastreabilidade total face aos requisitos.

Abaixo apresenta-se o *template* de instrução estruturado que norteou este processo de extração iterativa.

Template de Instrução: Extração de Casos de Uso por Domínio Lógico

Contexto e Missão:

Atuas como Senior Business Analyst e Arquiteto de Software. A tua tarefa é analisar o catálogo de Requisitos Funcionais (RFs) fornecido e extrair a lista inicial e exaustiva de Casos de Uso (UCs) para o projeto SIGD (Sistema Integrado de Gestão Desportiva).

Contexto do Sistema:

O SIGD é um ERP estruturado em blocos funcionais autónomos (Secretaria/Tesouraria, Clínica Médica, Direção Técnica e Portal B2C).

Instruções Estritas de Extração e Agrupamento:

- 1. Mapeamento de Interações: Lê todos os Requisitos Funcionais (RFs) fornecidos e identifica as interações diretas e completas entre os Atores e o Sistema.*
- 2. Estruturação em Duplo Nível: Agrupa obrigatoriamente os Casos de Uso relacionados em "Famílias" lógicas sequenciais (ex: Família UC-XX: Configuração da Estrutura Desportiva, Família UC-YY: Gestão do Calendário Desportivo).*
- 3. Abstração de Negócio: Cada UC deve representar um objetivo fim alcançável por um ator primário que entregue valor mensurável. Evita granularidade extrema (não crie UCs para ações mecânicas isoladas como cliques ou validações de campo simples).*
- 4. Normalização Gramatical: O nome de cada Caso de Uso tem de começar obrigatoriamente por um verbo no infinitivo que dite a ação (ex: Registrar perfil, Planear microciclo, Submeter ficha).*

Formato de Saída Exigido:

O output deve ser estruturado exclusivamente na seguinte hierarquia textual, sem recurso a matrizes tabulares:

Módulo [Letra] - [Nome do Módulo]

*Família UC-[XX]: [Nome da Família de Coerência Comportamental]
- UC-[XX.X] - [Nome do Caso de Uso no Infinitivo]: [Descrição sucinta e explícita do fluxo, contendo o impacto ou o ator com quem interage, limitada ao máximo de duas linhas].*

Garante que não ficam requisitos funcionais críticos sem cobertura de Casos de Uso e preserva a integridade do escopo do sistema.

—
DADOS DE ENTRADA:

O catálogo completo resultante da execução desta diretiva, mapeando as dezasseis famílias funcionais que governam a plataforma, encontra-se disponível para consulta na seguinte ligação: <https://tinyurl.com/52fbmkjd>.

Uma vez consolidada a listagem das dezasseis famílias funcionais e dos respetivos escopos, a equipa avançou para a fase de especificação detalhada de cada fluxo operacional. Os Casos de Uso foram submetidos e detalhados individualmente, caso a caso.

Esta modelação comportamental minuciosa foi regida pelo *template* de instrução apresentado a seguir. O caderno de encargos final, contendo a totalidade dos Casos de Uso especificados de forma atômica por este método, encontra-se disponível para consulta em: <https://tinyurl.com/mtemkdjk>.

Template de Instrução: Especificação Atômica de Casos de Uso

Atua como Analista de Sistemas Sénior. A tua tarefa é especificar um Caso de Uso para o SIGD (Sistema Integrado de Gestão Desportiva), um ERP web/mobile que centraliza tesouraria, relvado, clínica e portal de sócios.

Os atores do sistema são: Secretaria, Treinador, Médico, EE/Atleta, Direção Técnico, CFO e Admin.

O Caso de Uso a especificar é: [INSERIR ID E NOME — ex.: UC-09.1: Registrar assiduidade em sessão de treino]

Objetivo: [INSERIR OBJETIVO DO USE CASE]

Estrutura Obrigatória:

Use Case: [ID e Nome — sempre verbo no infinitivo]

Descrição: [1-2 frases com o objetivo funcional. Zero tecnologias ou UI.]

Atores: [Primário e Secundários, se aplicável]

Pré-condição: [Condição que tem de ser verdade ANTES do UC começar. Uma por linha.]

Pós-condição (Sucesso): [Estado do sistema após sucesso. Uma por linha.]

Fluxo Normal:

1. [Ator manifesta intenção/inicia interação].
2. [Sistema executa validação A].
3. [Sistema executa validação B — passo separado se for independente].
4. [Ator toma decisão/submete dados].
5. [Sistema persiste resultado e confirma sucesso].

Fluxos Alternativos:

[Nome do cenário]

Xa. [Desvio válido a partir do passo X].

Xb. [Reação do sistema/ator].

Xc. O fluxo retoma no passo [Y].

Fluxos de Exceção:

[Nome da falha]

Xa. [Sistema deteta erro/violação a partir do passo X].

Xb. [Sistema notifica o ator e cancela operação].

Xc. O Caso de Uso encerra.

Regras Absolutas de Escrita (Violação implica rejeição):

1. Numeração por Ramificação: Os desvios herdam o número do passo onde quebram. Se falha no passo 3, os sub-passos são 3a, 3b, 3c. É proibido usar categorias como FA-1 ou FE-1. Múltiplas falhas no mesmo passo começam todas em Xa.
2. Fecho Obrigatório: Nenhuma ramificação fica pendurada. Alternativos acabam sempre com "O fluxo retoma no passo [X]". Exceções acabam sempre com "O Caso de Uso encerra".
3. Validações Atômicas: Não agrupe validações do sistema no mesmo passo. Validar o login é um passo (ex: 2); verificar a elegibilidade do atleta é outro (ex: 3).
4. Abstração de Interface (Zero UI): Foca-te na intenção. Proibido referir botões, cliques ou menus dropdown. Escreve "O utilizador submete os dados", não "O utilizador clica no botão verde".
5. Abstração Técnica: Sem jargões de base de dados ou arquitetura (SGBD, ACID, roll-backs, timeouts). Se algo falha, o sistema "cancela a operação" ou "descarta a previsão", de forma compreensível para o negócio.
6. Alternativo vs. Exceção: Alternativo chega ao sucesso (pós-condição) por outro caminho. Exceção indica que a operação falha, não guarda dados e avisa o utilizador.
7. Escopo Estrito: Não inventes regras de negócio fora do catálogo de requisitos do SIGD.

Exemplo Rápido de Aplicação:

Fluxo Normal:

1. O treinador solicita acesso à avaliação.
2. O sistema valida a sessão.
3. O sistema apresenta a lista de atletas presentes.
4. O treinador submete as avaliações.
5. O sistema persiste os dados.

Fluxos Alternativos:

[Atleta em branco]

4a. O treinador submete a avaliação deixando um atleta em branco.

4b. O sistema regista esse atleta como "Não Avaliado".

4c. O fluxo retoma no passo 5.

Fluxos de Exceção:

[Sessão expirada]

2a. O sistema detecta que a sessão fechou há mais de 24h.

2b. O sistema avisa que o prazo expirou e revoga acesso.

2c. O Caso de Uso encerra.

Gera agora o Caso de Uso solicitado respeitando todos os pontos acima.

Anexo VIII. Modelo de Domínio Conceptual

A modelação da ontologia e das regras de negócio do Sistema Integrado de Gestão Desportiva (SIGD) foi conduzida através de uma abordagem puramente conceptual, abstendo-se de restrições físicas de bases de dados ou viés de implementação de código.

O objetivo do *prompt* foi forçar a ferramenta a agir como um projetista de software sénior, aplicando uma estratégia radial para atributos biográficos e financeiros. Ao isolar cada propriedade como uma caixa satélite exclusiva e ancorá-la rigidamente com direções ortogonais e links de ordenação ocultos, eliminou-se o cruzamento caótico de linhas e garantiu-se uma linguagem estritamente focada no negócio.

Abaixo apresenta-se o bloco de instrução unificado que governou a geração da arquitetura de domínio do sistema.

Template de Instrução: Engenharia de Domínio Conceptual Puro

Atuas como um Arquiteto de Software Sénior especialista em UML. O objetivo é gerar o código PlantUML para o Modelo de Domínio Conceptual Puro do SIGD (ERP Desportivo). Este modelo é uma abstração pura do negócio, sem qualquer ligação a tabelas físicas ou classes de programação.

Para garantir a clareza do diagrama e evitar linhas cruzadas, segue estas regras:

- 1. Classes Limpas: Não declares propriedades ou atributos dentro das classes (não uses chavetas). Cada entidade deve ser apenas uma caixa simples com o seu nome.*
- 2. Atributos Satélites: Transforma dados (como NIF, Morada, Datas ou Valores) em caixas externas exclusivas. Conecta-as radialmente à entidade principal usando direções curtas (-up->, -down->, -left->, -right->) para que orbitem coladas ao bloco pai. Separa conceitos equivalentes (ex: "Valor da Obrigacao" e "Valor do Pagamento").*
- 3. Hierarquias Organizadas: Usa herança (<|--) a partir da classe abstrata Utilizador para os 9 papéis do clube. Arruma-os numa coluna vertical à esquerda através de links ocultos (ex: Admin -[hidden]down-> Medico). Aplica a mesma lógica para EventoDesportivo (SessaoTreino/JogoOficial) e Pagamento (MBWay/Multibanco/Dinheiro).*
- 4. Relações Ativas: Proibido usar verbos vagos como "Tem" ou "Possui". Usa termos descritivos com sentido de leitura e cardinalidades explícitas (ex: Atleta -right-> DocumentoOficial : possui registado >).*
- 5. Layout Ortogonal: Injeta no topo as diretivas "skinparam linetype ortho" e "left to right direction". Agrupa as linhas de associação perto das entidades que os atores manipulam para manter as setas curtas.*

Mapeia todo o escopo do ERP: Estrutura Desportiva, Operações de Relvado (Assiduidade, Avaliacao, Convocatoria, FichaJogo, JustificacaoAusencia), Clínica (ExameMedicoDesportivo, OcorrenciaClinica, EvolucaoClinica, EstadoDocumental, EstadoElegibilidade, DocumentoOficial), Tesouraria (ObrigacaoFinanceira, FaturaRecibo, CentroResponsabilidade, ContaCorrente) e Segurança (Sessao, Registo de Atividades).

Devolve apenas o bloco de código contido entre @startuml e @enduml.

O código textual completo resultante desta engenharia de instrução (com extensão *.puml*), bem como o respetivo diagrama, encontram-se disponíveis na seguinte pasta: <https://tinyurl.com/3aj5twp9>.

Anexo IX. Prototipagem Assistida

O processo de desenho e especificação de interfaces para os diferentes módulos do ecossistema foi conduzido de forma estritamente mecânica através de uma arquitetura de engenharia de *prompting* dividida em duas etapas consecutivas. Esta abordagem garantiu o isolamento lógico das regras de negócio antes de qualquer conversão gráfica, forçando a ferramenta a agir como um validador de requisitos antes de atuar como projetista visual.

A primeira fase, designada por Fase de Alinhamento Funcional, consistiu na injeção cruzada dos casos de uso, requisitos e regras de validação previamente extraídos. O *template* de instrução apresentado de seguida foi utilizado para forçar o modelo a analisar criticamente as regras sistémicas e a estruturar a arquitetura lógica do ecrã, mapeando os fluxos e adaptando os componentes visuais consoante o paradigma de visualização exigido (*Desktop-First* ou *Mobile-First*).

Template: Blueprint Funcional e Arquitetura de UI

Atuas como um Arquiteto de Software Sénior, Product Owner e Especialista em UX/UI. O nosso objetivo é construir um ERP desportivo de excelência, pronto para o mundo real.

Nesta fase, estamos a iniciar o desenho do módulo para um novo ator do sistema. Vou fornecer-te os seguintes blocos de informação:

- 1. O NOME DO ROLE/ATOR em análise: [NOME DO ROLE / ATOR]*
- 2. O PARADIGMA DE INTERFACE EXIGIDO: [PARADIGMA: MOBILE-FIRST OU DESKTOP-FIRST]*
- 3. A LISTA DE REQUISITOS (RF/RNF) e CASOS DE USO (UCs) globais do sistema.*

ATENÇÃO CRÍTICA AO PARADIGMA DE NAVEGAÇÃO:

Se o paradigma for Mobile-First, a arquitetura de UI/UX tem de ser otimizada estritamente para ecrãs de telemóvel (toque com o polegar, leitura rápida, recurso a listas de "Cartões Empilhados" em vez de tabelas complexas, "Bottom Sheets" em vez de modais centrais, e navegação via "Bottom Navigation Bar" com o máximo de 4 ou 5 itens).

Se o paradigma for Desktop-First, a arquitetura deve focar-se na densidade de dados e produtividade de backoffice (recurso a "Data Tables" robustas com paginação e filtros múltiplos, menus laterais expansíveis, gráficos analíticos complexos e modais centrais de confirmação).

A tua tarefa é cruzar estas informações, analisar criticamente o documento inicial (caso tenha sido fornecido) e gerar um "Blueprint Funcional e Arquitetura de UI" detalhado. Deves ditar exatamente o que este módulo tem de conter para cumprir os requisitos, seguindo rigorosamente estas diretrizes:

1. Análise de Requisitos e Escopo:

- Analisa a documentação fornecida e identifica todas as responsabilidades, permissões e necessidades de visualização deste Role específico.*

2. Mapeamento de Funcionalidades (O que tem de existir):

- Lista as funcionalidades, botões, métricas, dados ou fluxos que são EXIGIDOS nos RFs/UCs para este ator.
- Traduz esses requisitos em elements visuais de UI específicos e adequados ao paradigma definido (Mobile ou Desktop).

3. Fronteiras de Atores e Permissões (Separation of Duties):

- Tem sempre em consideração os outros atores do sistema e define claramente os limites do módulo: o que é que este ator pode apenas VER (read-only) e o que é que pode EFETIVAMENTE EDITAR ou SUBMETER, garantindo que o perfil respeita a Matriz de Controlo de Acessos (RBAC) e não usurpa funções transacionais de terceiros.

4. Estrutura Lógica Sugerida (Navegação):

- Propõe uma organization lógica para o ecrã. Define o sistema de menus apropriado e mapeia que informações entram em cada aba, secção ou painel estrutural, servindo de guião estrito para o desenho de interface.

FORMATO DE SAÍDA OBRIGATÓRIO (Gera um Relatório em Markdown estruturado assim):

Blueprint de Arquitetura ([Inserir Paradigma Mobile/Desktop]): Módulo [Nome do Role]

1. Visão Global e Propósito do Módulo

(Resumo do objetivo principal deste ecrã para este ator específico).

2. Funcionalidades Obrigatórias (Mapeamento RF/UC)

(Lista detalhada do que o ecrã tem de ter obrigatoriamente, citando o RF/UC correspondente).

3. Fronteiras de Atores (Permissões e Read-Only)

(Clarificação estrita do que este ator NÃO faz para não entrar em conflito com o fluxo do sistema).

5. Esqueleto Estrutural Proposto (Guião para UI)

(A tua proposta de divisão do ecrã, detalhando abas, KPIs, tabelas ou listas de cartões, e o comportamento dos modais/bottom sheets necessários).

—

DADOS DE ENTRADA:

Uma vez obtido o consenso técnico sobre a organização estrutural do ecrã e as restrições operacionais de cada perfil, o fluxo avançou para a segunda etapa, denominada Fase de Especificação de Alta Fidelidade.

Nesta fase, recorreu-se ao *template* restritivo detalhado a seguir. O objetivo principal desta instrução consistiu em pegar no mapeamento conceptual da fase anterior e convertê-lo numa especificação de interface puramente textual, recorrendo ao conceito de "blindagem visual". Ao detalhar exaustivamente os estados dos componentes, as cores exatas de alertas e o comportamento de elementos móveis ou de *backoffice* à luz do *Design System* corporativo, bloqueou-se

a capacidade de alucinação ou inventividade gráfica das ferramentas de IA geradoras de código subsequentes (como o Figma).

Template: Especificação de Alta Fidelidade Visual e Blindagem de UI

Atuas como um Arquiteto de Software Sênior, Product Owner e Especialista em UX/UI. Tu já conheces o nosso projeto do ERP Desportivo e a nossa metodologia rigorosa de UI. Acabámos de focar a nossa atenção no Módulo: [NOME DO ROLE / ATOR].

ATENÇÃO CRÍTICA AO PARADIGMA DE INTERFACE:

Este módulo segue o paradigma [DEFINIR PARADIGMA: ESTRITAMENTE MOBILE-FIRST OU ESTRITAMENTE DESKTOP-FIRST]. Toda a arquitetura de informação deve responder de forma nativa e cega a esta restrição de tamanho, disposição e interação do ecrã.

Agora estamos na fase final de passagem para o Design/Desenvolvimento (via Figma). O problema destas ferramentas de IA geradoras de UI é que elas inventam coisas, misturam componentes de plataformas erradas ou deixam detalhes de fora se não estiverem explicitamente escritos. Para travar isso, criei um "Prompt de Regras" para a IA, obrigando-a a seguir um ficheiro .md de forma literal e cega.

A TUA TAREFA:

Vou fornecer-te dois elementos: O "Prompt do Figma" (para entenderes o rigor e o layout exigidos), o nosso DESIGN.md atualizado com as regras gerais do sistema, e o Blueprint Funcional que acabaste de gerar. Quero que transformes o Blueprint num ficheiro .md de Excelência Absoluta e Alta Fidelidade Visual ([NOME_DO_MODULO].md).

Diretrizes para a tua escrita:

1. Aplicar a Arquitetura do Paradigma:

Traduz a lógica do Blueprint para os componentes corretos da plataforma de destino. Se for Mobile-First, transforma listas de dados em "Cartões Empilhados", menus em "Bottom Navigation Bar" e formulários em "Bottom Sheets" que deslizam de baixo. Se for Desktop-First, estrutura em "Data Tables" densas com paginação superior, menus laterais "Sidebar" e painéis divididos em grelha. NUNCA quebres as regras do paradigma escolhido.

2. Exaustão Visual (O mais importante):

Define as cores exatas para os eventos, as badges de estado (ex: Sucesso/Pago = Verde, Alerta/Dívida = Vermelho), e marcadores visuais de prioridade. Descreve as dimensões relativas dos botões de ação, o comportamento quando ocupam a largura total (Full-width) e as áreas de toque ou clique. Não deixes NADA à imaginação do gerador de UI.

3. Fidelidade Literal:

O resultado do teu .md tem de ser exatamente o que vai aparecer no ecrã final. Sem tirar nem pôr. Define os textos exatos que devem ser exibidos, a anatomia e ordem de cada componente, os contadores e os rótulos de dados.

4. Consistência Transversal:

Respeita as secções de estilo e identidade visual descritas no documento DESIGN.md fornecido. A interface tem de parecer parte integrante e natural do ecossistema visual global do ERP.

FORMATO DE SAÍDA OBRIGATÓRIO:

- 1. Um breve resumo (em bullet points) das tuas principais decisões de UI aplicadas a este módulo com base no paradigma definido.*
- 2. O bloco de código final em formato Markdown com a especificação técnica visual totalmente finalizada e blindada para a IA desenhar.*

DADOS DE ENTRADA:

As especificações visuais finais resultantes da aplicação sequencial desta metodologia aos diferentes módulos, juntamente com os ecrãs interativos resultantes da interpretação literal destes documentos pela ferramenta de *design*, encontram-se disponíveis para consulta externa na seguinte ligação: <https://tinyurl.com/yc73vvt8>.